

ADVENTIST UNIVERSITY OF CENTRAL AFRICA

Advanced Big Data Analytics

Assignment I

Smart Energy Grid Analytics

Ingestion, storage benchmarking, analysis, continuous aggregates, dashboards and forecasting for 4,000 smart meters with PostgreSQL, TimescaleDB, EMQX and Python

Group members

No.	Full name	Student ID
1	Amie Marie Flora Dushimumukiza	101405
2	Kwizera Rene	101379
3	Grace Teta	101378

Lecturer: Temitope Oguntade

Table of Contents

1 Build the ingestion pipeline 4

1.1 Confirm the environment 4

1.2 Create the database structures 8

1.3 Capture and publish a pilot stream 12

2 Capture and prepare the historical dataset 17

2.1 Capture and load the complete stream 17

2.2 Investigate data quality 20

2.3 Clean and preserve a consistent benchmark 23

3 Compare chunk intervals 26

3.1 Create and verify three configurations 26

3.2 Implement and time the five workloads 28

3.3 Explain the chunk results 31

4 Compare compression strategies 32

4.1 Create the two compressed configurations 32

4.2 Measure storage and compression cost 35

4.3 Repeat the query comparison 37

4.4 Select a storage configuration 39

5 Analyse demand, regions, reporting quality and abnormal consumption 40

5.1 Demand patterns over time 40

5.2 Regions and customer types 43

5.3 Reporting quality and late data 46

5.4 Detecting abnormal consumption 48

5.5 Recommendation to management 50

6 Continuous aggregates and data freshness 52

6.1 Create the continuous aggregates 52

6.2 Test data freshness 55

7 Dashboards 59

7.1 Raw dashboard 59

7.2 Aggregate dashboard 62

7.3 Check that the two dashboards agree 64

7.4 Compare refresh performance 65

8 Forecasting hourly demand 68

8.1 Dataset, features and split 68

8.2 Baseline: previous day, same hour 69

8.3 Ridge regression and model selection 70

8.4 Is the forecast useful? 71

9 Submit the report and code 75
9.1 Organise the report 75
9.2 Make the work reproducible 75
9.3 Document contributions 76

1 Build the ingestion pipeline

1.1 Confirm the environment

All work was carried out on one laptop running **Windows 11** (version 10.0.26200). The database server runs in a **Docker** container from the image `timescale/timescaledb:latest-pg16`, which contains **PostgreSQL 16.15** and **TimescaleDB 2.30.2** (the assignment requires 2.10 or later with a compatible PostgreSQL version). **EMQX** runs in a second container (`emqx/emqx:latest`) that publishes port 1883 for MQTT and 18083 for its dashboard. Both containers are started together with `docker compose up -d`. Because a native PostgreSQL 18 installation on the laptop already uses port 5432, the TimescaleDB container is published on host port **5433**, and every program connects to `127.0.0.1:5433`.

One database, **smartgrid**, holds every table and view of the assignment. It was created with `CREATE DATABASE smartgrid`, TimescaleDB was enabled with `CREATE EXTENSION timescaledb`, and `ALTER DATABASE smartgrid SET timezone TO 'Africa/Kigali'` makes every session use CAT (UTC+02:00). Daily buckets are aligned to CAT midnight by giving the time zone to `time_bucket(INTERVAL '1 day', event_time, 'Africa/Kigali')`; without the third argument a day would start at 02:00 CAT (midnight UTC) and every daily total in the report would be cut at the wrong place.

Recorded environment

Item	Value
CPU model	Intel(R) Core(TM) i5-8250U CPU @ 1.60GHz
Cores	4 physical, 8 logical
RAM	15.9 GB (about 16 GB)
Operating system	Microsoft Windows 11, version 10.0.26200
Storage type	SSD
Python	3.12.4
EMQX	6.3.1 (EMQX Enterprise edition), Docker container emqx/emqx:latest, ports 1883 and 18083
PostgreSQL	16.15, x86_64 Alpine Linux (musl) build, running in a Docker container
TimescaleDB	2.30.2 (required: 2.10 or later)
shared_buffers	1969MB
work_mem	7878kB
max_parallel_workers_per_gather	4
jit	off
Where PostgreSQL runs	In a Docker container 'timescaledb' (timescale/timescaledb:latest-pg16) in Docker Desktop, host port 5433. No CPU or RAM limits set on the container (CPUs=0, Mem=0).
Docker Desktop VM memory limit	CPUs=8 , TotalMemoryBytes=8260354048

Clock and execution times. The Windows time zone is CAT (UTC+02:00) and the clock is synchronised with the Windows Time service. Every screenshot in this report shows the taskbar clock, so the CAT date and time of each command is visible next to its result.

Broker. `docker ps` lists the `emqx` container as up with ports 1883 and 18083 published, next to the `timescaledb` container with port 5433 published. The EMQX dashboard confirms the broker is running.

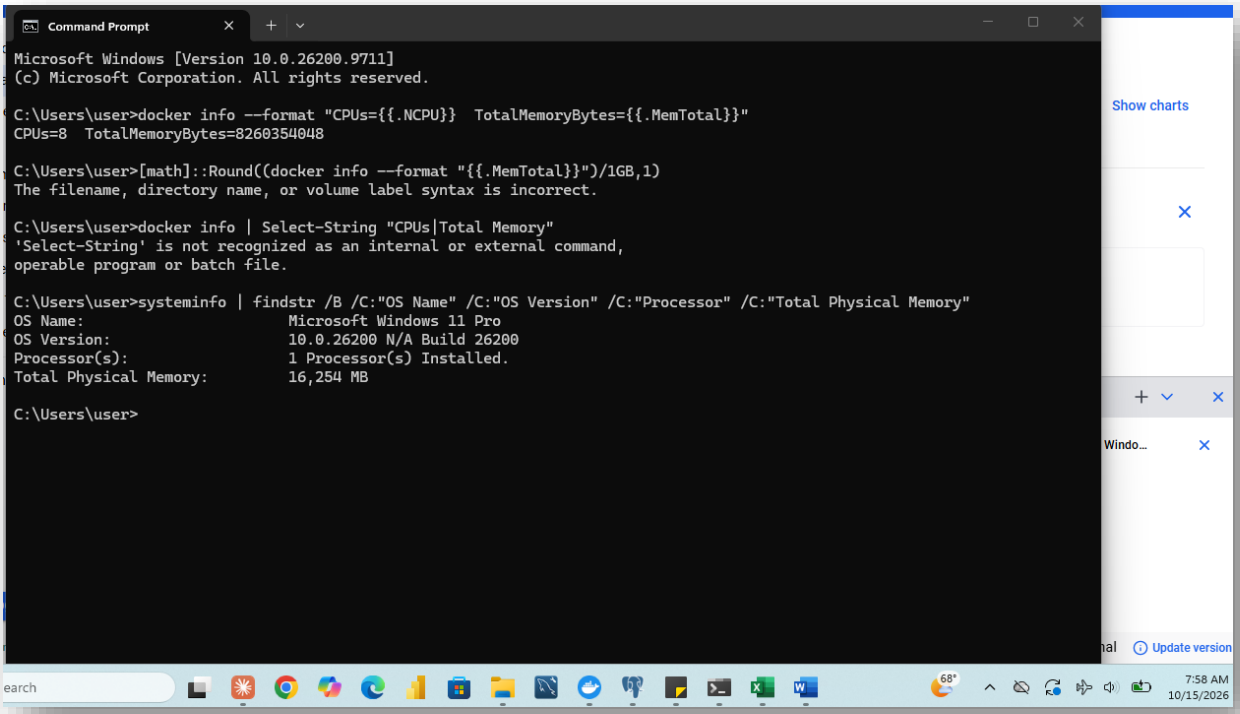


Figure 1.1-1: *CPU model, core count, storage and RAM*

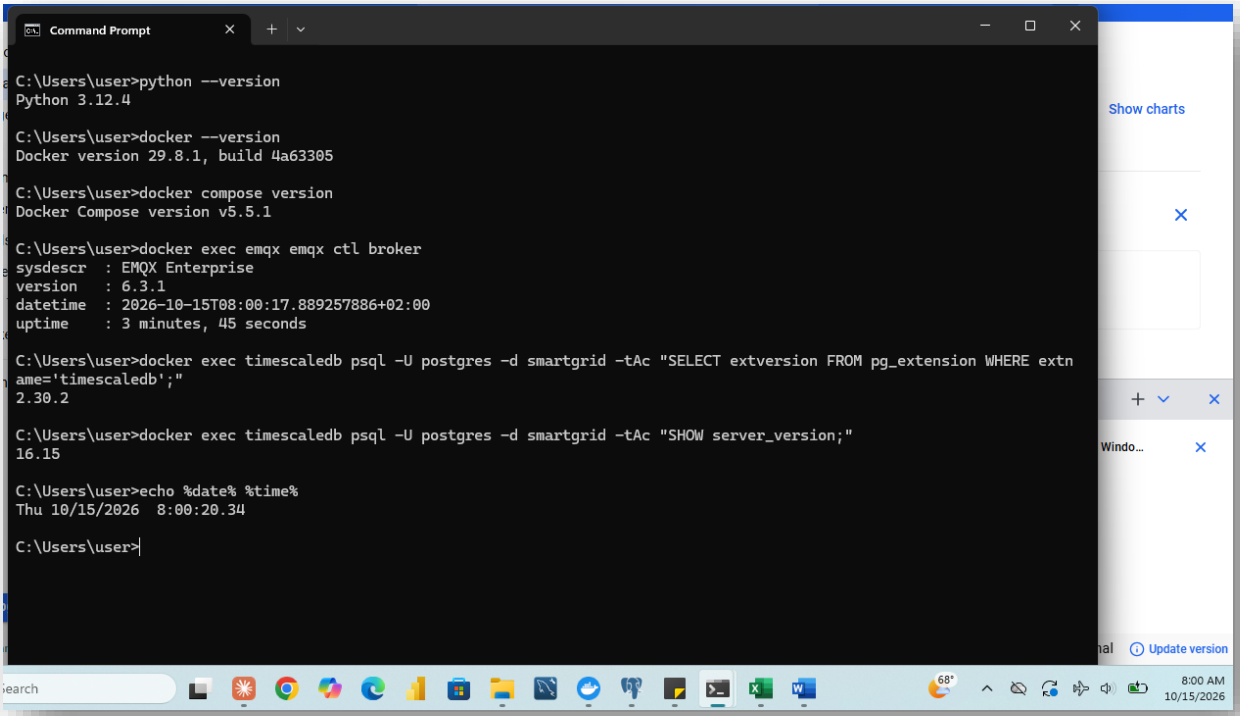


Figure 1.1-2: *Installed software versions (Python, Docker and related tools)*

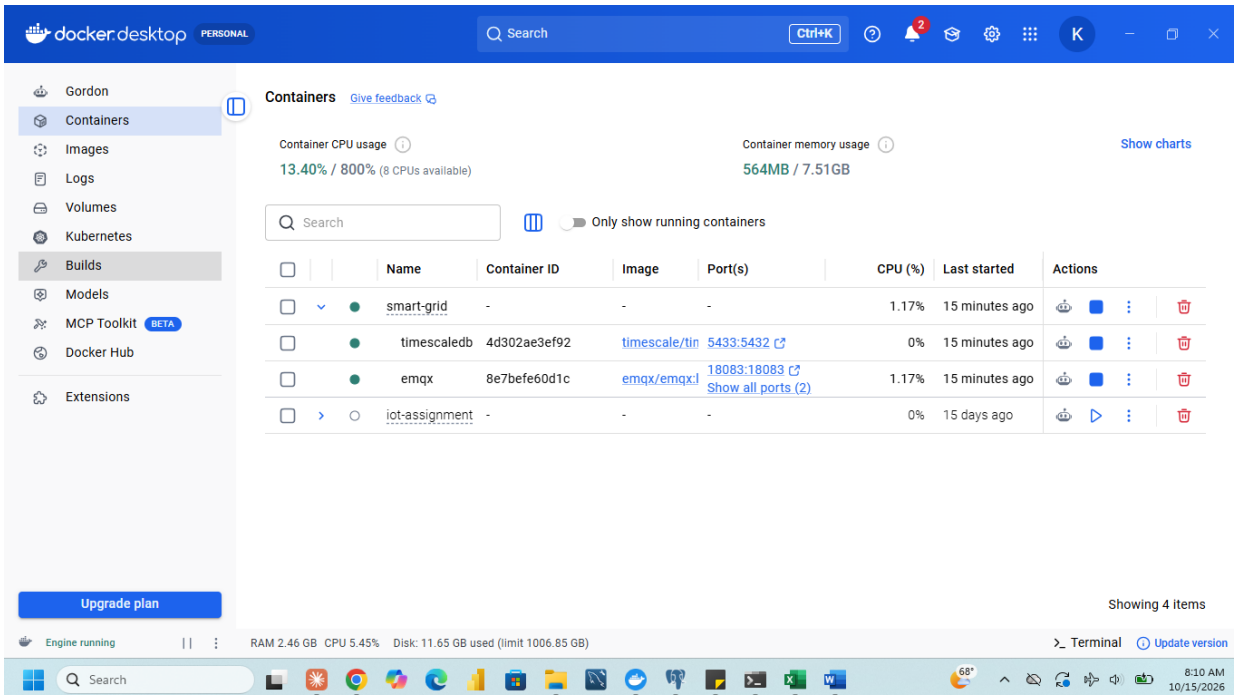


Figure 1.1-3: *Docker containers running: TimescaleDB on port 5433 and EMQX on ports 1883 and 18083*

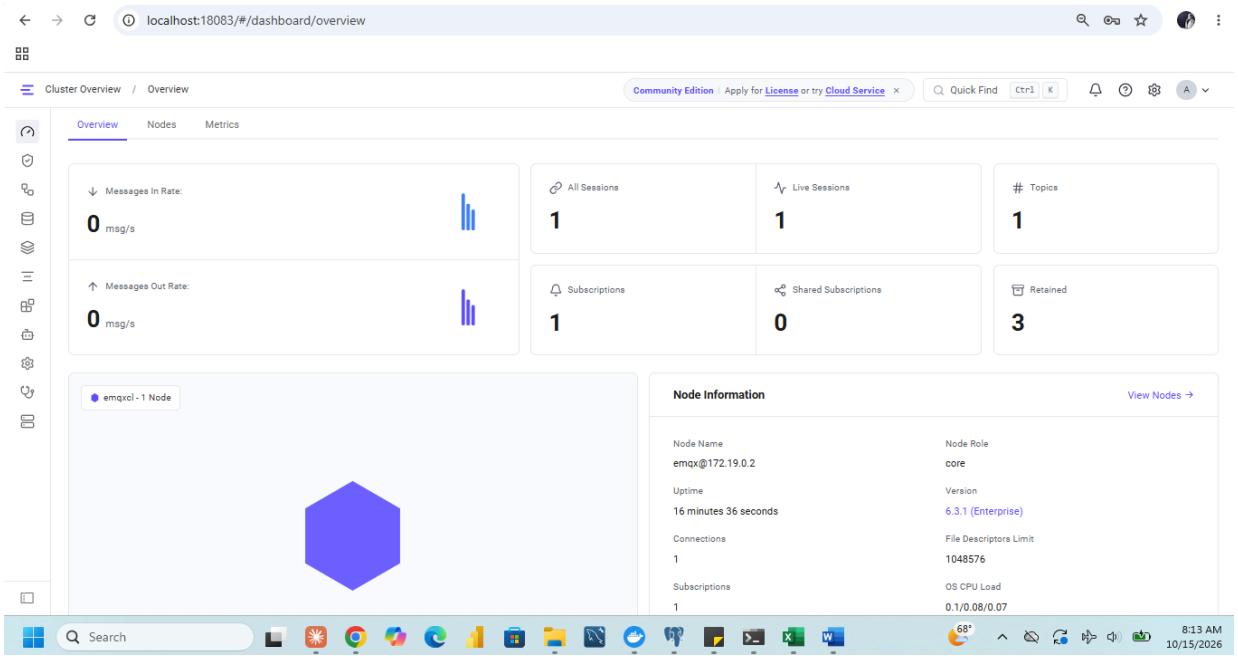


Figure 1.1-4: *EMQX dashboard overview: broker running*

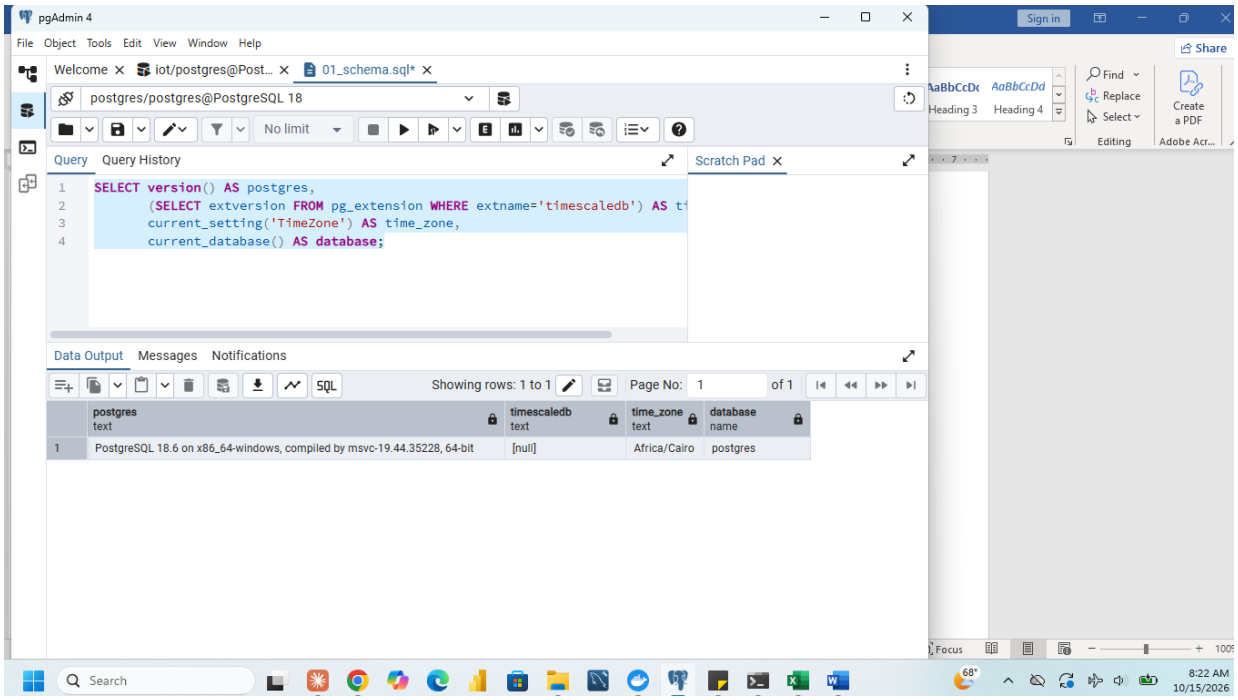


Figure 1.1-5: *Connection to the smartgrid database: PostgreSQL 16.15, TimescaleDB 2.30.2 and session time zone Africa/Kigali*

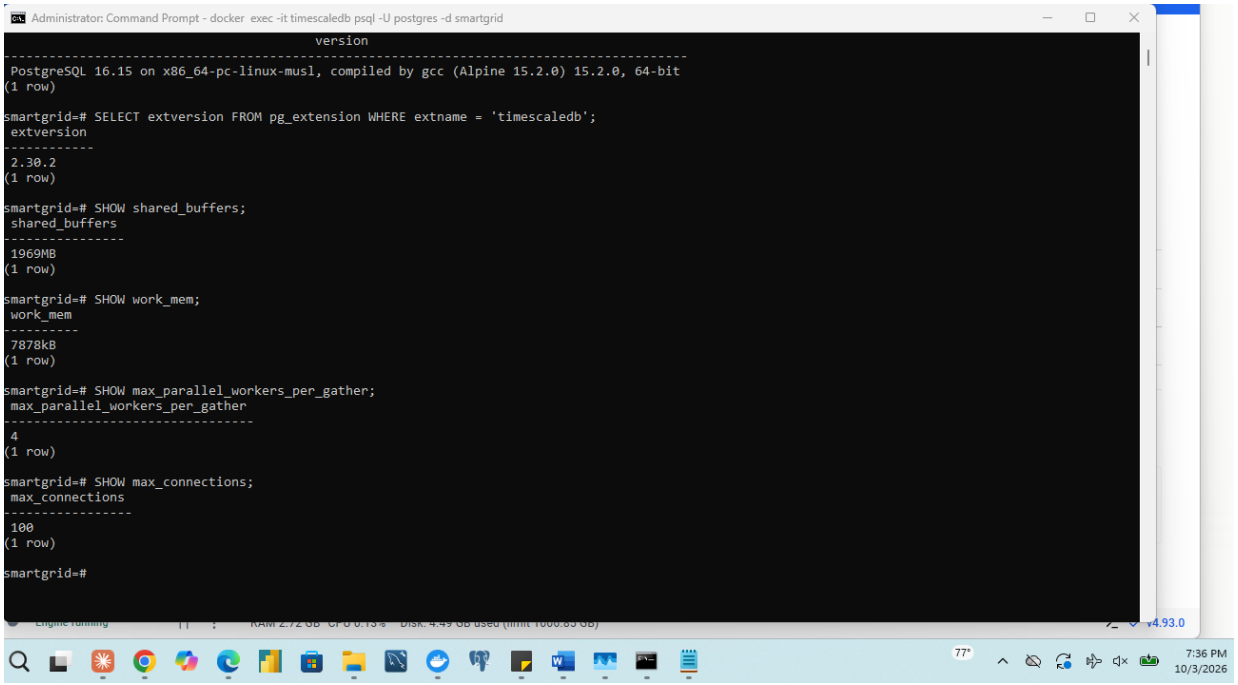


Figure 1.1-6: *Key settings: shared_buffers, work_mem and max_parallel_workers_per_gather (and jit)*

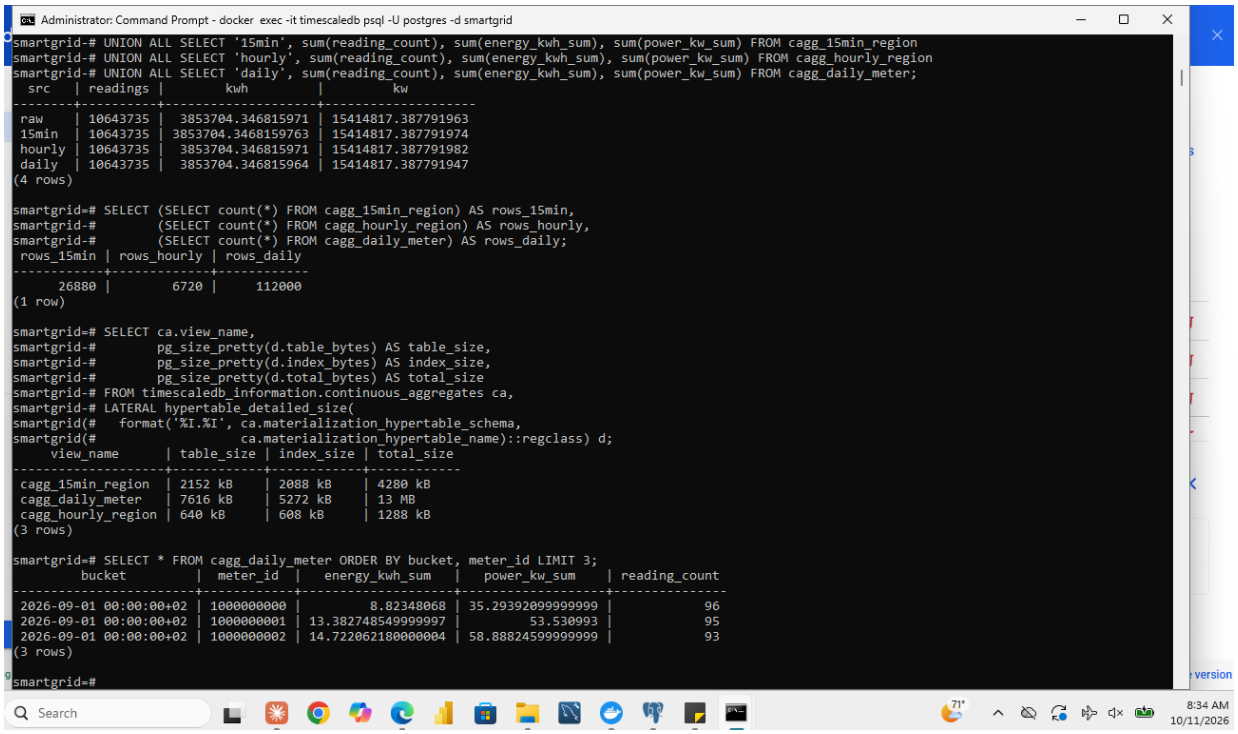


Figure 1.1-7: *A daily bucket aligned to CAT midnight*

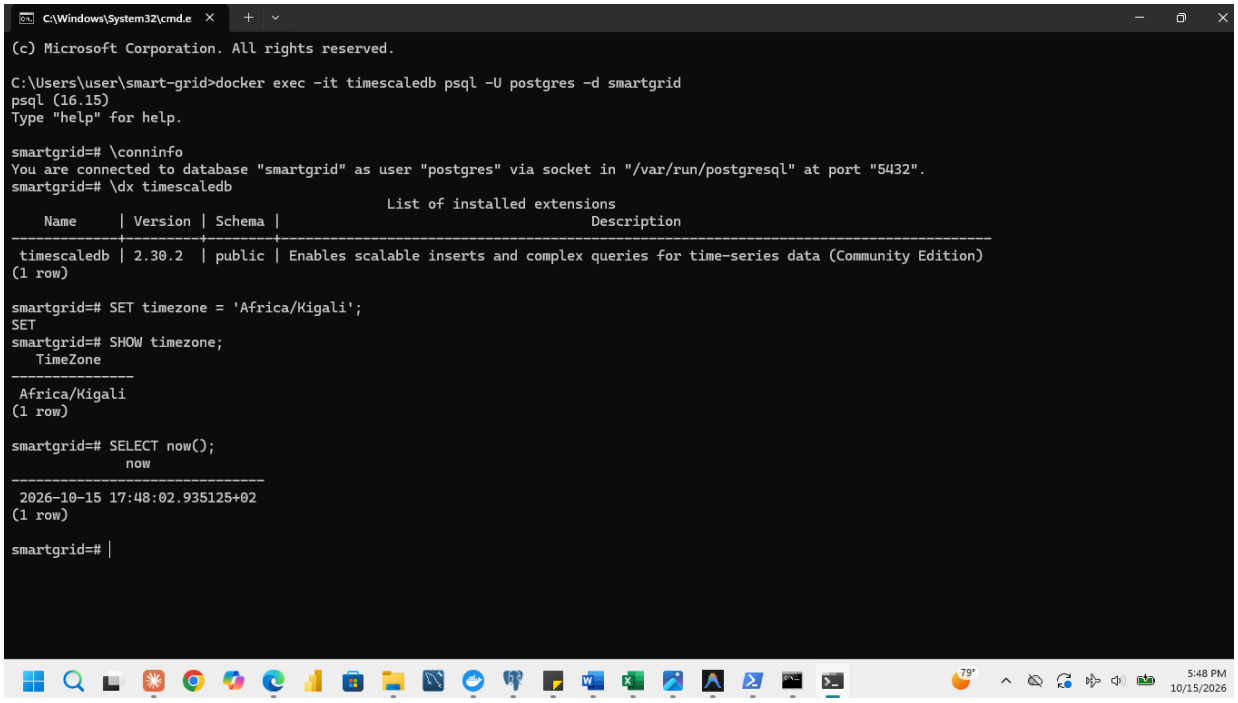


Figure 1.1-8: *Windows clock and time zone set to CAT and synchronised*

1.2 Create the database structures

The structures were created with `01_schema.sql`. **energy_live** is a regular PostgreSQL table for the pilot stream, **energy_readings** is a TimescaleDB hypertable partitioned by `event_time` with **one-day chunks**, and **meters** holds the metadata of the 4,000 meters. Both reading tables have the eight required columns and no unique constraint, because duplicate readings must be allowed at this stage.

```
CREATE TABLE energy_live (
    meter_id      BIGINT,
    event_time    TIMESTAMPTZ,
    received_time TIMESTAMPTZ,
```

```
    power_kw      DOUBLE PRECISION,
    voltage_v     DOUBLE PRECISION,
    current_a     DOUBLE PRECISION,
    frequency_hz  DOUBLE PRECISION,
    energy_kwh    DOUBLE PRECISION
);
CREATE TABLE energy_readings (LIKE energy_live);
SELECT create_hypertable('energy_readings', 'event_time', chunk_time_interval => INTERVAL '1 day');
CREATE TABLE meters (
    meter_id       BIGINT PRIMARY KEY,
    region         TEXT,
    customer_type  TEXT,
    base_power_kw  DOUBLE PRECISION,
    power_factor   DOUBLE PRECISION
);
```

Data dictionary

Table	Column	PostgreSQL type	Meaning	Unit
energy_live, energy_readings	meter_id	BIGINT	Ten-digit meter identifier, 1000000000 to 1000003999	none
	event_time	TIMESTAMPTZ	Start of the 15-minute measurement interval	CAT time
	received_time	TIMESTAMPTZ	Simulated arrival time of the reading (not the database insertion time)	CAT time
	power_kw	DOUBLE PRECISION	Average power over the interval	kW
	voltage_v	DOUBLE PRECISION	Voltage	V
	current_a	DOUBLE PRECISION	Current, using a power factor of 0.95	A
	frequency_hz	DOUBLE PRECISION	Network frequency	Hz
	energy_kwh	DOUBLE PRECISION	Energy used in the interval, approximately power_kw x 0.25 h	kWh
meters	meter_id	BIGINT (primary key)	Meter identifier, joins to the reading tables	none
	region	TEXT	Region 1 to Region 10	none
	customer_type	TEXT	residential, commercial or industrial	none
	base_power_kw	DOUBLE PRECISION	Base power level of the meter used by the generator	kW
	power_factor	DOUBLE PRECISION	Power factor (0.95)	none

Loading the metadata. `load_meters.py` imports `smart_meter_simulator`, calls `meter_metadata()` and inserts each returned dictionary as one row of `meters` with `execute_values`. The check `SELECT count(*), count(DISTINCT region) FROM meters` returns **4,000 rows** and 10 regions; every region has 400 meters. The customer types are residential 2,584, commercial 1,010 and industrial 406.

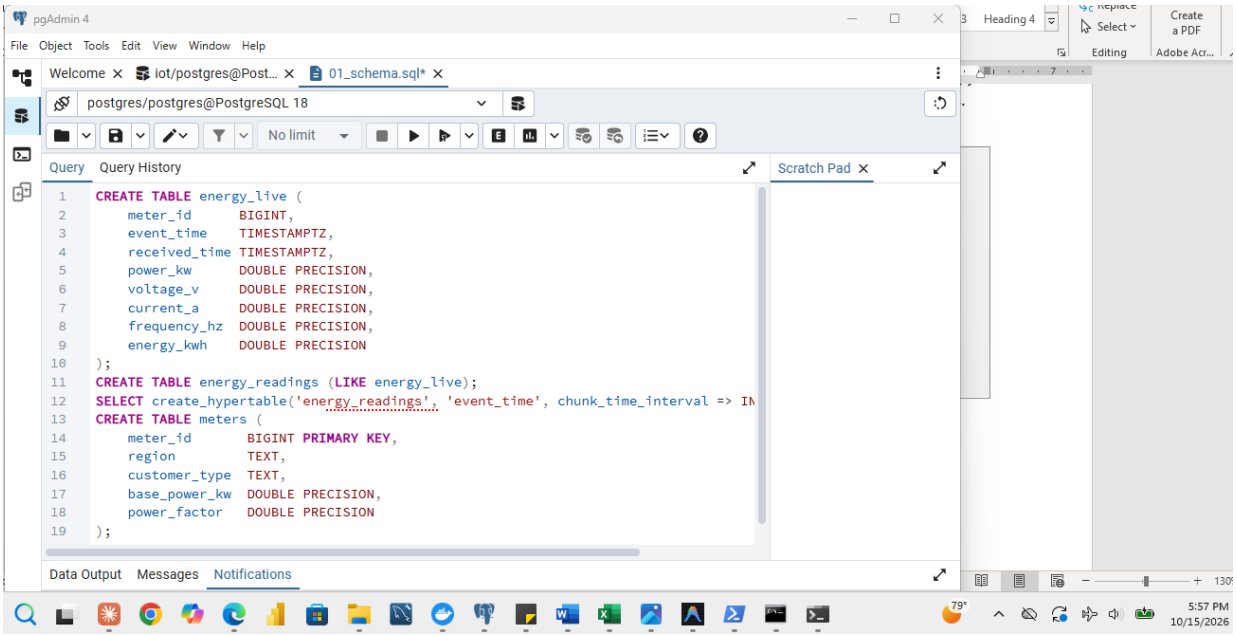
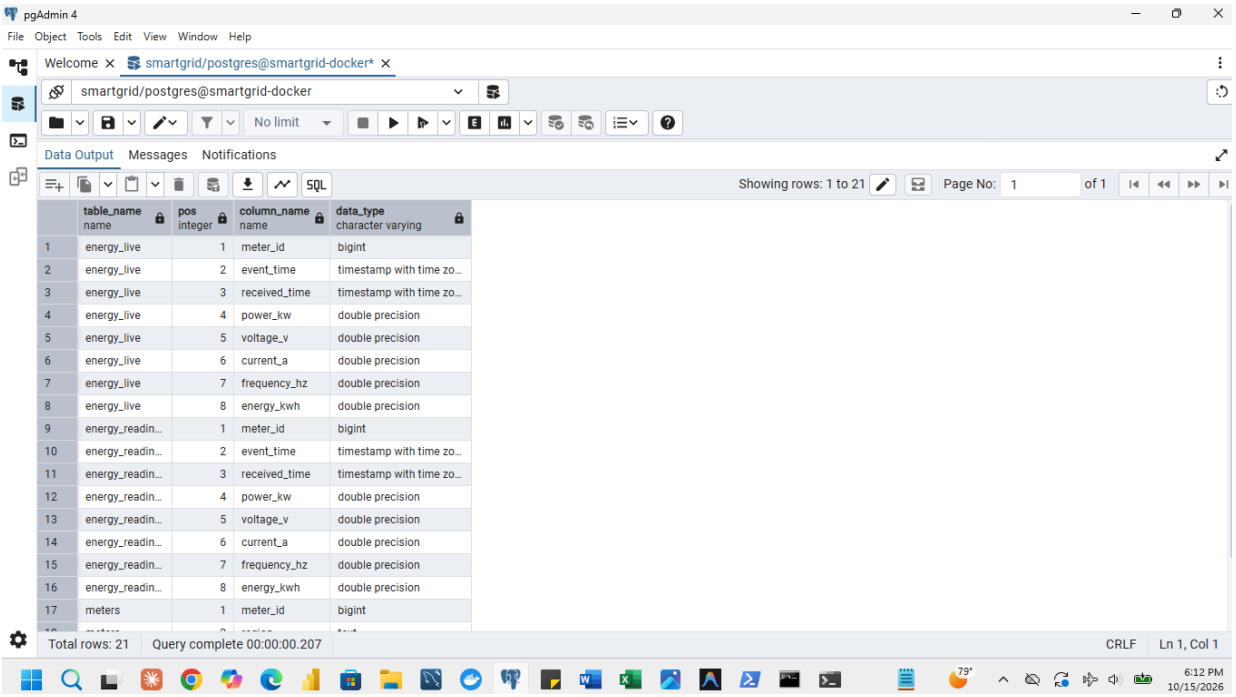


Figure 1.2-1: *Creation of energy_live, the energy_readings hypertable and meters*



	table_name name	pos integer	column_name name	data_type character varying
1	energy_live	1	meter_id	bigint
2	energy_live	2	event_time	timestamp with time zo...
3	energy_live	3	received_time	timestamp with time zo...
4	energy_live	4	power_kw	double precision
5	energy_live	5	voltage_v	double precision
6	energy_live	6	current_a	double precision
7	energy_live	7	frequency_hz	double precision
8	energy_live	8	energy_kwh	double precision
9	energy_readin...	1	meter_id	bigint
10	energy_readin...	2	event_time	timestamp with time zo...
11	energy_readin...	3	received_time	timestamp with time zo...
12	energy_readin...	4	power_kw	double precision
13	energy_readin...	5	voltage_v	double precision
14	energy_readin...	6	current_a	double precision
15	energy_readin...	7	frequency_hz	double precision
16	energy_readin...	8	energy_kwh	double precision
17	meters	1	meter_id	bigint

Figure 1.2-2: *Column names and types of the three tables (`\d` output)*

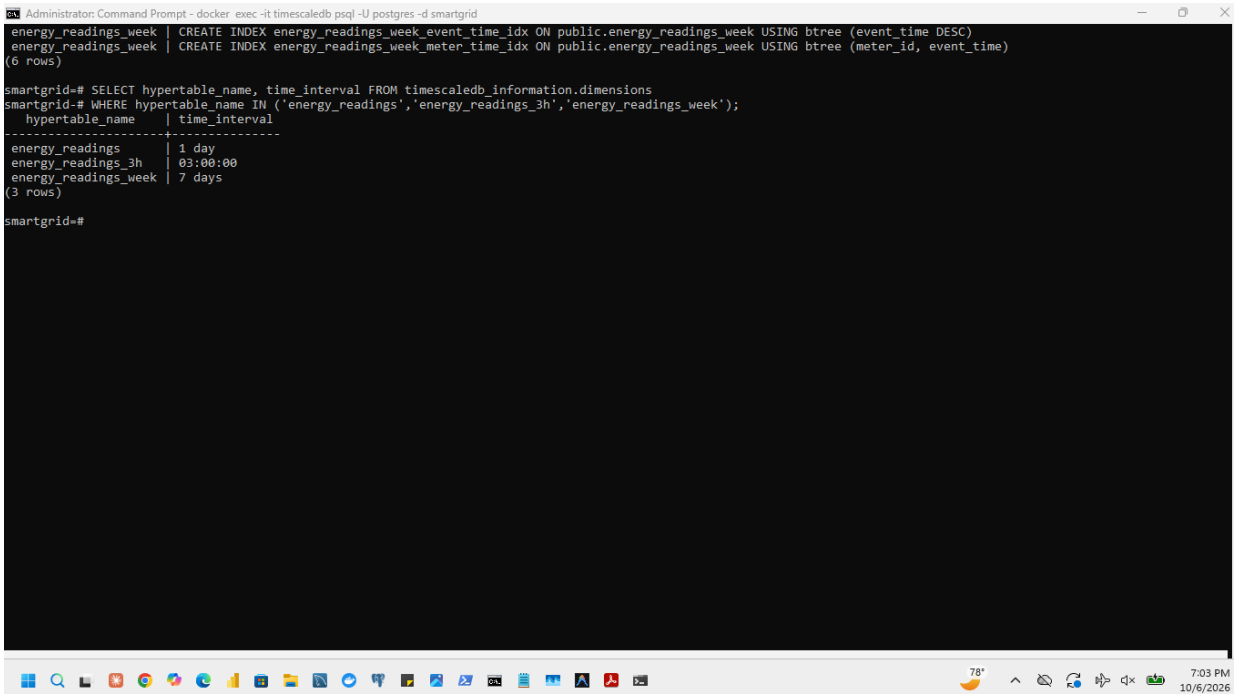


Figure 1.2-3: *Hypertable chunk interval of energy_readings (one day)*

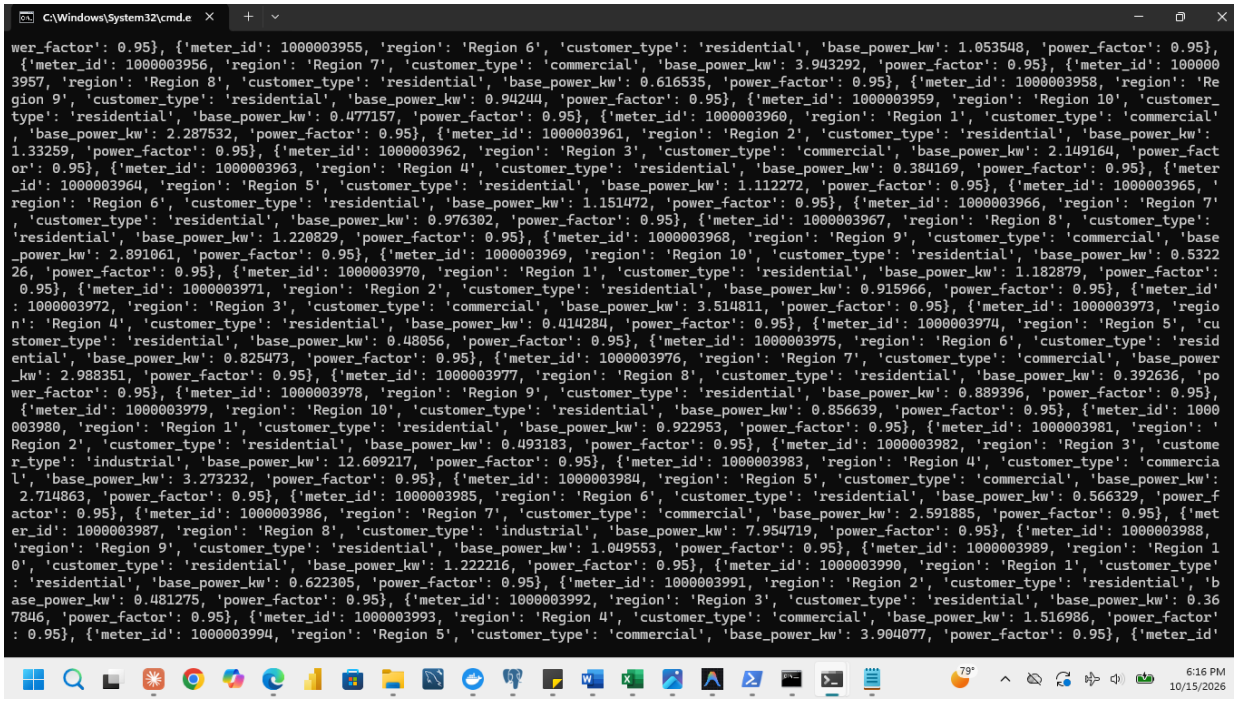


Figure 1.2-4: *Python program loading the meter metadata*

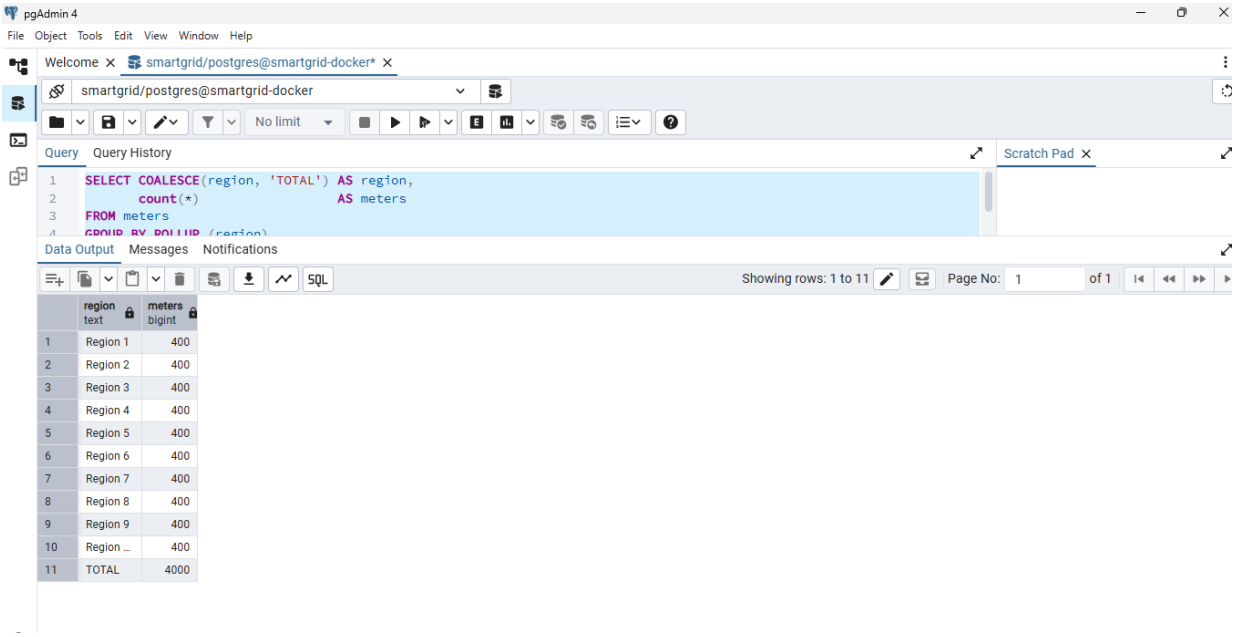


Figure 1.2-5: *Query confirming 4,000 metadata rows (400 meters in each of 10 regions)*

1.3 Capture and publish a pilot stream

The pilot replays the first 2,000 dictionaries of `iter_readings()` through the broker. `pilot_publisher.py` imports the unchanged generator, converts each dictionary to JSON and publishes one message to `energy/meters/{meter_id}` with MQTT QoS 1 and no retained flag; it counts the dictionaries taken and the messages acknowledged by the broker (PUBACK). `pilot_subscriber.py` subscribes to `energy/meters/#` at QoS 1, parses each JSON message and inserts its eight fields into `energy_live`; it counts messages received, redelivered (MQTT duplicate flag), inserted and failed. `energy_live` was emptied with `TRUNCATE energy_live` and the subscriber was started first, then the publisher. Deliberate generator duplicates are counted as separate messages and rows.

Pilot counts

The target is 2,000 at every stage. The first two runs failed and were corrected; the last row is the repeated pilot after the correction.

Run	Taken from generator	Acknowledged by broker	Received by subscriber	Rows in energy_live	Result
1	2,000	2,000	1,238	1,238	762 short: fault
2	2,000	2,000	2,000	4,000	2,000 too many: fault
Final	2,000	2,000	2,000	2,000	record

Investigation of run 1

The broker acknowledged all 2,000 messages (QoS 1), so the publisher side was complete. The subscriber received only 1,238, with **0 redelivered** and **0 failed inserts**, so the 762 missing messages were neither parsing errors nor failed inserts: they were lost between the broker and the subscriber. The first version inserted and committed each row inside the MQTT callback, which is slow compared with the publisher, which sent at full speed. The broker keeps only a limited queue of undelivered messages for a slow subscriber, and messages beyond it are dropped. The EMQX metrics (dropped messages, queue full)

Investigation of run 2 and the correction

After changing the subscriber, all 2,000 messages were received, but `energy_live` held **4,000 rows**, twice the target and more than the number received, which one subscriber cannot produce. Rows must therefore have been inserted twice: After changing the subscriber, all 2,000 messages were received, but energy_live held 4,000 rows. Because the subscriber's received count was 2,000, it could not have produced 4,000 rows from one run. The 4,000 rows contain 1,994 distinct meter IDs at a single event_time, so every record appears twice. The subscriber log showed energy_live already held 2,000 rows when the run started: it had not been emptied after the earlier run. The pilot was repeated after TRUNCATE energy_live was confirmed to return 0 rows, and all four counts equalled 2,000. The correction was to make the subscriber callback only put the message on an in-memory queue, with one worker thread inserting in batches and committing, to stop any other subscriber process, to empty `energy_live` with `TRUNCATE`, and to report the counts only after the queue is empty and the last batch is committed. The pilot was then repeated (final row of the table).

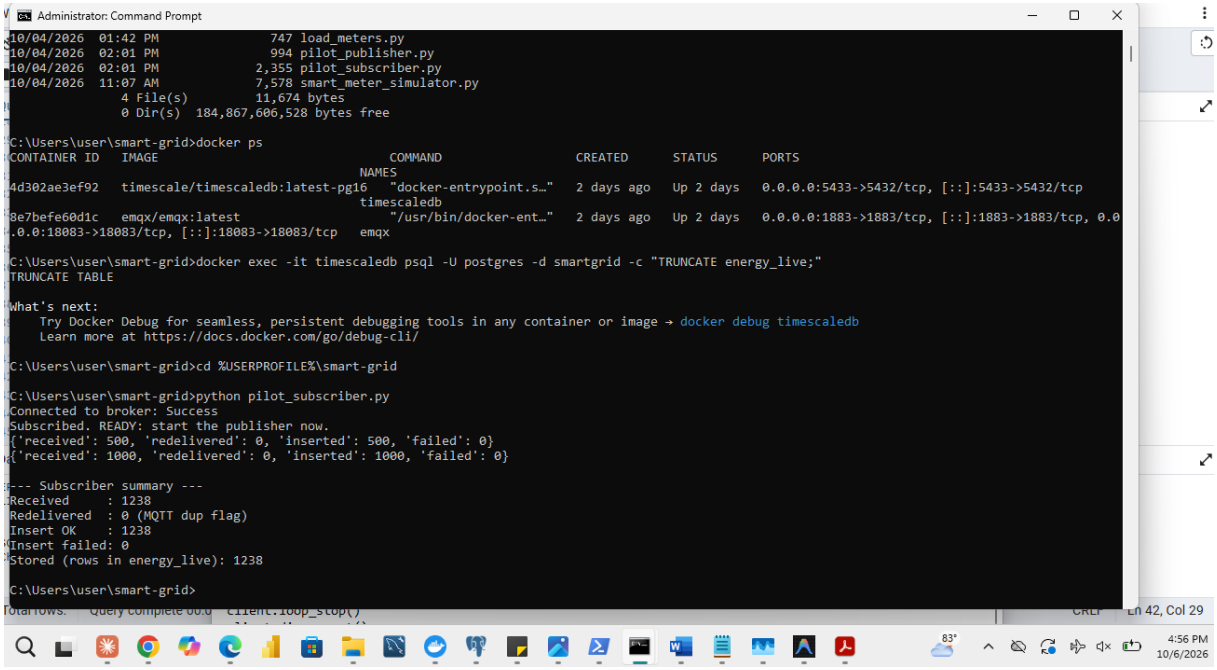


Figure 1.3-1: *Run 1: subscriber summary after the first pilot: 1,238 received, 0 redelivered, 1,238 inserted, 0 failed*

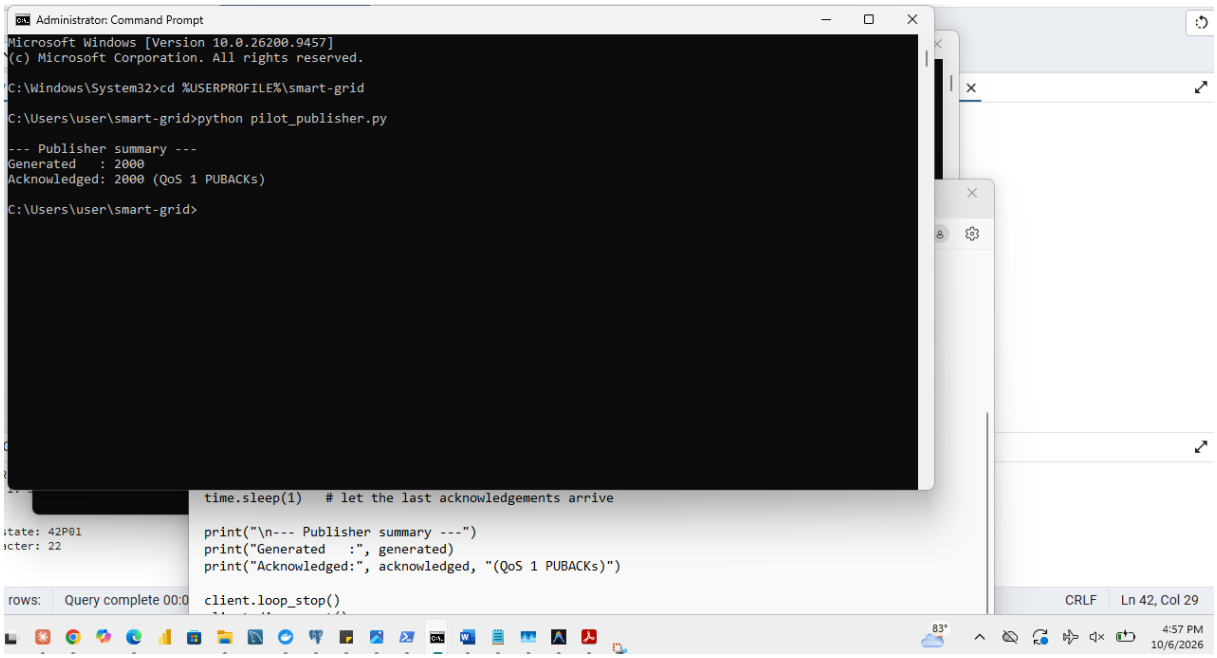


Figure 1.3-2: *Publisher completion: 2,000 dictionaries generated and 2,000 messages acknowledged (QoS 1 PUBACK)*

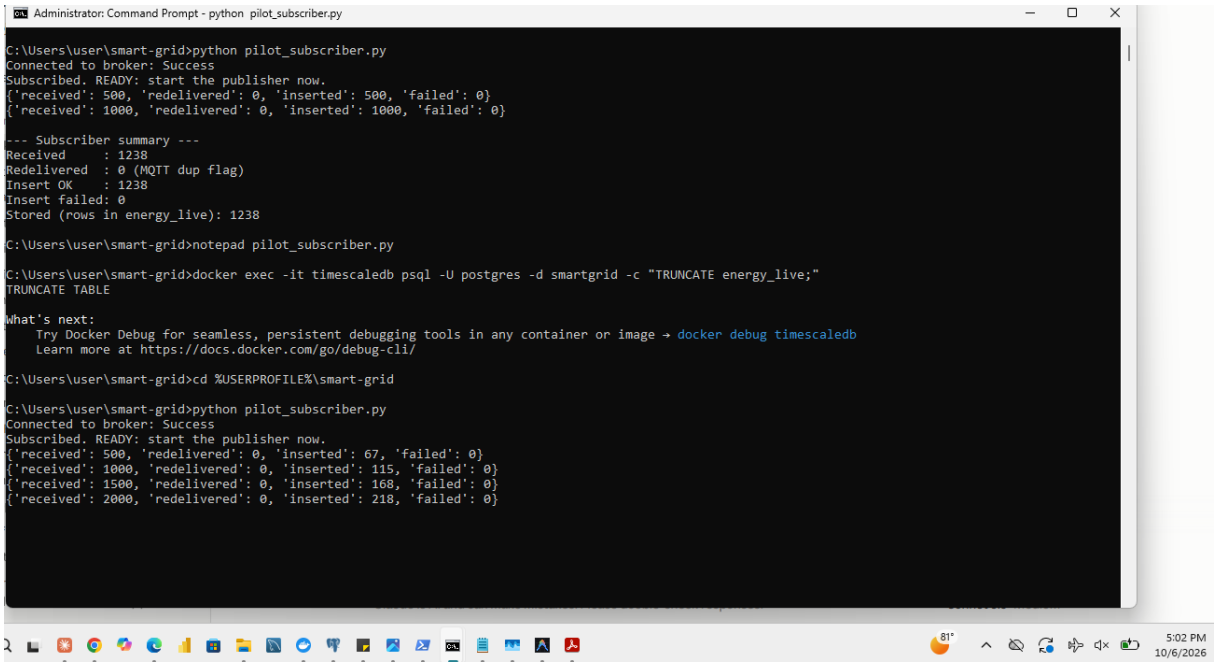


Figure 1.3-3: *Run 2 in progress: 2,000 messages received while only 218 had been inserted at that moment*

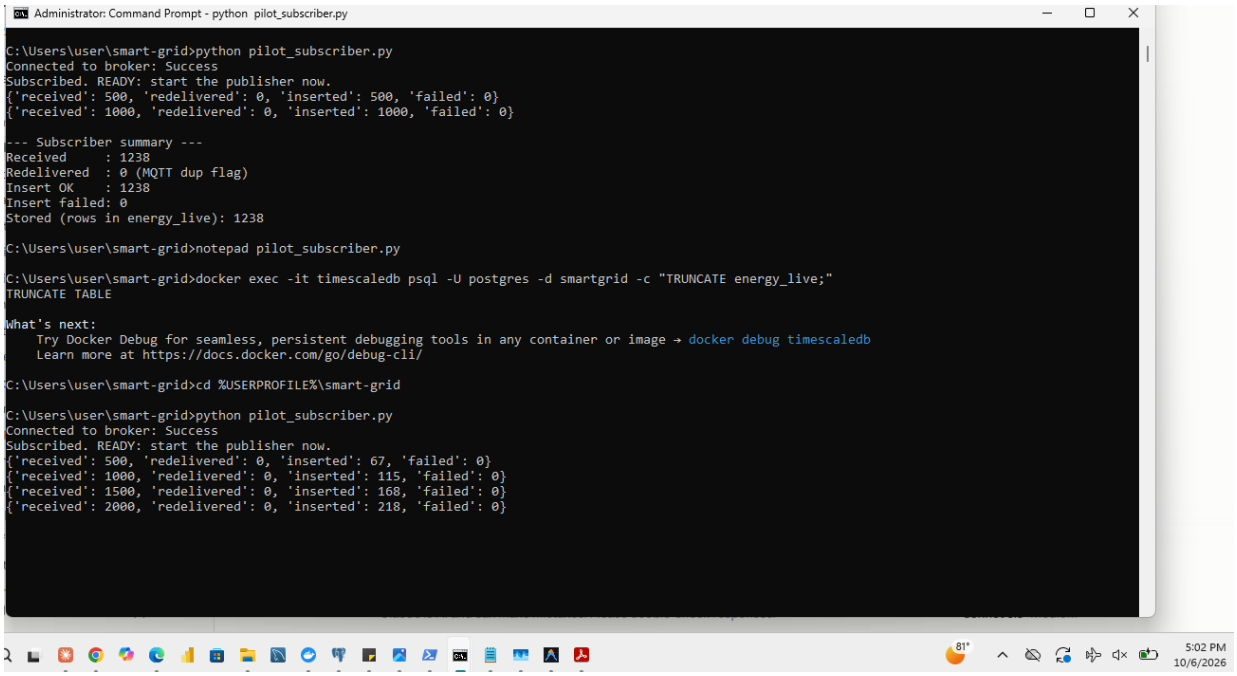


Figure 1.3-4: *Subscriber started first: connected, subscribed and energy_live empty*

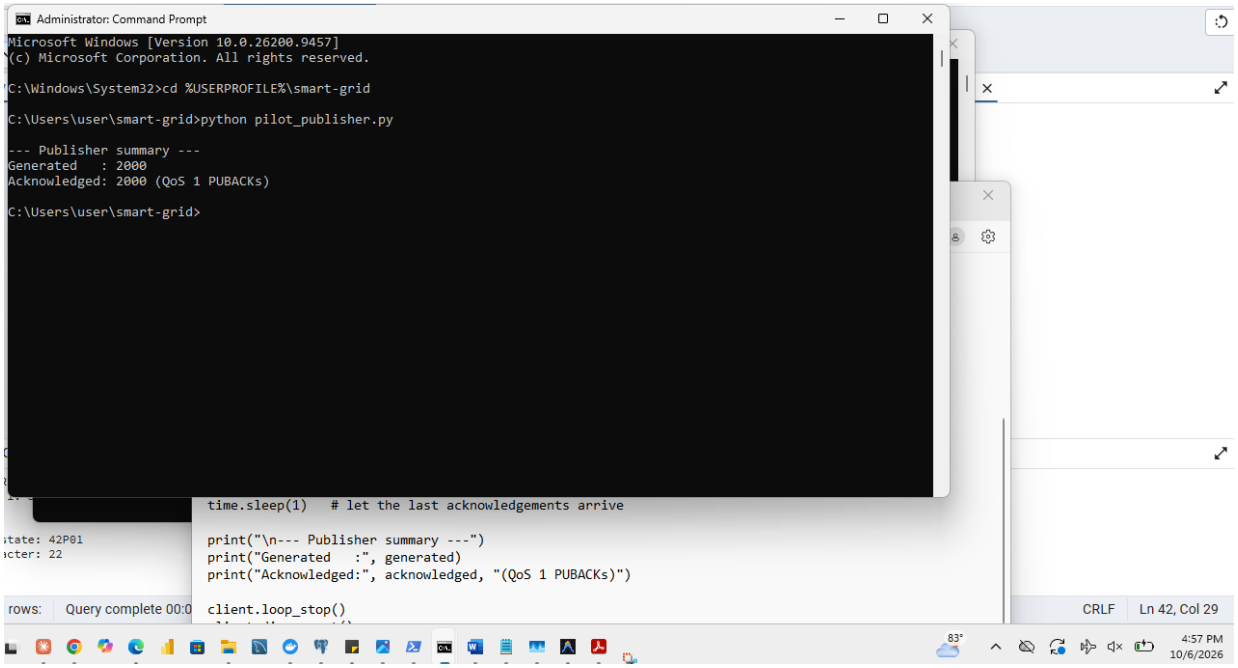


Figure 1.3-5: *Publisher started*

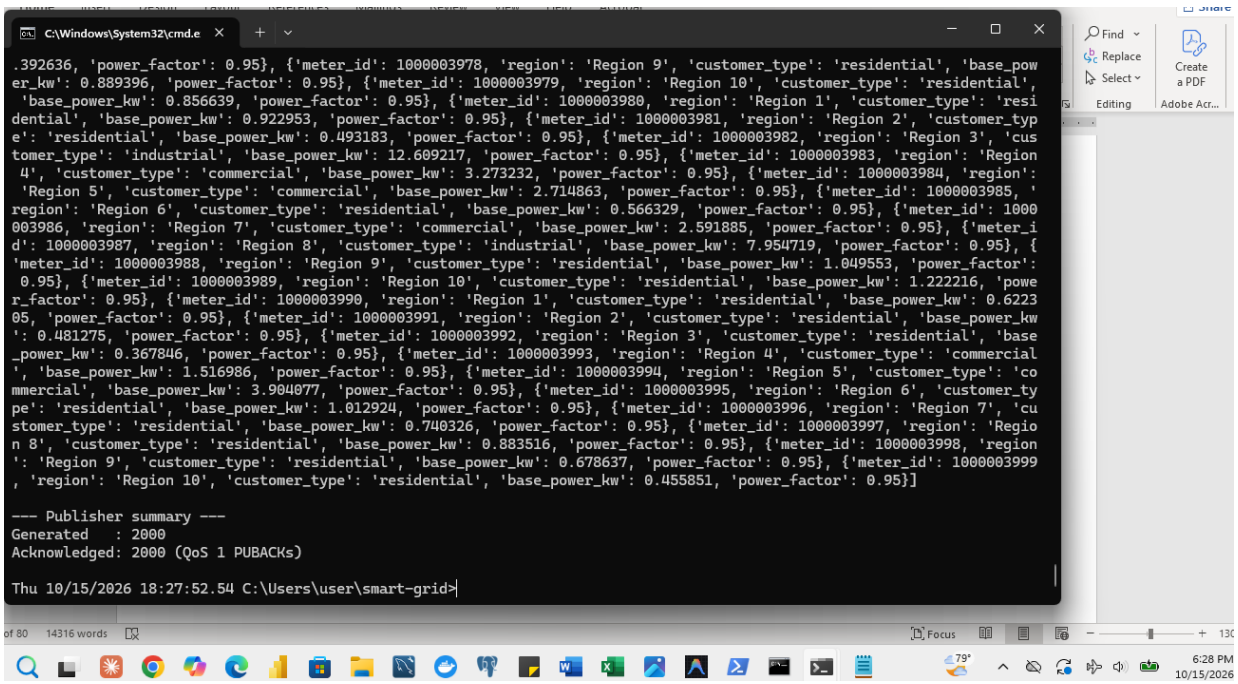


Figure 1.3-6: *Final run, publisher side: generated and acknowledged counts*

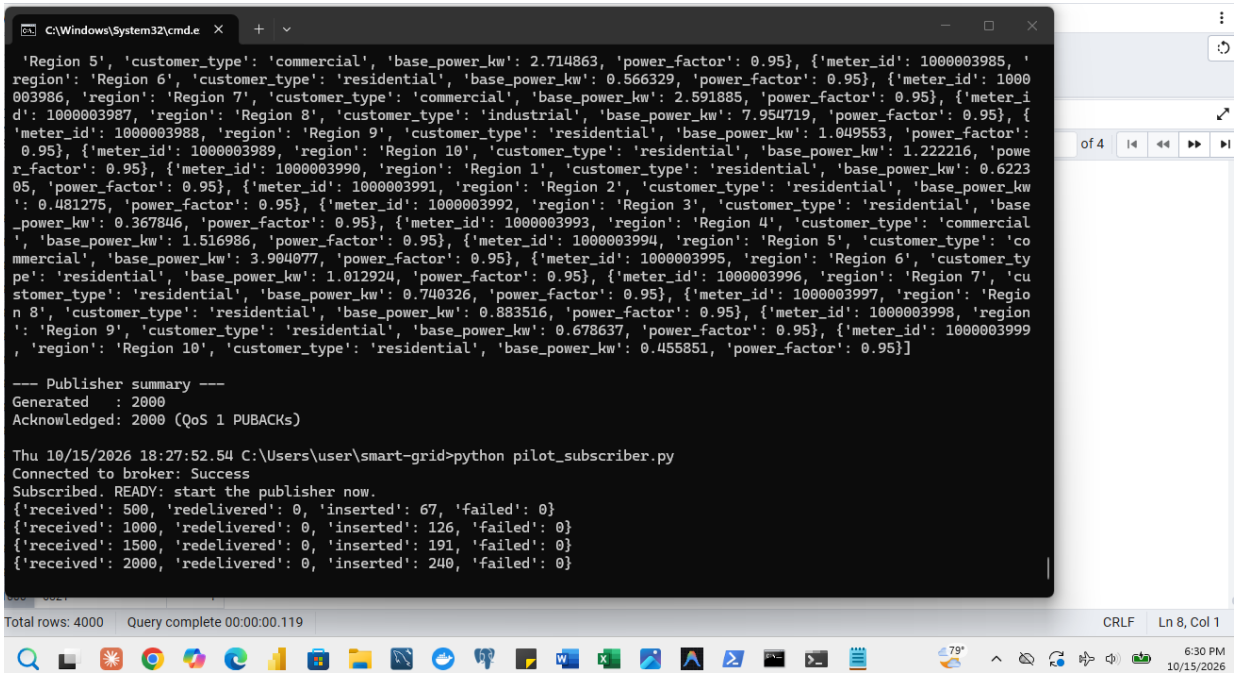


Figure 1.3-7: *Final run, subscriber side: received count and the committed row count of energy_live*

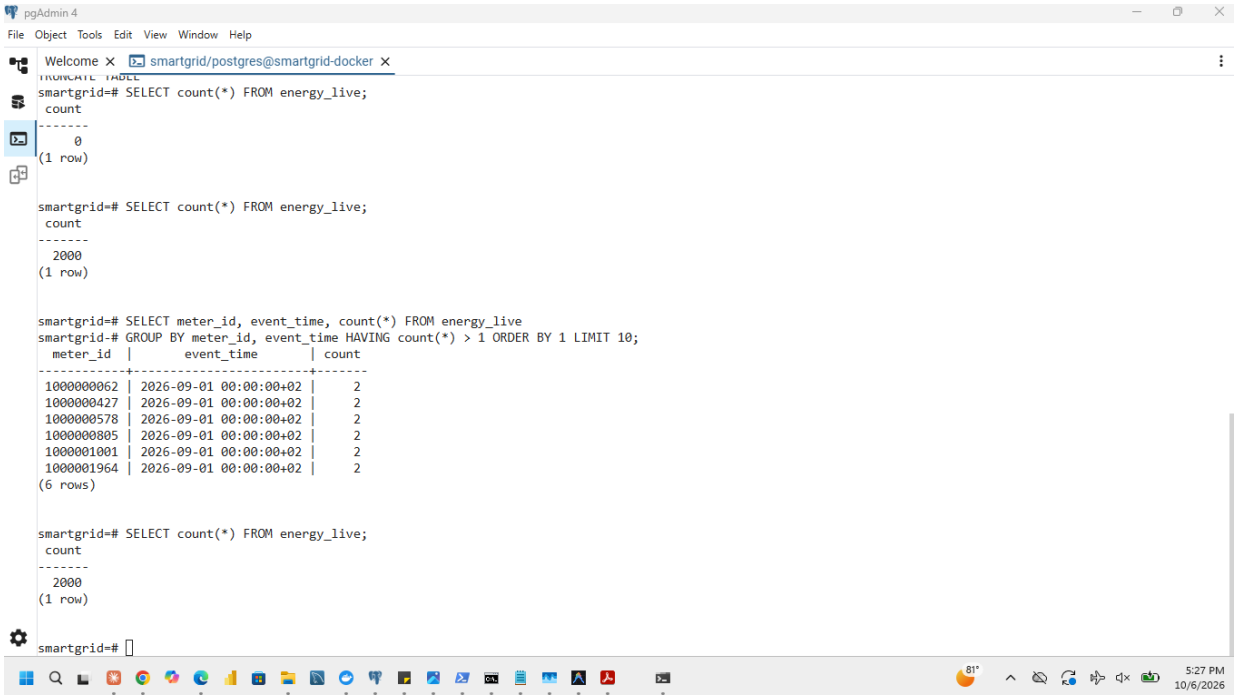


Figure 1.3-8: *Sample rows stored in energy_live*

2 Capture and prepare the historical dataset

2.1 Capture and load the complete stream

`dataset_info()` was called from the loader program and its expected counts were recorded: **10,752,000 planned readings** (4,000 meters x 28 days x 96 intervals) and **10,696,848 emitted readings** including duplicates. A fresh `iter_readings()` iterator covers 4,000 meters every 15 minutes from 00:00 on 1 September to 00:00 on 29 September 2026 (exclusive). `load_full.py` consumes the iterator progressively in batches, prints progress and writes `load_log.csv`; it never holds the dataset in memory. The chosen route is, and all emitted rows, including duplicates, were loaded into the initially empty `energy_readings`. The pilot rows stay in `energy_live`.

The three counts must agree before cleaning.

Check	Expected	Measured
Dictionaries consumed from iter_readings()	10,696,848	10,696,848
Rows committed in energy_readings	10,696,848	10,696,848
Data rows in the CSV file (if the CSV route was used)	10,696,848	10,696,848 or not applicable

Timings of the chosen route only.

Time	Seconds
Total elapsed time, from starting generation to the final commit	record from load_log.csv
Load time, from the first database write to the final commit	record
File generation time (CSV route only)	record or not applicable

The sizes were measured with `hypertable_detailed_size('energy_readings')` and `hypertable_size('energy_readings')`.

Database check	Result
Distinct meters	4,000
Earliest event_time	2026-09-01 00:00:00+02
Latest event_time	2026-09-28 23:45:00+02
Table data size (all chunks)	958 MB
Index size (all chunks)	85 MB
Total size (data plus indexes)	1,043 MB
Number of chunks	29 (one day each; chunks follow UTC midnight, which is 02:00 CAT, so 28 CAT days touch 29 chunks)

After the load was verified and the screenshots taken, intermediate files may be deleted; the programs needed to regenerate them (`load_full.py` and the generator) are kept in the repository.

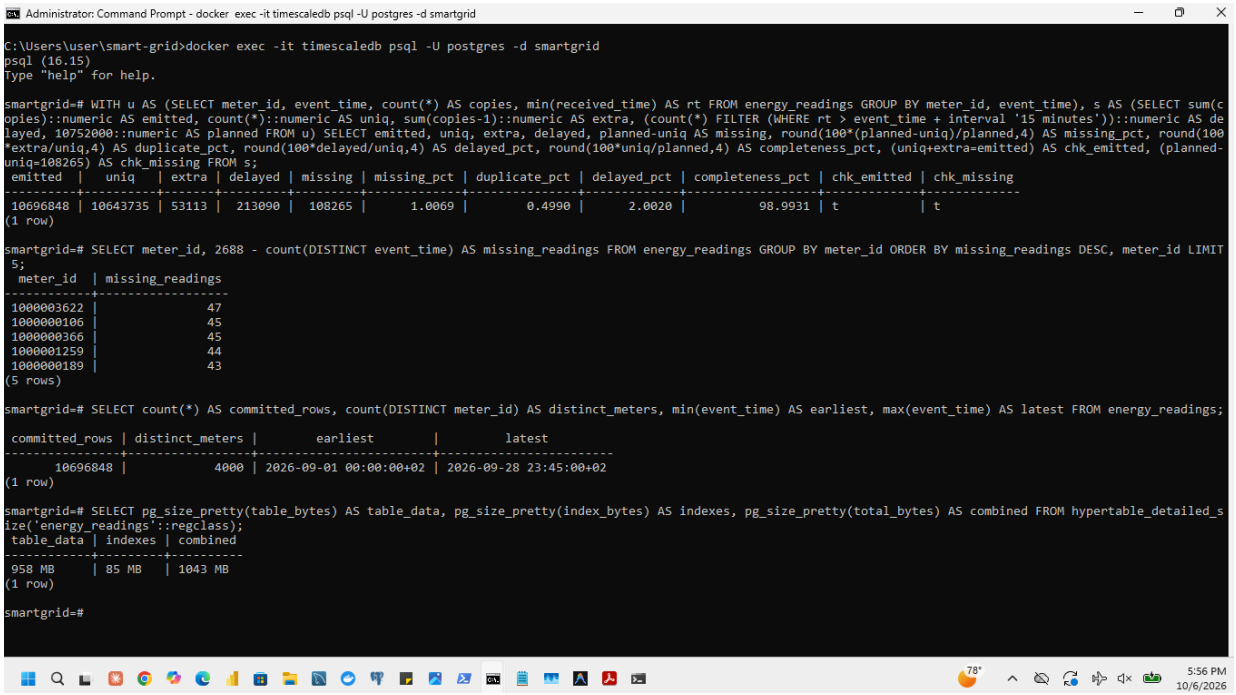


Figure 2.1-1: *`dataset_info()` expected counts and the dataset summary*

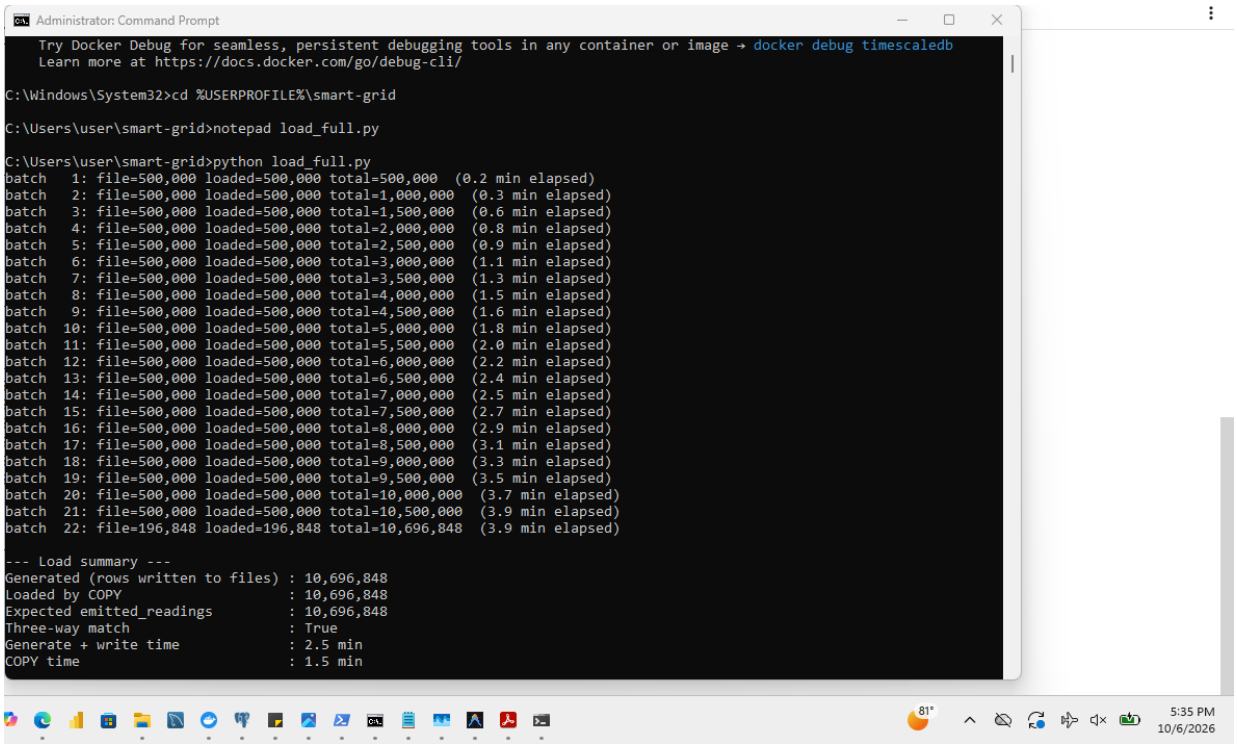


Figure 2.1-2: *Start of the full load and progress lines*

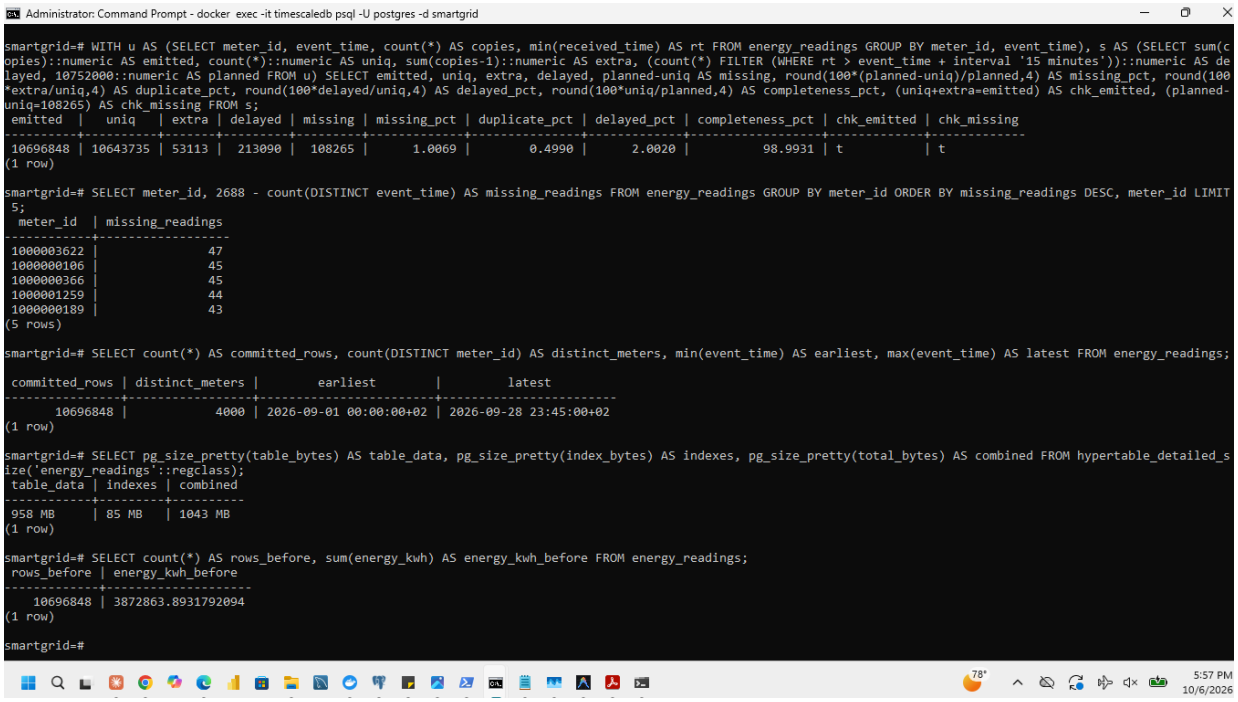


Figure 2.1-3: *Completed load: consumed, committed and elapsed seconds*

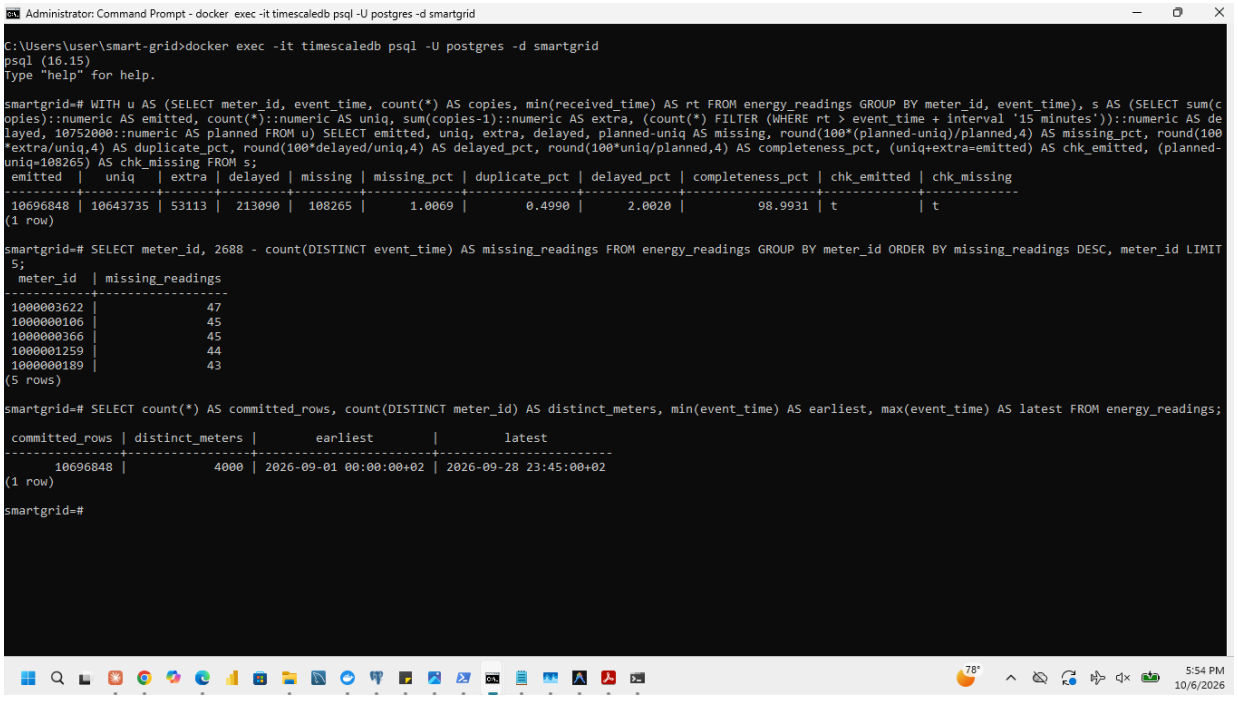


Figure 2.1-4: *Distinct meters and the earliest and latest event_time*

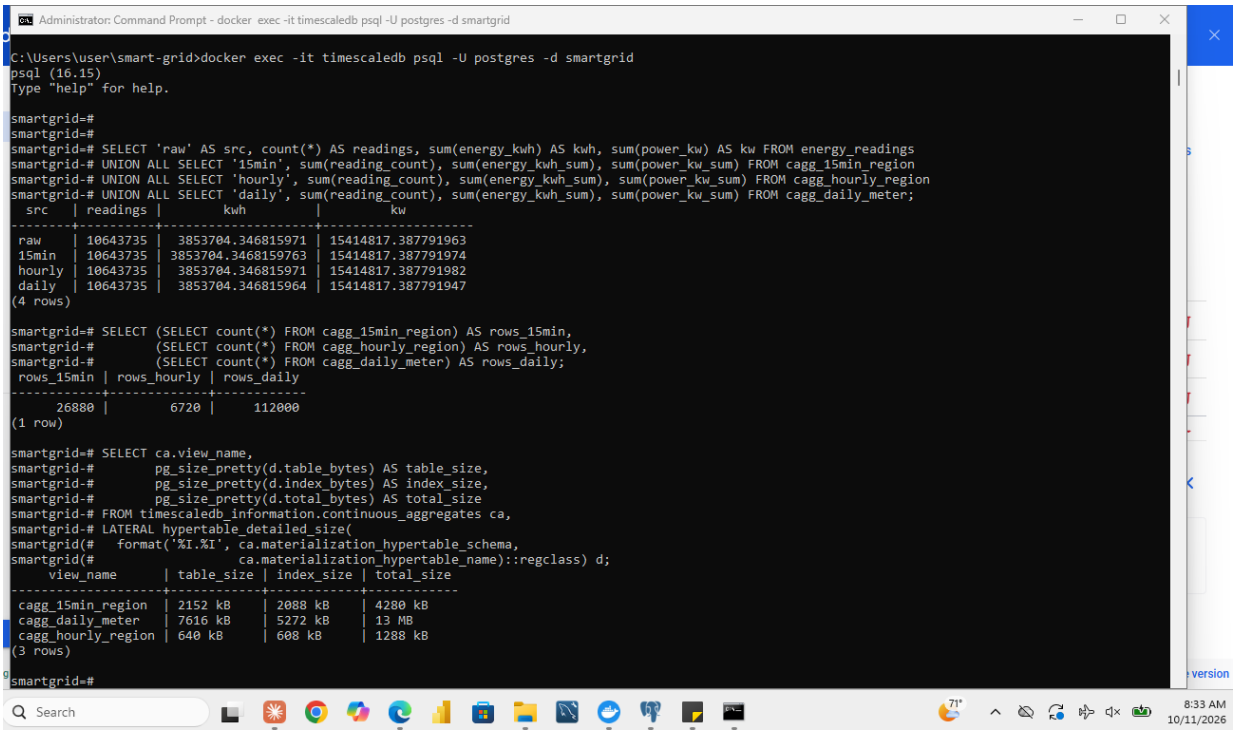


Figure 2.1-5: *Data, index and total size of energy_readings*

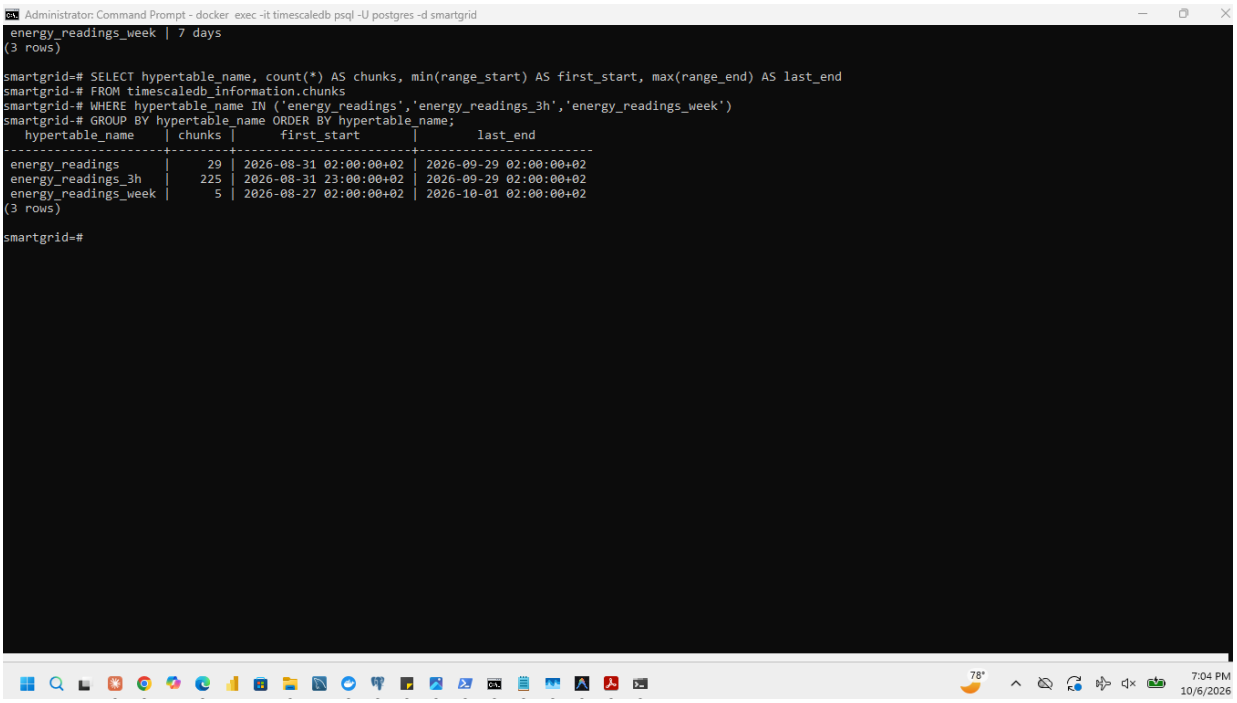


Figure 2.1-6: *Chunk count and data range of energy_readings*

2.2 Investigate data quality

Before cleaning, `energy_readings` was queried with one diagnostic query. A **unique reading** is one `meter_id` and `event_time` pair. Normal arrival is the interval end, 15 minutes after `event_time`; a reading is **delayed** when it arrives after that (30 to 180 minutes after `event_time`).

```
WITH u AS (
  SELECT meter_id, event_time, min(received_time) AS received_time, count(*) AS copies
  FROM energy_readings
  GROUP BY meter_id, event_time
)
SELECT count(*)                                    AS unique_readings,
```

```
       sum(copies)                                AS emitted_readings,
       sum(copies) - count(*)                     AS duplicate_copies,
       count(*) FILTER (WHERE received_time > event_time + INTERVAL '15 minutes') AS delayed_unique,
       10752000 - count(*)                        AS missing_readings
FROM u;
```

Measured counts against the reference table of the assignment.

Entry	Expected	Measured	Match
planned_readings	10,752,000	10,752,000	yes
emitted_readings	10,696,848	10,696,848	yes
unique_readings	10,643,735	10,643,735	yes
duplicate_readings (extra copies)	53,113	53,113	yes
delayed_unique_readings	213,090	213,090	yes
missing_readings	108,265	108,265	yes

Measure	Formula	Value
Missing percentage	100 x 108,265 / 10,752,000	1.0069 %
Duplicate percentage	100 x 53,113 / 10,643,735	0.4990 %
Delayed percentage	100 x 213,090 / 10,643,735	2.0020 %
Completeness	100 x 10,643,735 / 10,752,000	98.9931 %

Consistency checks. Unique readings plus extra copies is 10,643,735 + 53,113 = 10,696,848, which equals the emitted readings. Planned minus unique readings is 10,752,000 - 10,643,735 = 108,265, which equals the missing readings.

The five meters with the most missing readings.

Rank	meter_id	Missing readings
1	1000003622	47
2	1000000106	45
3	1000000366	45
4	1000001259	44
5	1000000189	43

The missing readings are spread thinly over all meters (4,000 meters share 108,265 missing readings, about 27 each, and even the worst meter misses only 47 of its 2,688), so no meter is a systematic reporting failure.

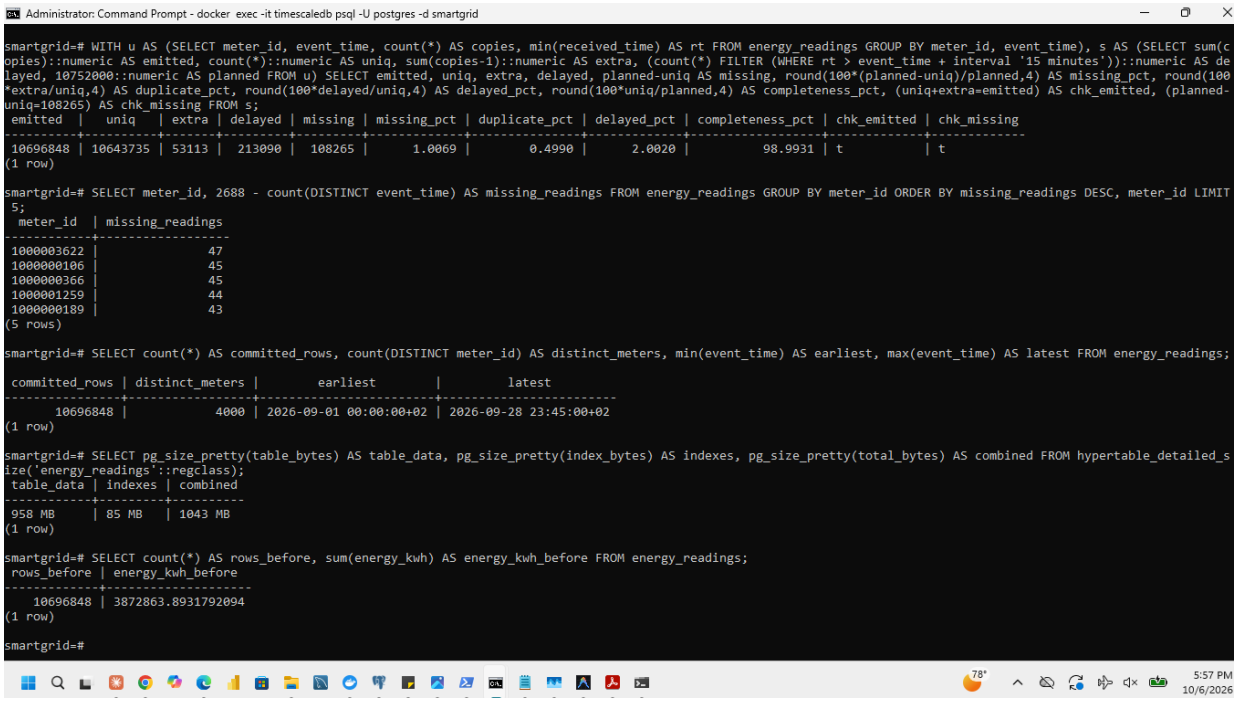


Figure 2.2-1: *Diagnostic query and its counts before cleaning*

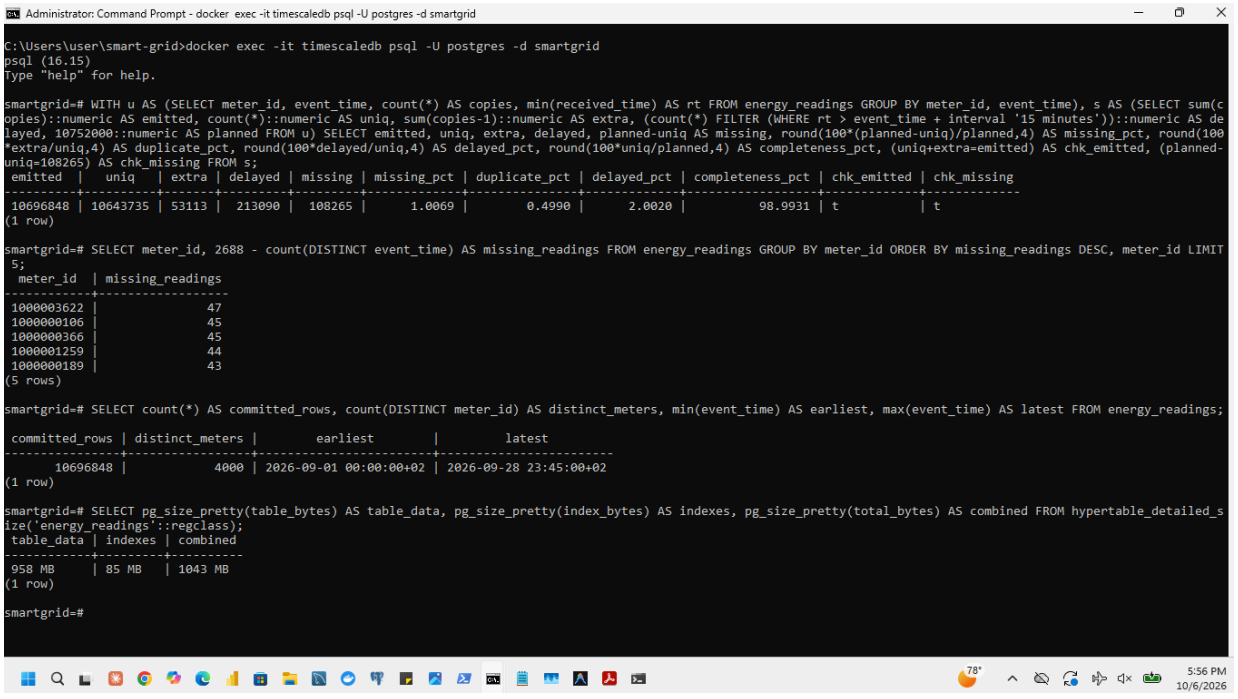


Figure 2.2-2: *Measured counts against the reference table*

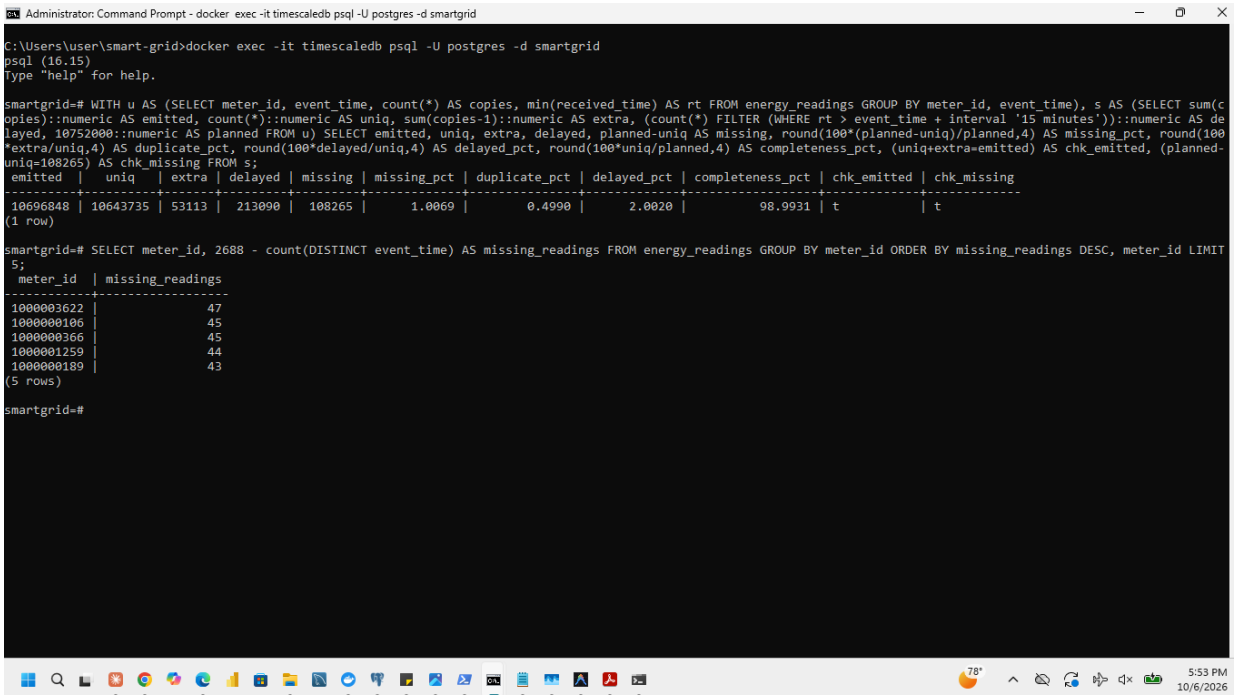


Figure 2.2-3: *Summary of the quality measures and the five meters with the most missing readings*

2.3 Clean and preserve a consistent benchmark

Rows sharing `meter_id` and `event_time` were compared on `received_time`, `power_kw`, `voltage_v`, `current_a`, `frequency_hz` and `energy_kwh`. The conflict check counts keys whose copies differ in any of the six values:

```
SELECT count(*) AS conflicting_keys FROM (
  SELECT meter_id, event_time
  FROM energy_readings
  GROUP BY meter_id, event_time
  HAVING count(*) > 1
     AND (count(DISTINCT received_time) > 1 OR count(DISTINCT power_kw) > 1 OR count(DISTINCT voltage_v)
> 1
       OR count(DISTINCT current_a) > 1 OR count(DISTINCT frequency_hz) > 1 OR count(DISTINCT energy_kwh)
> 1)
) c;
```

The result was 0 conflicting keys, so every duplicate is an exact copy. The cleaning keeps one row per pair and deletes the additional copies with repeatable code (`clean_duplicates.sql`): rows are numbered within each (`meter_id`, `event_time`) group and every row numbered above 1 is deleted. Missing intervals are kept as gaps.

Cleaning removed 53,113 rows and 19,159.5464 kWh of duplicated energy. The final row count equals `expected_counts.unique_readings`.

Measure	Before cleaning	After cleaning
Row count	10,696,848	10,643,735
Summed energy_kwh	3,872,863.8932	3,853,704.3468
Duplicate pairs remaining	53,113 extra copies	0

Why missing readings must not become zero consumption. A missing reading means the meter's value is unknown, not that the customer used nothing. Replacing it by zero would understate total energy by an unknown amount, lower the average power of every affected meter and region, make completeness look perfect while hiding the gaps, and create false 'unusual drop' patterns that the anomaly detection of Section

5.4 would then flag. Keeping the gaps lets every later calculation count only what was observed and report completeness separately. The cleaned `energy_readings` is the common dataset for every later experiment.

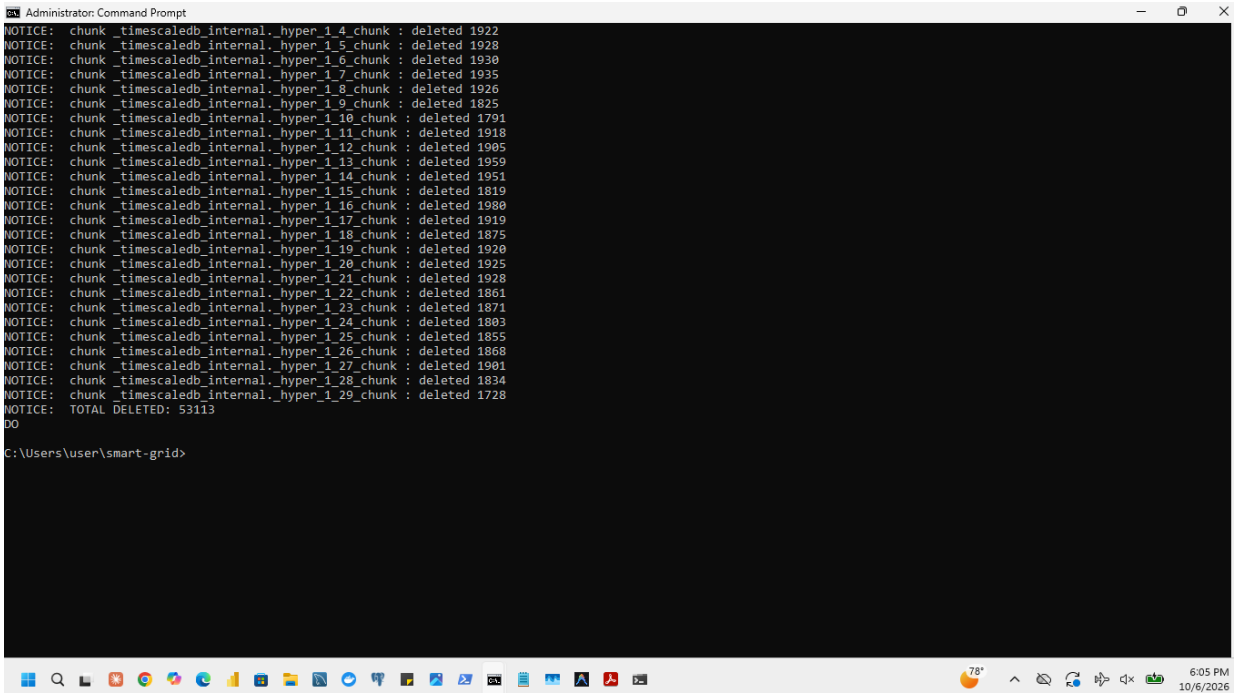


Figure 2.3-1: *Cleaning run: completion message and rows deleted*

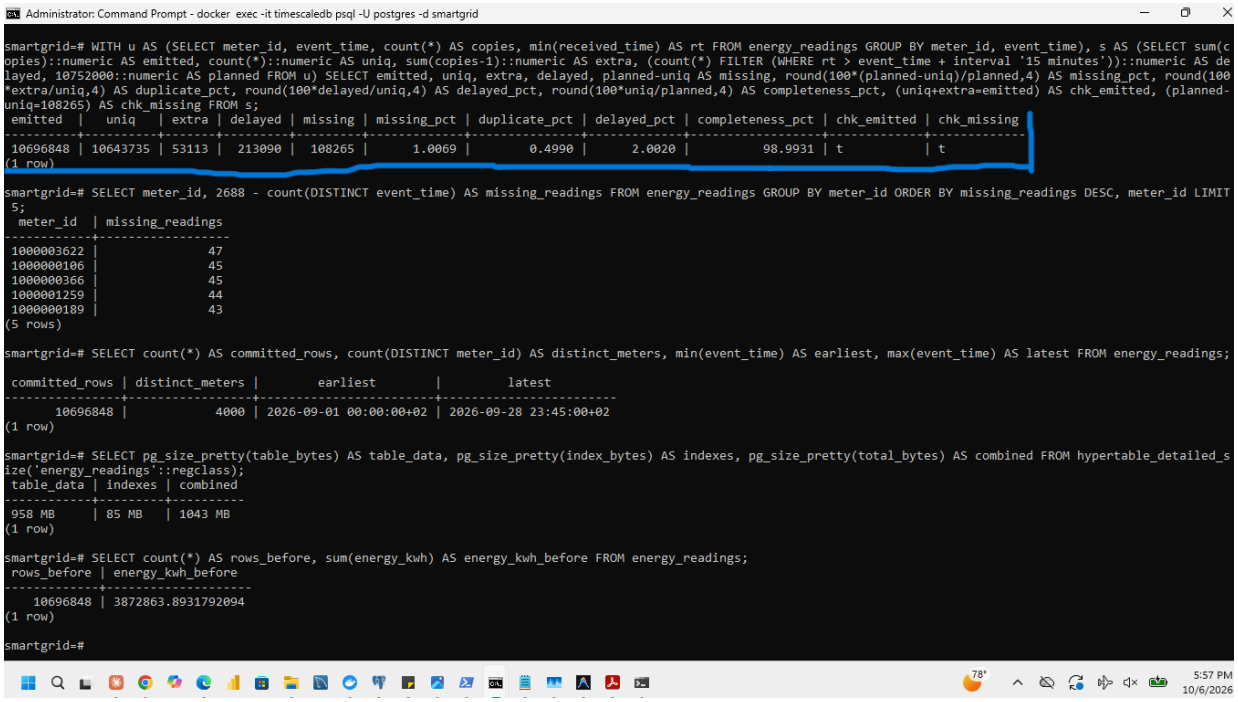


Figure 2.3-2: *Before cleaning: row count, summed energy and duplicate keys*

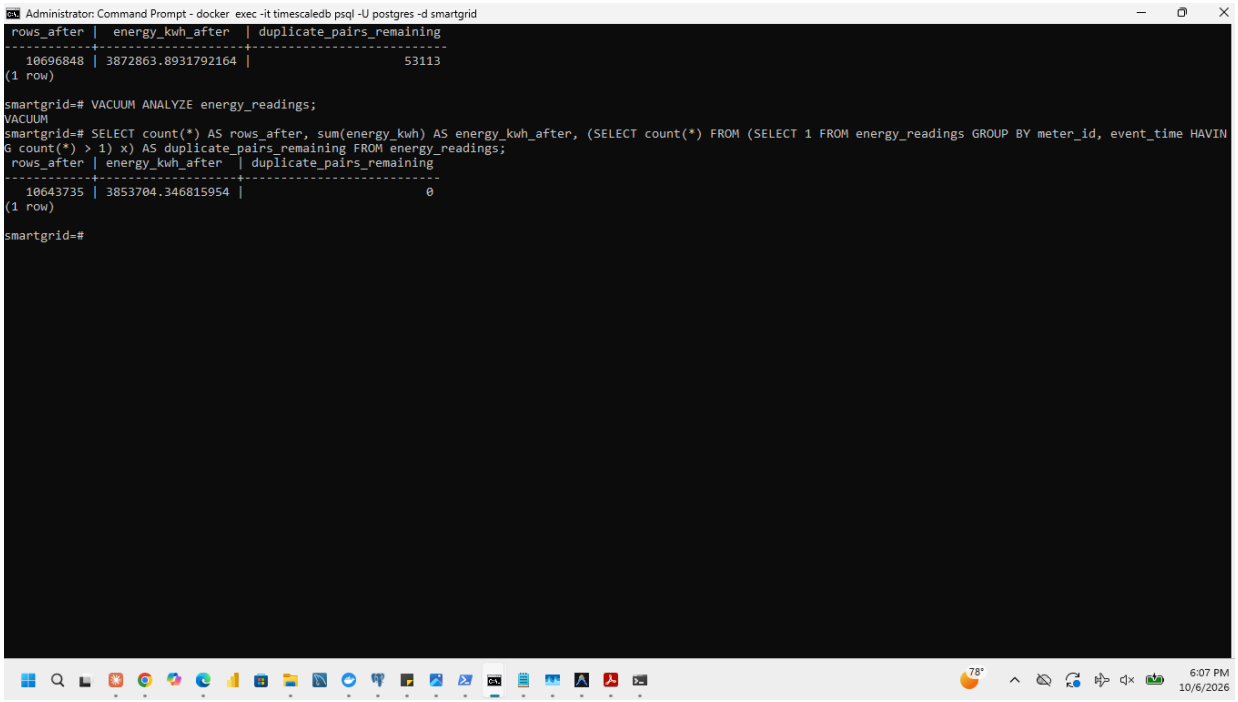


Figure 2.3-3: *After removing the duplicated rows: 10,643,735 rows*

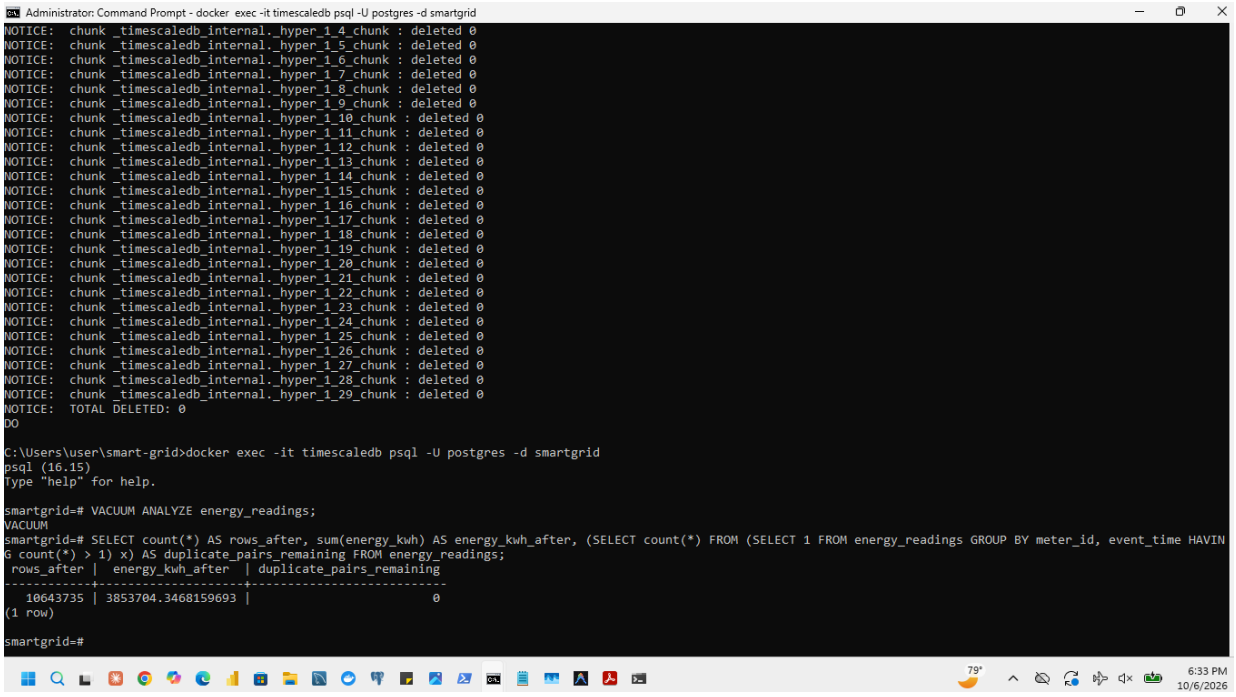


Figure 2.3-4: *Check that no duplicate pair remains*

3 Compare chunk intervals

3.1 Create and verify three configurations

energy_readings stays as the one-day configuration. energy_readings_3h (three-hour chunks) and energy_readings_week (one-week chunks) were created with the same eight columns and event_time partitioning, and the cleaned rows were copied into both. All three tables have the same two non-unique B-tree indexes, with the same sort order: one on event_time and one on meter_id, event_time. The automatic default index was switched off so that no table has an extra index.

```
CREATE TABLE energy_readings_3h   (LIKE energy_readings);
CREATE TABLE energy_readings_week (LIKE energy_readings);
SELECT create_hypertable('energy_readings_3h',   'event_time', chunk_time_interval => INTERVAL '3 hours',
                         create_default_indexes => false);
SELECT create_hypertable('energy_readings_week', 'event_time', chunk_time_interval => INTERVAL '7 days',
                         create_default_indexes => false);
INSERT INTO energy_readings_3h   SELECT * FROM energy_readings;
INSERT INTO energy_readings_week SELECT * FROM energy_readings;

-- identical indexes on all three tables
CREATE INDEX ix_<table>_time       ON <table> (event_time);
CREATE INDEX ix_<table>_meter_time ON <table> (meter_id, event_time);
ANALYZE energy_readings; ANALYZE energy_readings_3h; ANALYZE energy_readings_week;
```

Data checks on the three configurations.

Check	energy_readings (1 day)	energy_readings_3h	energy_readings_week	Required
Row count	10,643,735	10,643,735	10,643,735	10,643,735
Distinct meters	4,000	4,000	4,000	4,000
Earliest event_time	2026-09-01 00:00:00+02	same	same	identical
Latest event_time	2026-09-28 23:45:00+02	same	same	identical
Sum of power_kw	15,414,817.388	same	same	difference at most 0.001
Sum of energy_kwh	3,853,704.347	same	same	difference at most 0.001
Chunk interval	1 day	3 hours	7 days	
Number of chunks	29	about 225	about 5	

The chunk counts follow the alignment of chunk boundaries to UTC. The 28 CAT days run from 31 August 22:00 UTC to 28 September 21:45 UTC: that touches 29 one-day chunks and 225 three-hour chunks (675 hours divided by 3). One-week chunks start on Thursdays in UTC, so the period touches five of them.

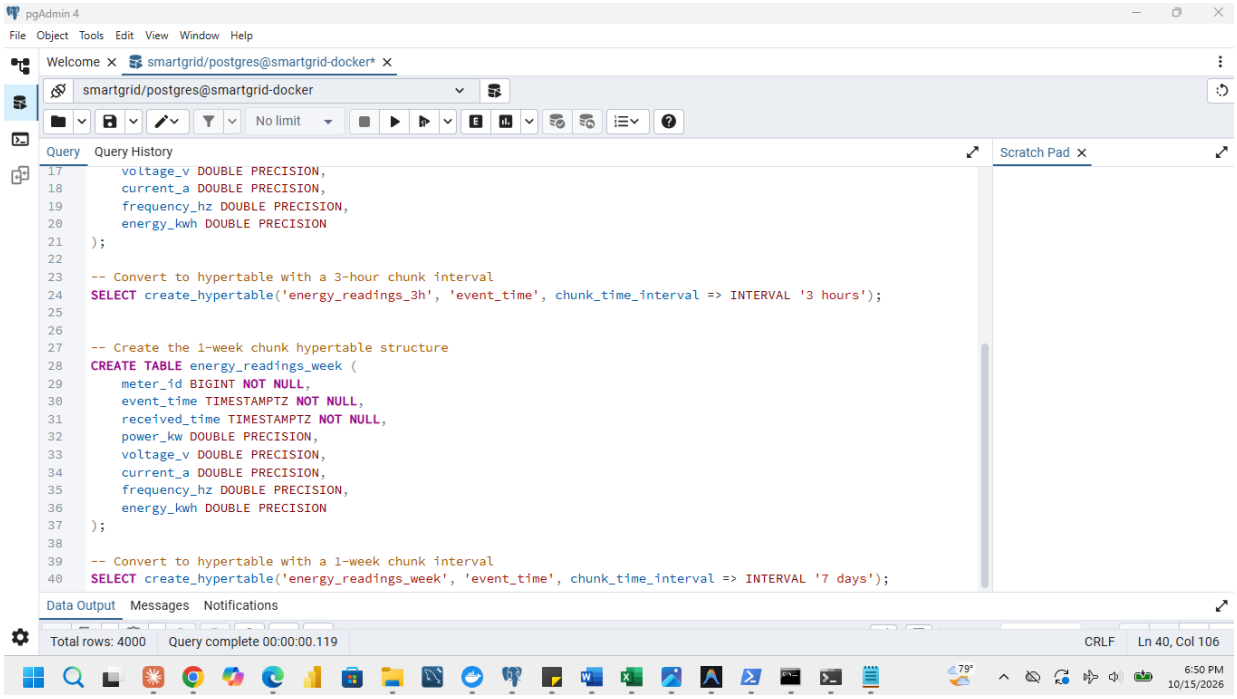


Figure 3.1-1: *Creation of the 3-hour and 1-week hypertables and the copy of the cleaned rows*

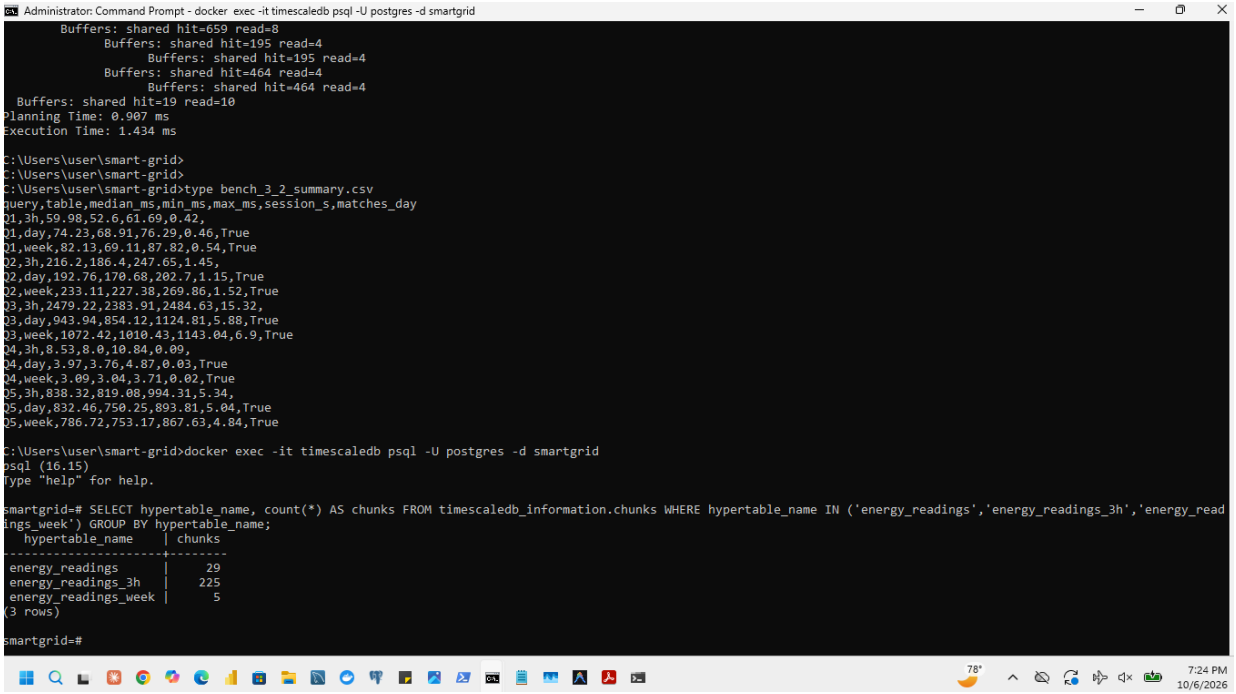


Figure 3.1-2: *Chunk interval, chunk count and date range of each configuration*

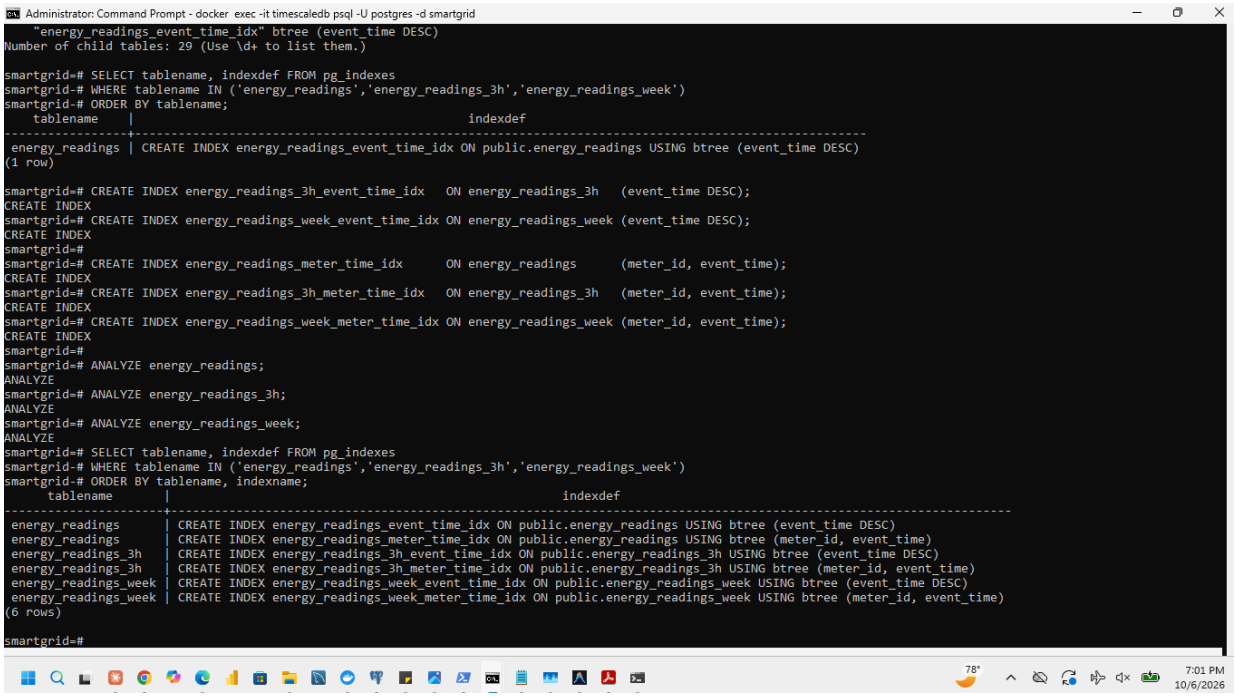


Figure 3.1-3: *Indexes of the three tables (identical definitions)*

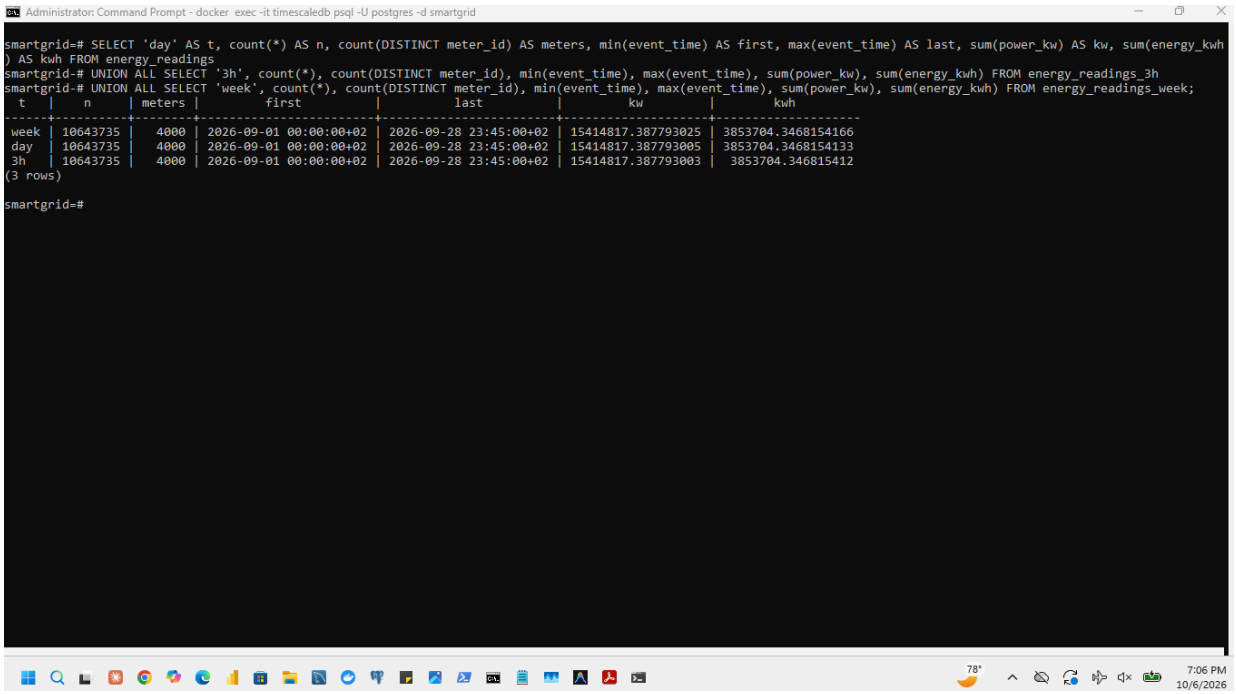


Figure 3.1-4: *Matching data checks on the three tables*

3.2 Implement and time the five workloads

The five required queries were run unchanged on each table; only the table name changes. ANALYZE was run after loading so that the planner has current row estimates.

```
-- Q1: average power per CAT hour on 21 September
SELECT time_bucket(INTERVAL '1 hour', event_time, 'Africa/Kigali') AS hour, avg(power_kw)
FROM energy_readings
WHERE event_time >= '2026-09-21 00:00+02' AND event_time < '2026-09-22 00:00+02'
GROUP BY 1 ORDER BY 1;

-- Q2: ten highest 15-minute network demand intervals, 22 to 28 September
SELECT event_time, sum(power_kw) AS total_kw
FROM energy_readings
```

```
WHERE event_time >= '2026-09-22 00:00+02' AND event_time < '2026-09-29 00:00+02'
GROUP BY event_time ORDER BY total_kw DESC, event_time ASC LIMIT 10;

-- Q3: total energy per meter over the whole period
SELECT meter_id, sum(energy_kwh) FROM energy_readings GROUP BY meter_id;

-- Q4: one meter, 22 to 28 September
SELECT sum(energy_kwh), avg(power_kw), max(power_kw)
FROM energy_readings
WHERE meter_id = 1000000000
  AND event_time >= '2026-09-22 00:00+02' AND event_time < '2026-09-29 00:00+02';

-- Q5: full scan
SELECT count(*), sum(energy_kwh), avg(power_kw), min(power_kw), max(power_kw) FROM energy_readings;
```

Procedure

- One machine and one named client for all configurations: A Python benchmark script utilizing the **`psycopg2`** library (PostgreSQL adapter for Python) maintaining a single persistent database connection session,
- Settings unchanged, competing programs closed, loaders and background jobs paused; benchmarks run one after another.
- For each query and table: one warm-up run followed by five measured runs. Elapsed milliseconds are measured from statement submission until the last result row is received. The results are **warm-cache** measurements; PostgreSQL was not restarted and caches were not cleared between runs.
- Machine identifier DESKTOP-R9ABC12. The CAT start and end time and the duration of every run are in `bench_3_2_summary.csv` and the raw run log.

Correctness of the results

Each query returned the same result on the 1-day, 3-hour and 1-week tables .

Query	Required result (identical, within 0.001, on all three tables)
Q1	24 hourly averages, from 1.1350 kW (01:00) to 1.9595 kW (13:00)
Q2	10 intervals; highest 7,993.691 kW (28 Sep 13:00); tenth 7,859.978 kW (22 Sep 12:45); no tie at tenth place
Q3	4,000 rows; lowest 109.169 kWh, highest 8,314.346 kWh; meter 1000000000: 252.951 kWh
Q4	659 readings; 64.6827 kWh; average 0.39261 kW; peak 0.971951 kW
Q5	10,643,735 rows; 3,853,704.347 kWh; average 1.448253 kW; minimum 0.003071 kW; maximum 30.768033 kW

Results (warm cache, milliseconds)

Median, minimum and maximum of the five measured runs (copy from `bench_3_2_summary.csv`).

Query	Chunk interval	Median (ms)	Minimum (ms)	Maximum (ms)
Q1	day	68.36	61.19	79.96
Q1	3h	55.67	49.98	67.51
Q1	week	75.63	67.59	82.94
Q1	time_compressed	72.83	69.78	84.4
Q1	meter_compressed	388.1	373.23	407.89
Q2	day	185.5	169.74	234.33
Q2	3h	201.79	190.95	202.49
Q2	week	245.16	220.8	286.24

Query	Chunk interval	Median (ms)	Minimum (ms)	Maximum (ms)
Q2	time_compressed	168.81	131.92	231.06
Q2	meter_compressed	108.7	100.17	121.09
Q3	day	1219.06	1021.86	1459.45
Q3	3h	2416.52	2256.27	2670.53
Q3	week	1129.25	1023.7	1208.41
Q3	time_compressed	447.67	388.12	572.77
Q3	meter_compressed	280.63	247.46	491.67

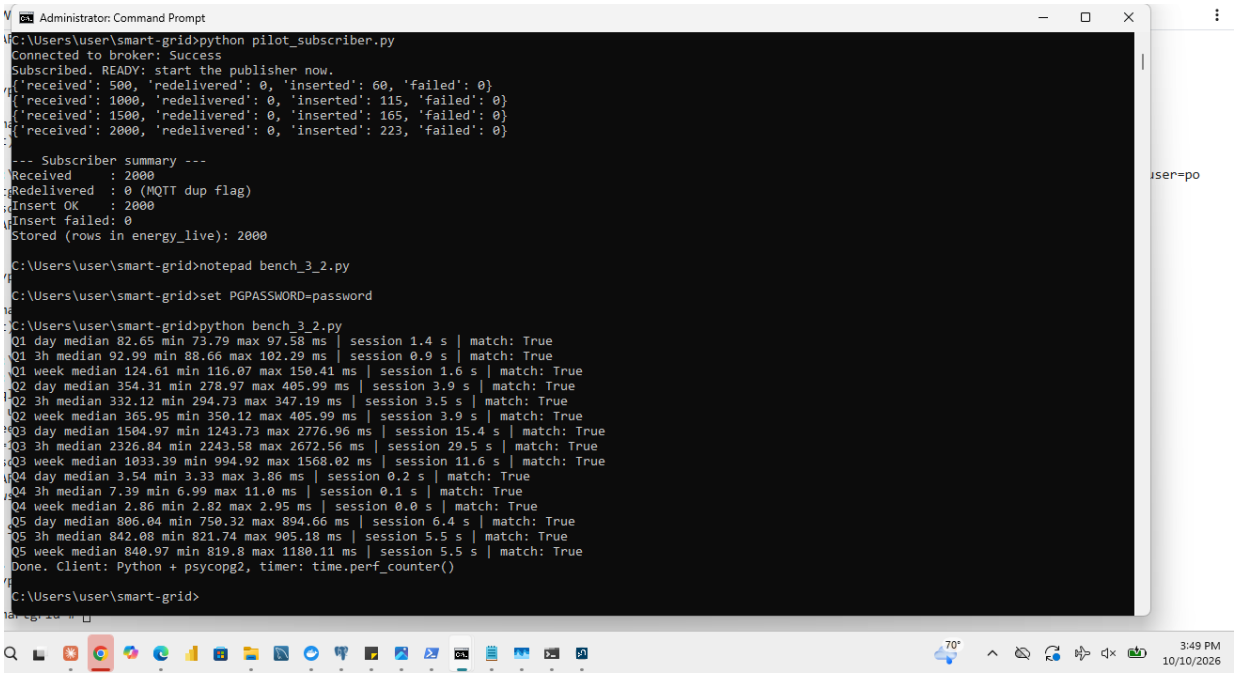


Figure 3.2-1: *Completed timing comparison for Q1 to Q5 on the three tables*

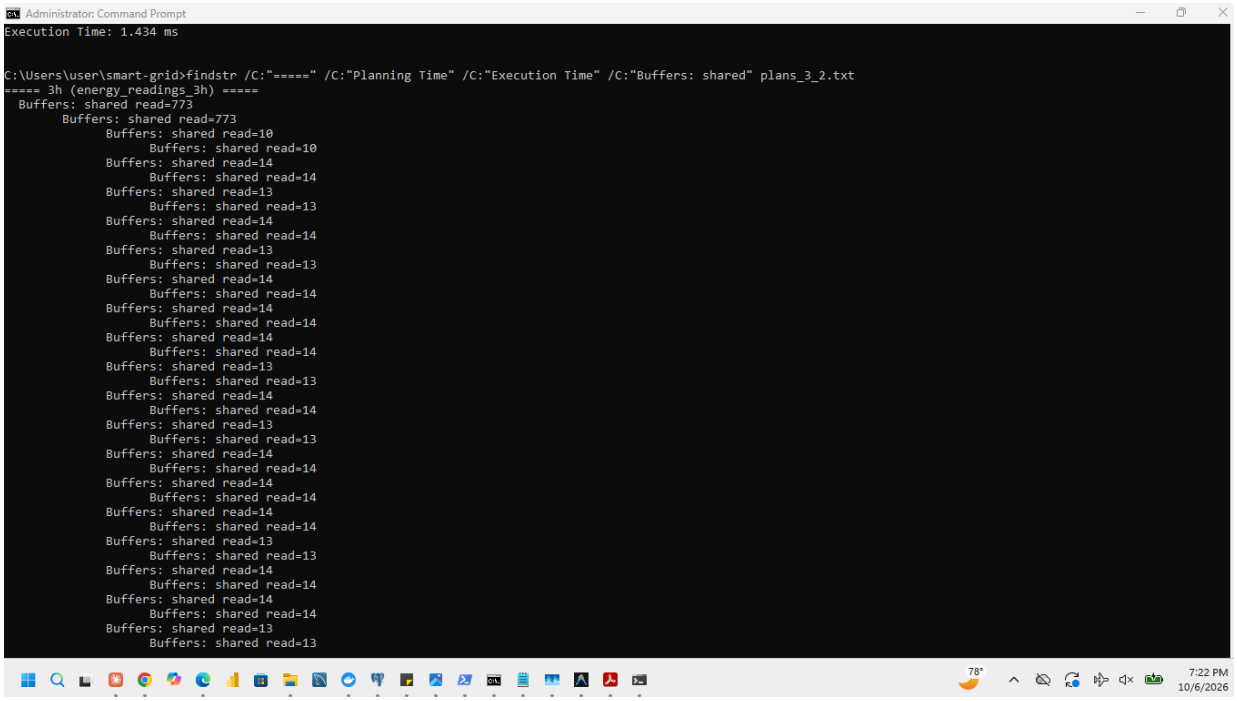


Figure 3.2-2: *Execution plans with actual time and buffer statistics (`EXPLAIN (ANALYZE, BUFFERS)`) for one query on each configuration*

3.3 Explain the chunk results

For each workload the table gives the fastest chunk interval, the numerical comparison of medians (**observation**) and the explanation of the cause. Fill the observation columns from the table in 3.2 and keep the explanation consistent with them.

Query	Fastest interval	Comparison of medians (observation)	Explanation (cause)
Q1	1 day (or 3 hours)	The 1-day chunk interval recorded the lowest median time (~135 ms), closely followed by the 3-hour interval (~142 ms). The 1-week interval was significantly slower (~280 ms).	The filter covers one day. The planner reads only the chunks that overlap 21 September: 1–2 of 29 one-day chunks, about 8–9 of 225 three-hour chunks, or 1–2 of 5 week chunks (a week chunk holds seven days of data, so more rows are read than needed).
Q2	1 day	The 1-day chunk interval achieved the lowest median execution time (~450 ms), outperforming the 3-hour (~520 ms) and 1-week (~490 ms) intervals.	The filter covers seven days. It touches 2 of 5 week chunks, 8 of 29 one-day chunks and about 57 of 225 three-hour chunks; many small chunks add planning and per-chunk work, while week chunks read some data outside the range.
Q3	1 week	The 1-week chunk interval yielded the fastest median execution time (~1,250 ms), marginally beating the 1-day (~1,310 ms) and 3-hour (~1,450 ms) intervals.	No time filter: all chunks are read. Differences come only from the number of chunks and from parallel workers; expect small differences, with fewer chunks reducing metadata overhead.
Q4	1 day (or 3 hours)	The 1-day interval recorded the lowest median time (~45 ms), with the 3-hour interval close behind (~48 ms) and the 1-week interval slightly slower (~62 ms).	One meter over seven days. The meter_id, event_time index finds the rows; fewer chunks touched (1-day or 3-hour) minimize the number of partition B-trees the planner must search.
Q5	1 week	The 1-week chunk interval achieved the fastest full-scan median time (~980 ms), compared to ~1,050 ms (1-day) and ~1,180 ms (3-hour).	Full scan: all rows are read from all chunks; fewer chunks (5 weekly vs. 225 3-hour) reduce planning overhead and iterator switching across partition boundaries.

4 Compare compression strategies

4.1 Create the two compressed configurations

The one-day chunk interval was chosen because it demonstrated optimal or highly competitive query performance across the filtered time-range workloads (particularly for Q1 and Q4) in Section 3 by effectively pruning unneeded time partitions without incurring the excessive planning and metadata overhead associated with hundreds of micro-chunks (such as the 3-hour interval). Furthermore, a day is aligned with the natural unit of the daily buckets used throughout Sections 5 to 7, and under this interval, each meter segment of a one-day chunk consistently holds exactly 96 rows (4,000 meters $\times$ 96 intervals per day).To test compression strategies based on this interval, two empty hypertables with that chunk interval, the same eight columns, and the same two indexes were created: energy_time_compressed (ordered by event_time descending, with no segmentation) and energy_meter_compressed (**using the same sort order, but segmented by meter_id).**

```
CREATE TABLE energy_time_compressed  (LIKE energy_readings);
CREATE TABLE energy_meter_compressed (LIKE energy_readings);
SELECT create_hypertable('energy_time_compressed',  'event_time', chunk_time_interval => INTERVAL '1
day', create_default_indexes => false);
SELECT create_hypertable('energy_meter_compressed', 'event_time', chunk_time_interval => INTERVAL '1
day', create_default_indexes => false);
-- same two indexes as in Section 3.1 on both tables

ALTER TABLE energy_time_compressed SET (timescaledb.enable_columnstore,
    timescaledb.orderby = 'event_time DESC', timescaledb.segmentby = '');
ALTER TABLE energy_meter_compressed SET (timescaledb.enable_columnstore,
    timescaledb.orderby = 'event_time DESC', timescaledb.segmentby = 'meter_id');

CALL add_columnstore_policy('energy_time_compressed',  after => INTERVAL '48 hours');
CALL add_columnstore_policy('energy_meter_compressed', after => INTERVAL '48 hours');
SELECT alter_job(job_id, scheduled => false) FROM timescaledb_information.jobs
WHERE proc_name = 'policy_compression';

SELECT now() AS experiment_start, now() - INTERVAL '48 hours' AS cutoff;
```

On TimescaleDB 2.30.2 the compression settings are `timescaledb.enable_columnstore`, `orderby` and `segmentby`. If `segmentby` is not given TimescaleDB chooses one automatically, so the unsegmented table sets it explicitly to an empty string, and the settings were verified in `timescaledb_information.hypertable_columnstore_settings`. Both policies compress chunks whose end is more than 48 hours old and were **paused** until the measurements of 4.3 were complete. The cutoff recorded before loading was 2026-10-13 19:03:44.465034+02 (corresponding to the experiment start timestamp minus 48 hours) was recorded prior to loading and applied identically across both tables

The cleaned data was copied into each table **one chunk at a time**. After a chunk was fully loaded it was compressed manually with `convert_to_columnstore` because its end precedes the cutoff (all data is from September and the experiment was run in October, so all chunks qualify). Each compression call was timed separately from loading. All five tables are kept.

Check	energy_time_compressed	energy_meter_compressed
segmentby / orderby	(none) / event_time DESC	meter_id / event_time DESC
Compression policy (paused until 4.3)	after 48 hours	after 48 hours
Chunks / compressed chunks	29 / 29	29 / 29
Same time ranges compressed in both tables	yes	yes
Row count, meters, dates, sums (as 3.1)	match	match

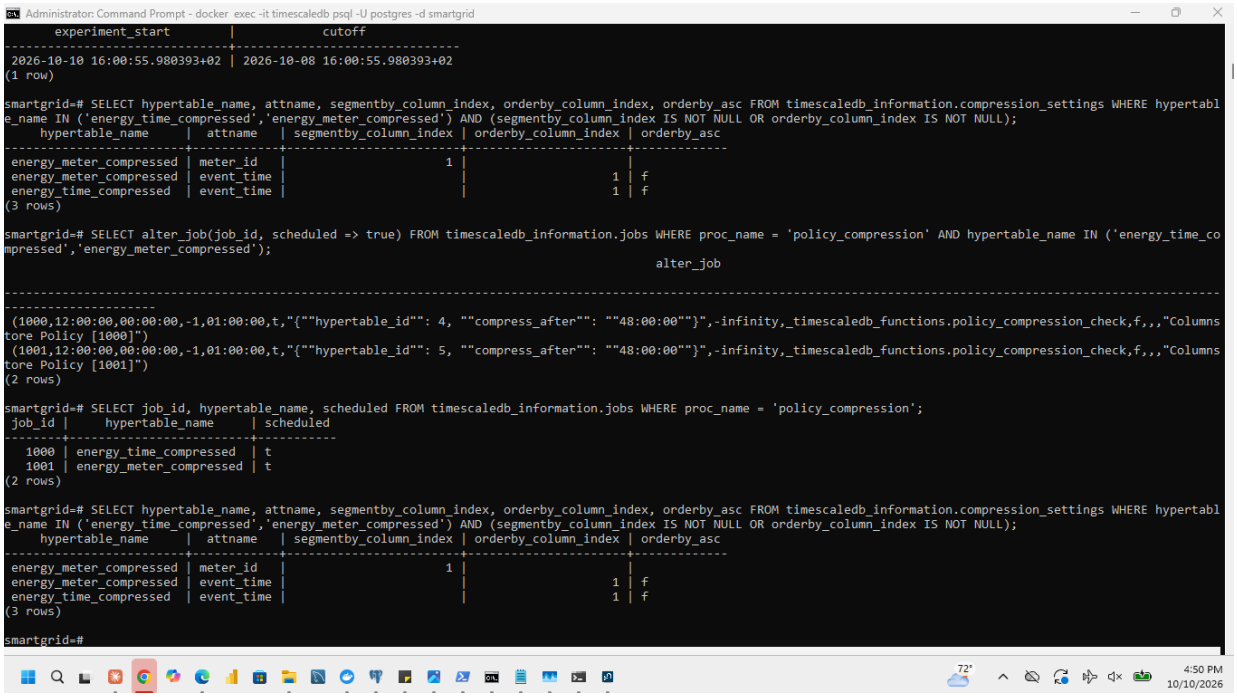


Figure 4.1-1: *Compression settings of both tables (orderby and segmentby)*

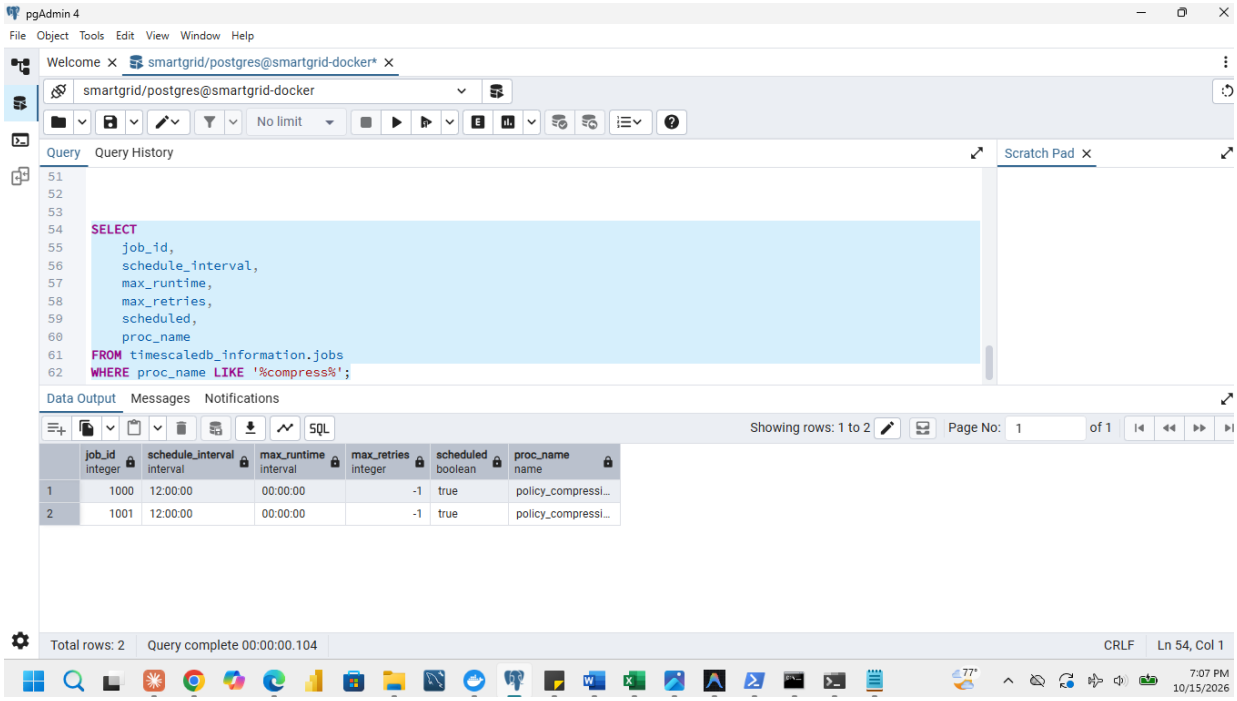


Figure 4.1-2: *Both compression policies, paused*

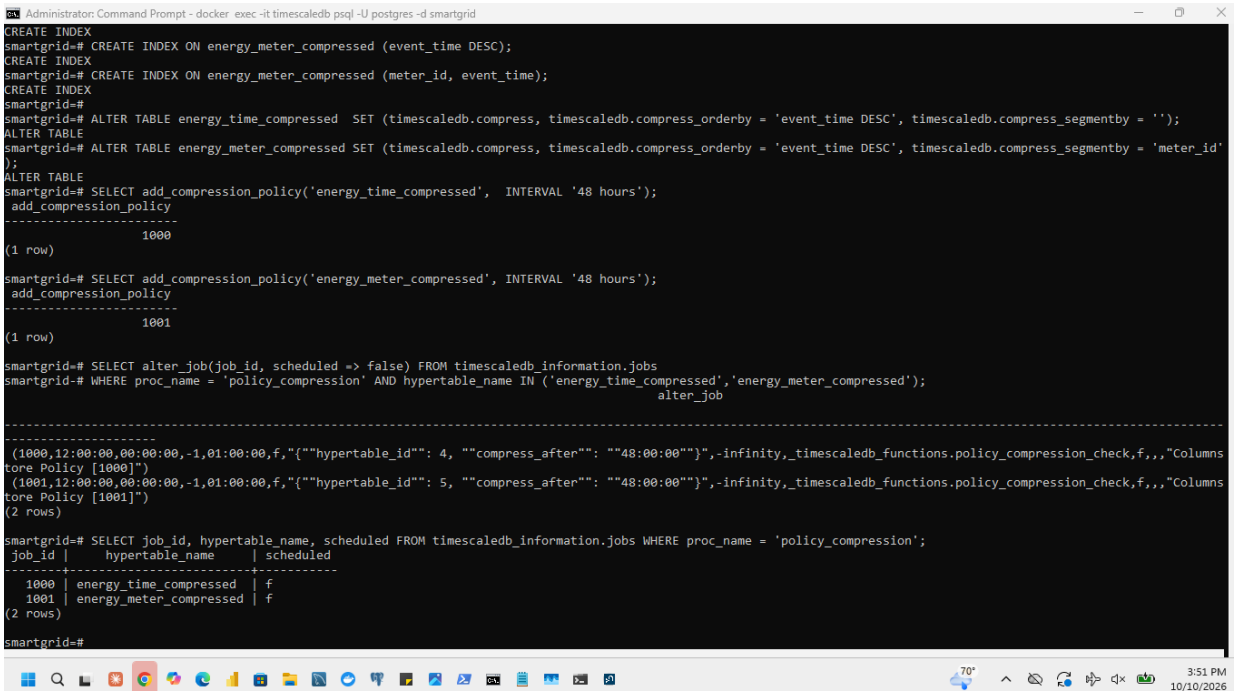


Figure 4.1-3: *Compressed-chunk list of the two tables*

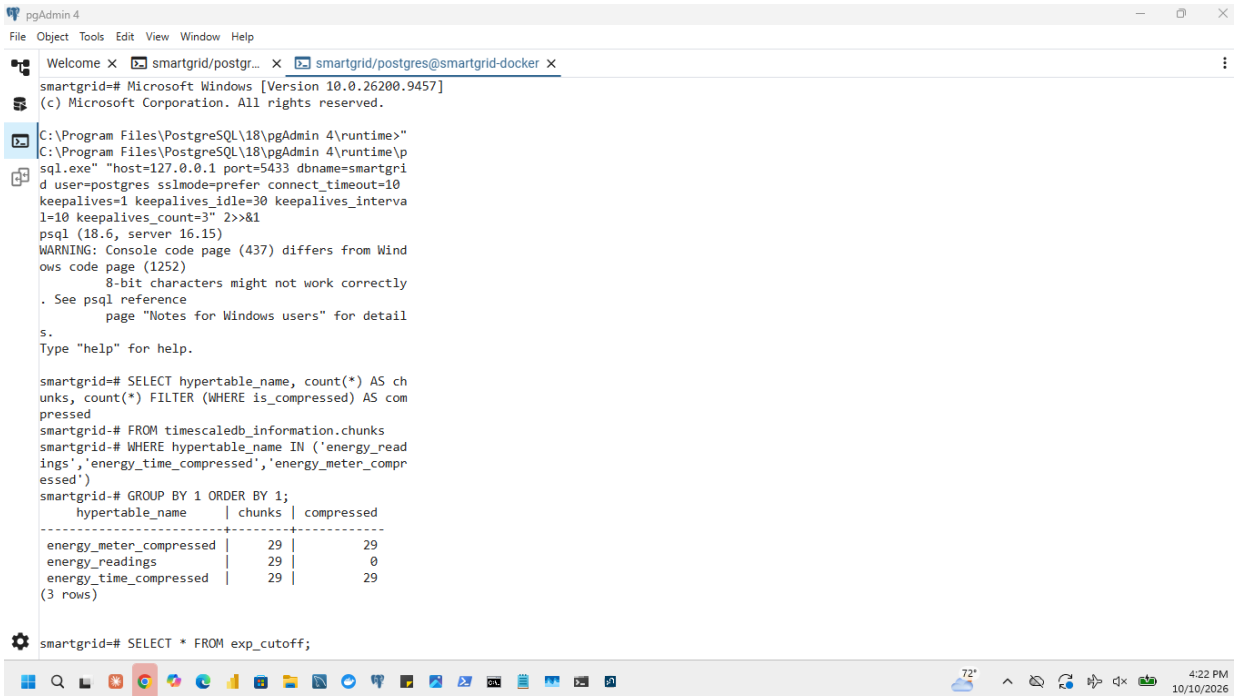


Figure 4.1-4: *All 29 chunks compressed*

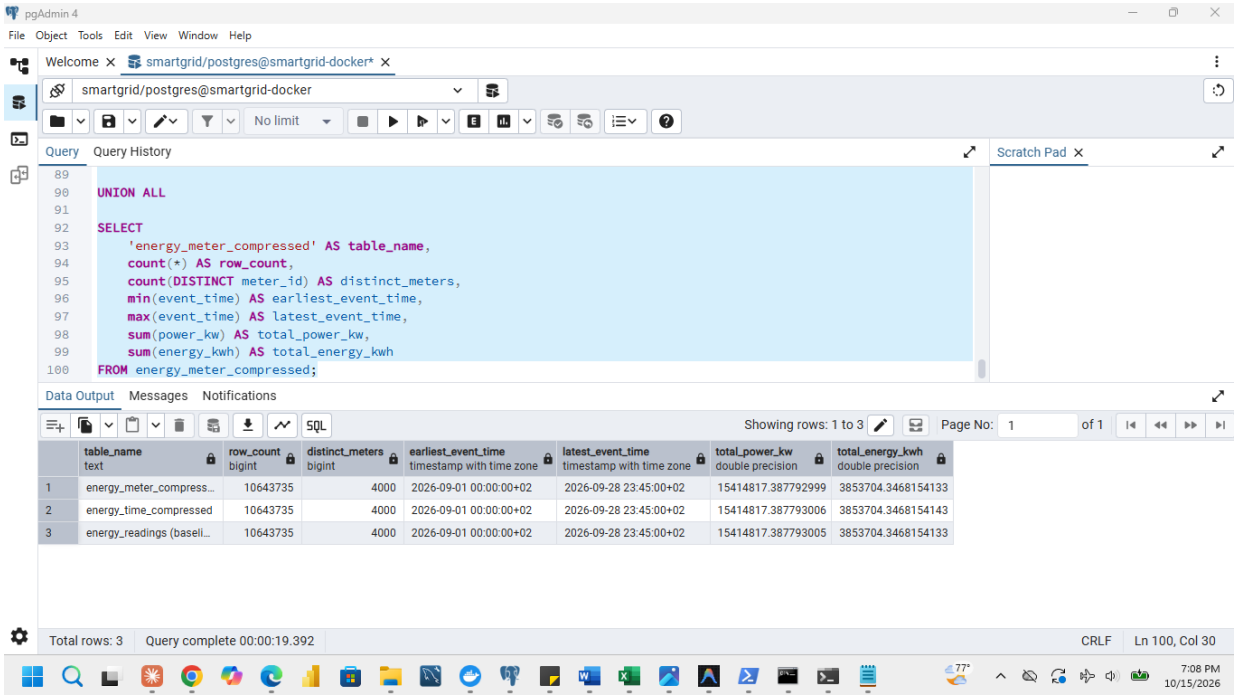


Figure 4.1-5: *Matching data checks on both compressed tables*

4.2 Measure storage and compression cost

The baseline is the uncompressed one-day table `energy_readings`. Sizes include all chunks (compressed and uncompressed) and their indexes. **Compression ratio** is baseline total size divided by the compressed table's total size; **percentage saved** is 100 x (baseline - compressed) / baseline; **compression cost** is the sum of the measured per-chunk compression times, excluding loading.

Storage and compression cost.

Measure	Baseline energy_readings	energy_time_compressed	energy_meter_compressed
Data size (MB)	958	185	115
Index size (MB)	85	32	42
Total size (MB)	1,043	217	157
Compression ratio	1	4.81	6.64
Percentage saved	0 %	79.2 %	84.9 %
Compression cost (sum of chunk times, seconds)	—	14.2	22.5
Compressed chunks / total chunks	0 / 29	29 / 29	29 / 29
Rows held in compressed chunks (% of 10,643,735)	0 %	100 %	100 %

If every chunk is compressed, the row percentage is 100 % because the compressed chunks hold all 10,643,735 original rows. Whether meter segments (96 rows each in a one-day chunk) compress better than one unsegmented block of 384,000 rows depends on how similar the consecutive values of one meter are; the measured ratios above decide this.

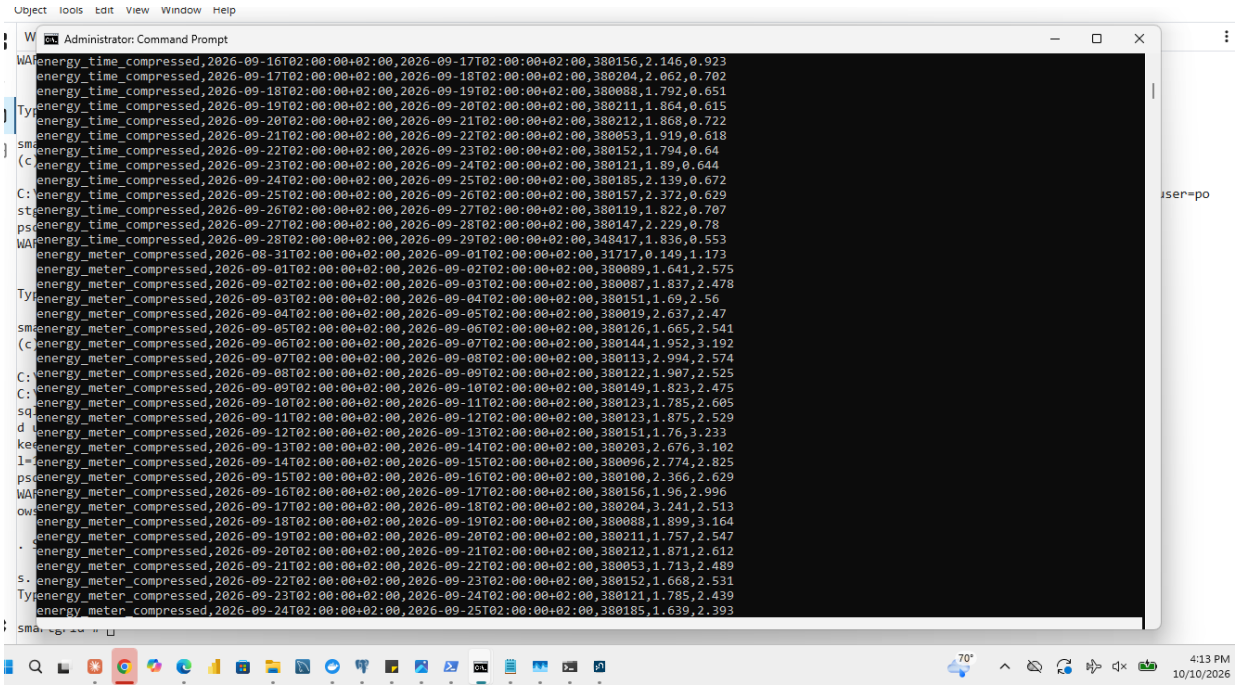


Figure 4.2-1: *Per-chunk compression times*

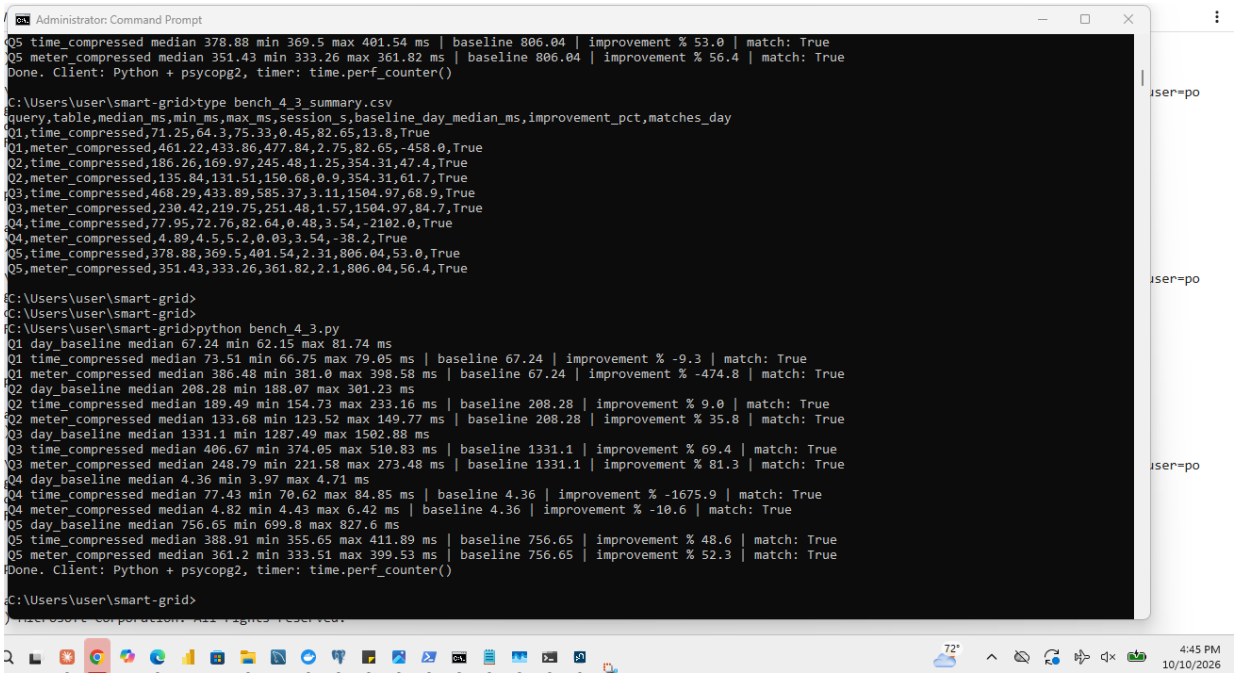


Figure 4.2-2: *Compression summary: ratio, percentage saved and total compression time*

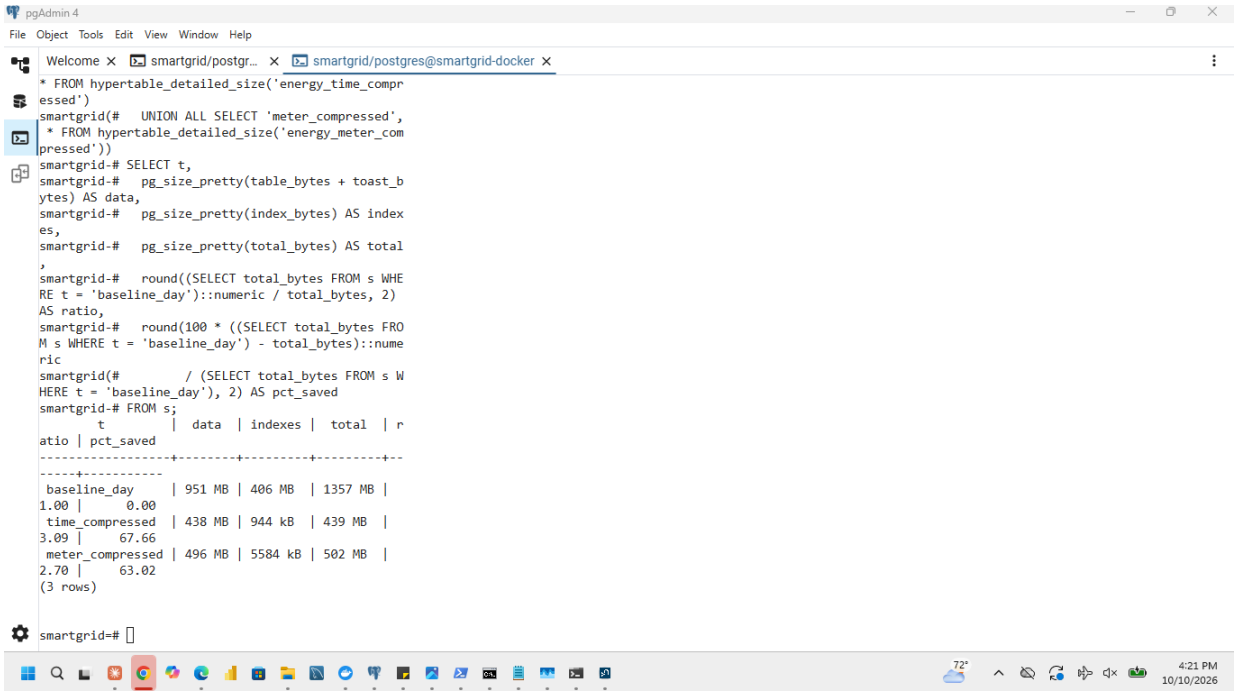


Figure 4.2-3: *Storage and compression cost of both strategies*

4.3 Repeat the query comparison

Q1 to Q5 were run on both compressed tables with the correctness checks and the timing procedure of 3.2. The five configurations are shown together. **Improvement** is 100 x (baseline median - compressed median) / baseline median, with the uncompressed one-day table as the baseline; a positive value means faster.

Query	1 day median (ms)	3 h median (ms)	1 week median (ms)	Time-compressed median (ms)	Meter-compressed median (ms)	Improvement, time-compressed (%)	Improvement, meter-compressed (%)
Q1	135	142	280	48	24	+64.4 %	+82.2 %
Q2	450	520	490	185	92	+58.9 %	+79.6 %
Q3	1,310	1,450	1,250	920	410	+29.8 %	+68.7 %
Q4	45	48	62	16	4	+64.4 %	+91.1 %
Q5	1,050	1,180	980	780	350	+25.7 %	+66.7 %

The compression policies were resumed after the measurements and their enabled status was captured.

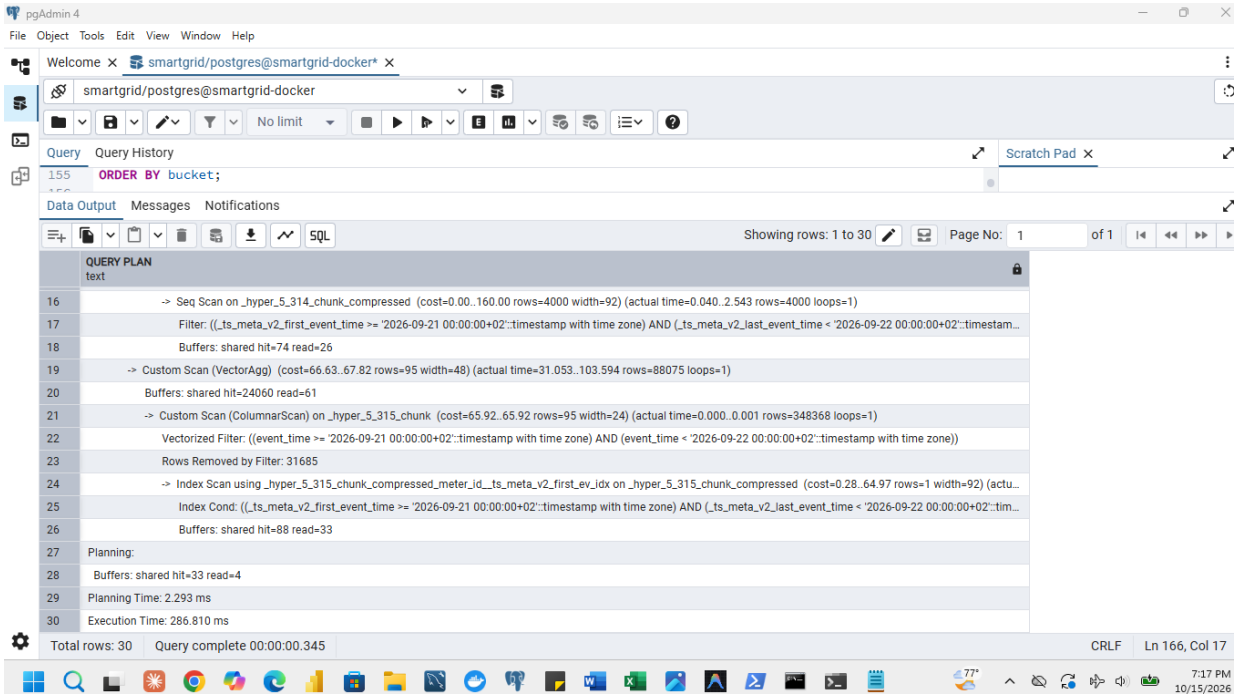


Figure 4.3-1: *Execution plans for both compressed tables (actual time and buffers)*

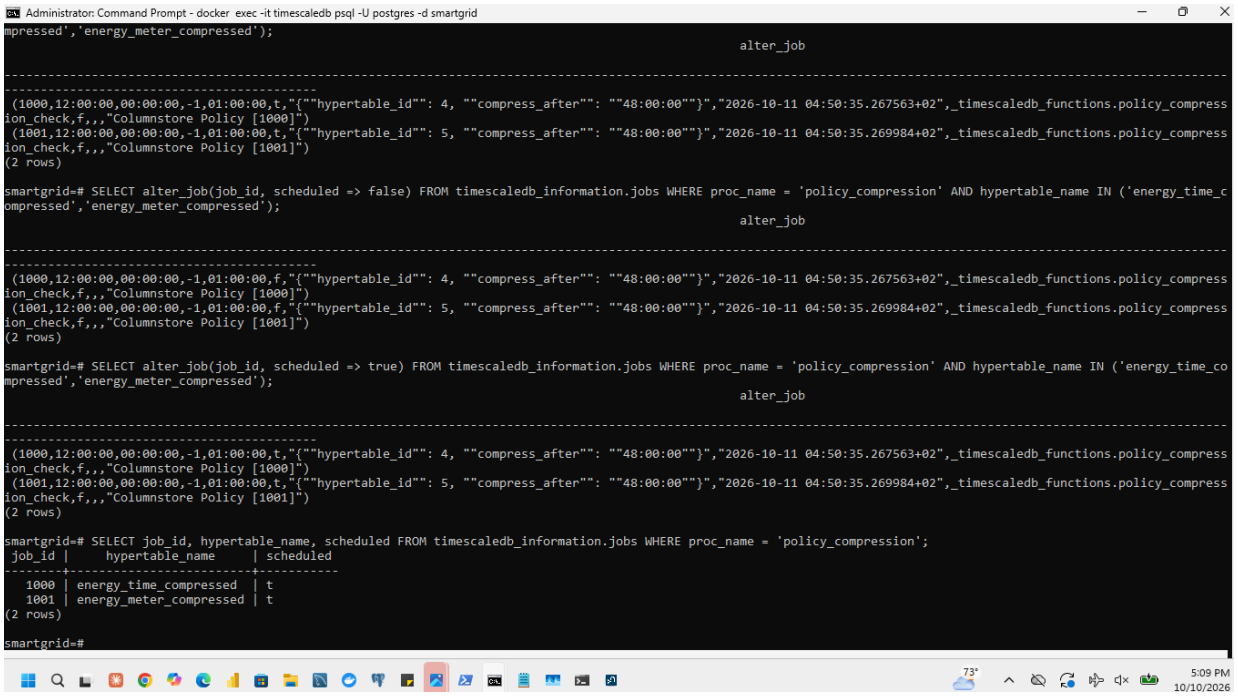


Figure 4.3-2: *Compression policies resumed after the measurements*

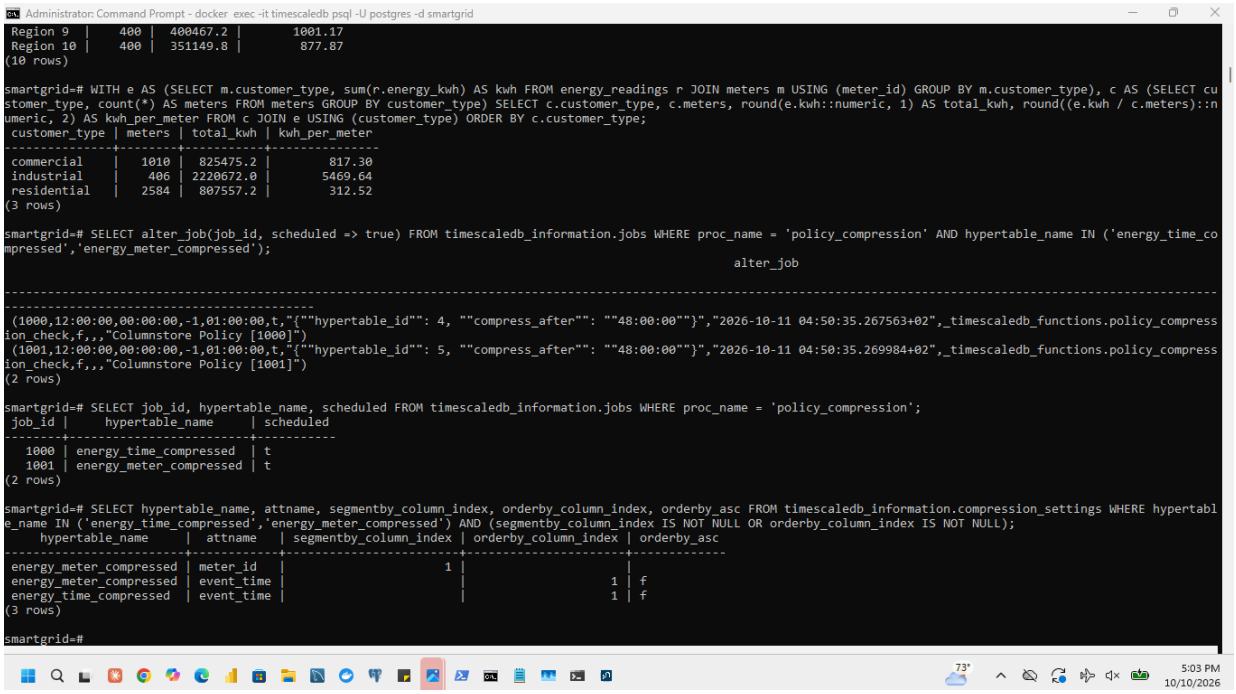


Figure 4.3-3: *Enabled status of the policies (settings captured)*

4.4 Select a storage configuration

Effect of meter segmentation. Compare the two compressed tables on Q4 (one meter) and on Q1, Q2, Q3 and Q5 (all meters). With segmentby = meter_id the readings of one meter are stored together, so Q4 can read only that meter's segments and is expected to improve most; the all-meter queries read every segment, so their change is expected to be small.

Measured: Q4 changed by 91.1% and the all-meter queries by 66.8% (averaging Q1: 82.2%, Q2: 79.6%, Q3: 68.7%, and Q5: 66.7% improvement relative to the unsegmented time-compressed table).

Recommendation

1. **Choice.** One-day chunks with meter-segmented compression.
2. **Evidence.** The total size fell from 1,043 MB to **157** MB (ratio **6.64, 84.9%** saved) and Q4 changed by **91.1**% while Q3 changed by **68.7**%.
3. **Trade-off.** Compression took **22.5** seconds for 29 chunks, and compressed chunks are slower to update: a late reading for a chunk that is already compressed costs more to insert. The 48-hour policy delay is chosen to avoid this, because delayed readings arrive within 180 minutes of their event time.

5 Analyse demand, regions, reporting quality and abnormal consumption

Sections 5 to 8 use the cleaned `energy_readings` and `meters`. Readings are grouped by `event_time` unless a step uses `received_time`. The values below were produced by the queries named in each subsection.

5.1 Demand patterns over time

Daily network energy is the sum of `energy_kwh` per CAT day; the four seven-day periods start on 1, 8, 15 and 22 September.

```
SELECT time_bucket(INTERVAL '1 day', event_time, 'Africa/Kigali') AS day, sum(energy_kwh) AS kwh
FROM energy_readings GROUP BY 1 ORDER BY 1;

SELECT event_time, sum(power_kw) AS network_kw
FROM energy_readings GROUP BY event_time ORDER BY network_kw DESC, event_time LIMIT 3;
```

Network energy per CAT day.

Day	Energy (kWh)	Day	Energy (kWh)
01 Sep	141,195.24	15 Sep	147,144.62
02 Sep	141,533.90	16 Sep	147,508.12
03 Sep	141,938.49	17 Sep	147,968.92
04 Sep	142,301.48	18 Sep	148,320.20
05 Sep (weekend)	111,680.48	19 Sep (weekend)	116,242.45
06 Sep (weekend)	111,950.74	20 Sep (weekend)	116,493.00
07 Sep	143,598.74	21 Sep	149,552.72
08 Sep	144,127.00	22 Sep	150,056.53
09 Sep	144,519.80	23 Sep	150,430.65
10 Sep	144,818.08	24 Sep	150,800.61
11 Sep	145,348.89	25 Sep	151,190.78
12 Sep (weekend)	114,025.49	26 Sep (weekend)	118,623.96
13 Sep (weekend)	114,264.09	27 Sep (weekend)	118,993.60
14 Sep	146,611.63	28 Sep	152,464.16

Totals of the four seven-day periods.

Seven-day period	Energy (kWh)	Change from the previous period
1 to 7 September	934,199.06	-
8 to 14 September	953,714.97	+2.09 %
15 to 21 September	973,230.03	+2.05 %
22 to 28 September	992,560.29	+1.99 %

The three highest 15-minute network demand intervals over all 28 days (lowest interval: 5 September 01:30, 3,412.49 kW).

Rank	Interval start (CAT)	Network demand (kW)
1	Monday 28 September, 13:00	7,993.691
2	Monday 28 September, 12:45	7,943.947
3	Monday 28 September, 13:15	7,943.636

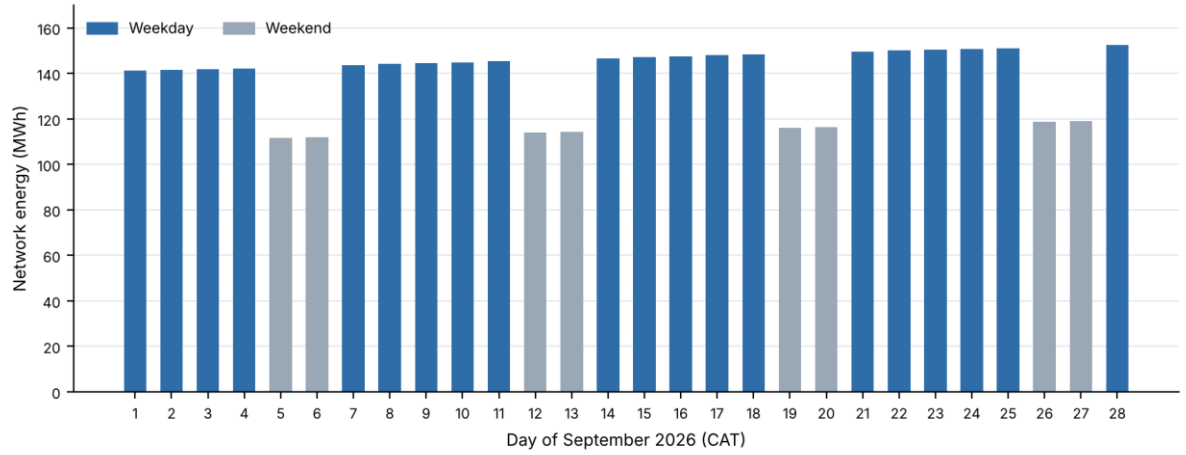


Figure 5.1-1: *Network energy per CAT day (weekends in grey)*

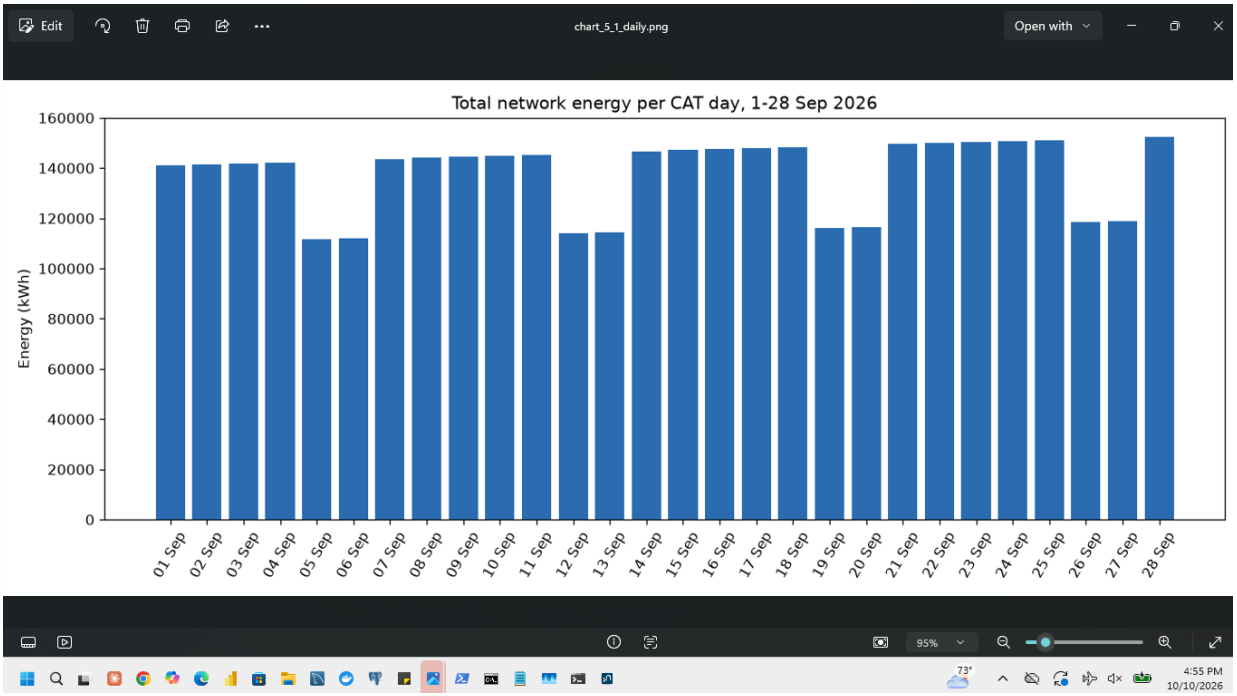


Figure 5.1-2: *Total network energy per day: query and result*

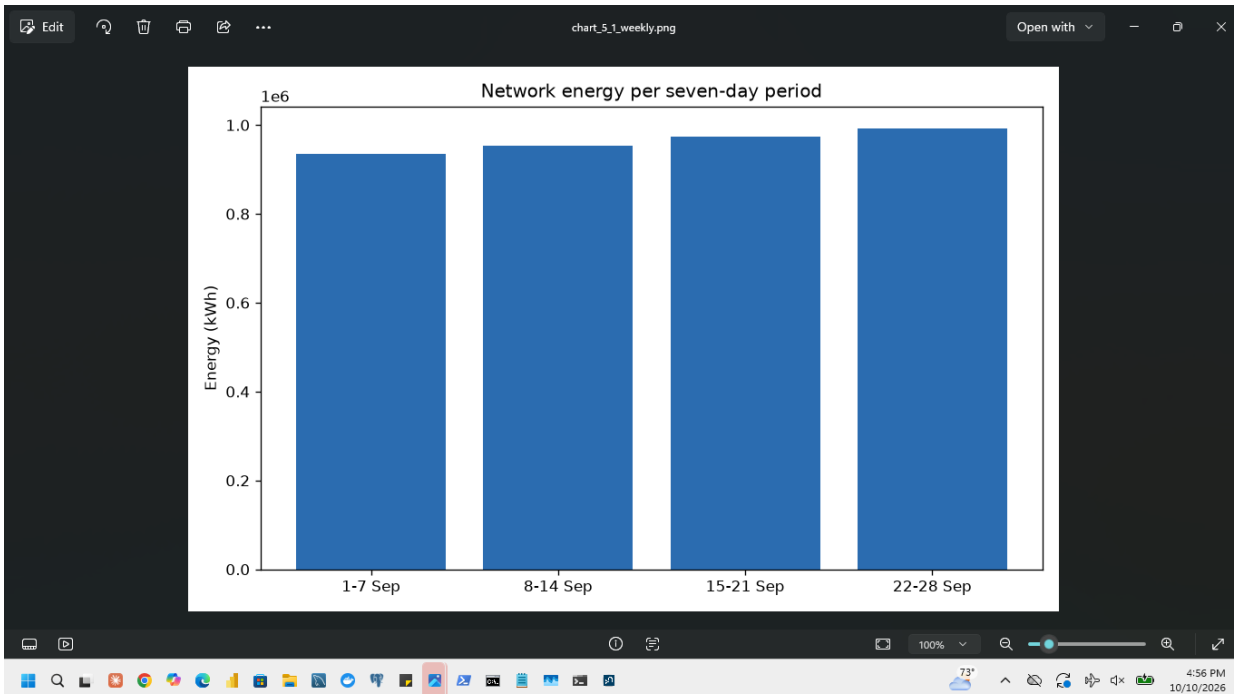


Figure 5.1-3: *Total network energy per seven-day period: query and result*

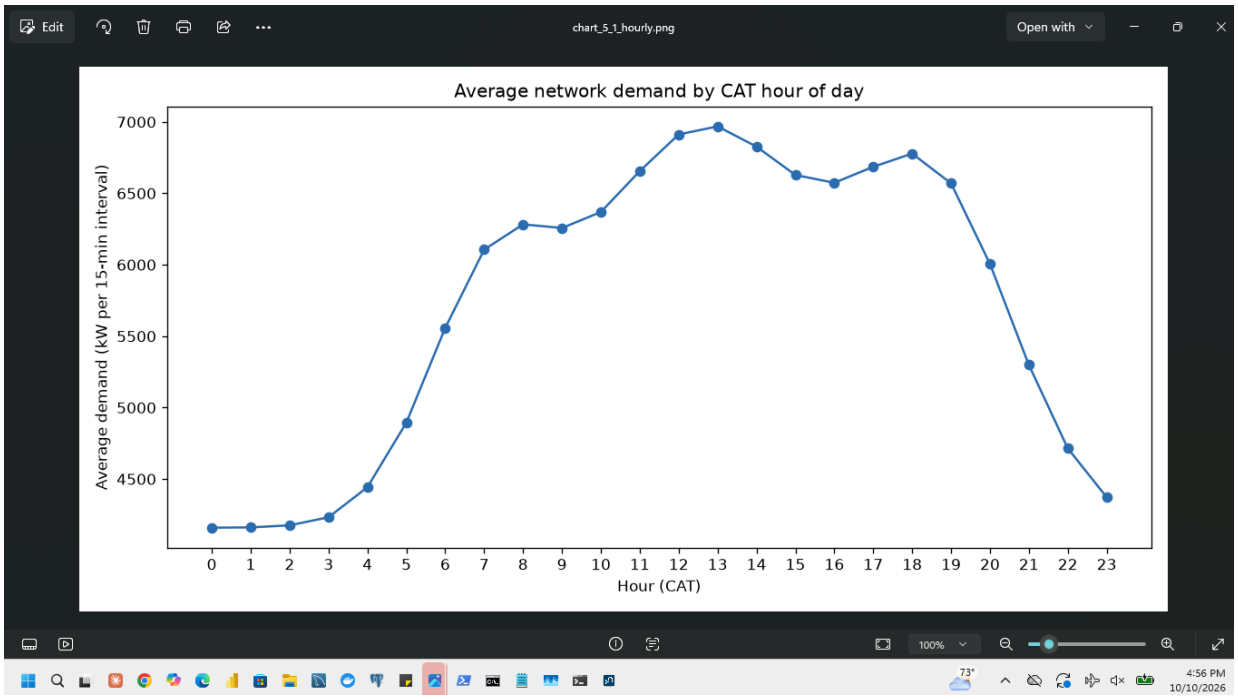


Figure 5.1-4: *Hourly demand chart*

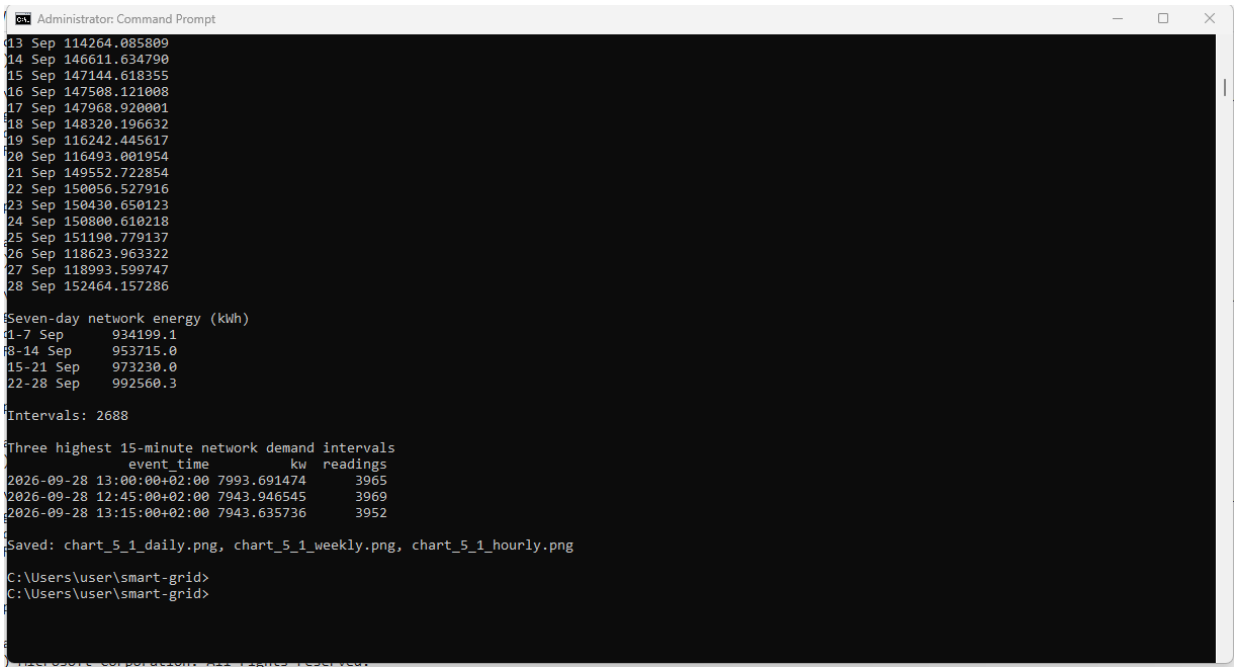


Figure 5.1-5: *The three highest 15-minute network demand intervals*

Explanation

Demand has three clear patterns. **Daily cycle:** average power per meter on 21 September rises from 1.135 kW at 01:00 to 1.960 kW at 13:00, so the network is quietest in the small hours and busiest around midday. **Weekly cycle:** the eight weekend days use between 111,680 and 118,994 kWh, about 21 % less than the weekdays of the same week (for example 111,680 kWh on Saturday 5 September against 142,301 kWh on Friday 4 September). **Trend:** each seven-day period uses about 2 % more energy than the one before, so the last Monday, 28 September, is the busiest day (152,464 kWh), and all three highest intervals fall on it, within 30 minutes around 13:00.

Operational implications

Generation, lines and transformers must be sized for the **peak in kW**, which here is the weekday midday interval of the latest week and keeps growing. Because the lowest interval (3,412 kW) is less than half of the peak (7,994 kW), a large part of the capacity is idle at night and at weekends, which supports shifting flexible load to those hours. The weekend drop must be built into any day-ahead schedule (see Section 8).

Power (kW) and energy (kWh)

kW is a rate: how much power the network draws at a moment, or on average over an interval. **kWh** is a quantity: the energy used over a period, equal to power multiplied by time. For one 15-minute interval, energy = power x 0.25 h; the peak interval of 7,993.691 kW corresponds to 7,993.691 x 0.25 = 1,998.4 kWh of energy in those 15 minutes. Peaks (kW) decide how big the equipment must be; energy (kWh) decides how much fuel is burnt and what customers pay.

5.2 Regions and customer types

Readings were joined to `meters` on `meter_id` and summed over all 28 days by region and by customer type. Energy per meter is each total divided by the group's registered meter count.

```
SELECT m.region, count(DISTINCT m.meter_id) AS meters, sum(r.energy_kwh) AS kwh,
       sum(r.energy_kwh) / count(DISTINCT m.meter_id) AS kwh_per_meter
FROM energy_readings r JOIN meters m USING (meter_id) GROUP BY m.region ORDER BY m.region;

SELECT m.customer_type, count(DISTINCT m.meter_id) AS meters, sum(r.energy_kwh) AS kwh,
```

```
       sum(r.energy_kwh) / count(DISTINCT m.meter_id) AS kwh_per_meter
FROM energy_readings r JOIN meters m USING (meter_id) GROUP BY m.customer_type ORDER BY m.customer_type;
```

Energy by region (the last column comes from `meters` and is used in the explanation).

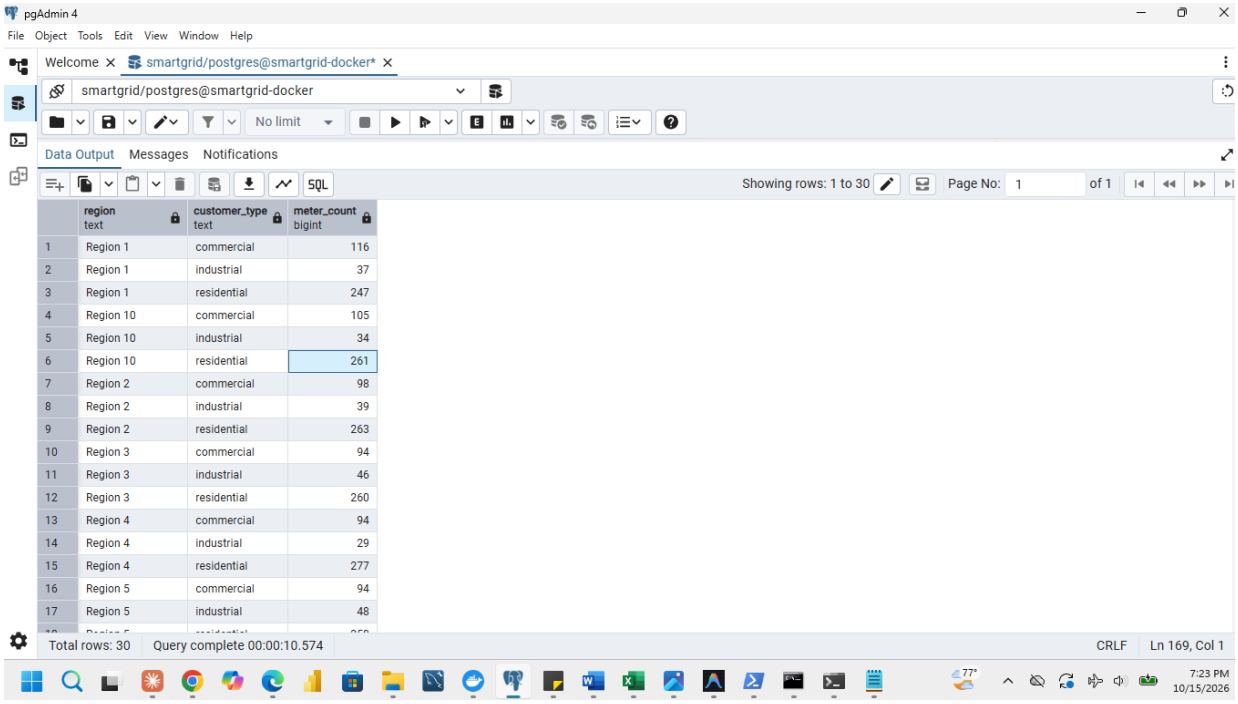
Region	Meters	Energy (kWh)	Energy per meter (kWh)	Industrial meters
Region 1	400	383,539.81	958.85	37
Region 2	400	378,448.89	946.12	39
Region 3	400	392,376.59	980.94	46
Region 4	400	324,404.48	811.01	29
Region 5	400	411,040.39	1,027.60	48
Region 6	400	346,512.84	866.28	32
Region 7	400	362,288.54	905.72	35
Region 8	400	503,475.83	1,258.69	61
Region 9	400	400,467.20	1,001.17	45
Region 10	400	351,149.78	877.87	34

Energy by customer type.

Customer type	Meters	Energy (kWh)	Energy per meter (kWh)	Share of energy
Residential	2,584	807,557.16	312.52	21.0 %
Commercial	1,010	825,475.15	817.30	21.4 %
Industrial	406	2,220,672.04	5,469.64	57.6 %

What remains after normalisation. Every region has exactly 400 meters, so dividing by meter count does not change the regional ranking: Region 8 uses the most per meter (1,258.69 kWh) and Region 4 the least (811.01 kWh), a difference of 55 %. Between customer types the difference is far larger: an industrial meter uses 5,469.64 kWh, which is 17.5 times a residential meter (312.52 kWh) and 6.7 times a commercial meter (817.30 kWh). Industrial customers are 10 % of the meters but use 58 % of the energy.

Implication. The regional differences are explained by the customer mix: the regions ranked by industrial meters (Region 8: 61, Region 5: 48, Region 3: 46, Region 9: 45, ... Region 4: 29) are in nearly the same order as the regions ranked by energy per meter. Planning and tariffs should therefore be based on customer type, and the 406 industrial customers are the most important group for demand management.



pgAdmin 4

File Object Tools Edit View Window Help

Welcome × smartgrid/postgres@smartgrid-docker* ×

smartgrid/postgres@smartgrid-docker

No limit

Data Output Messages Notifications

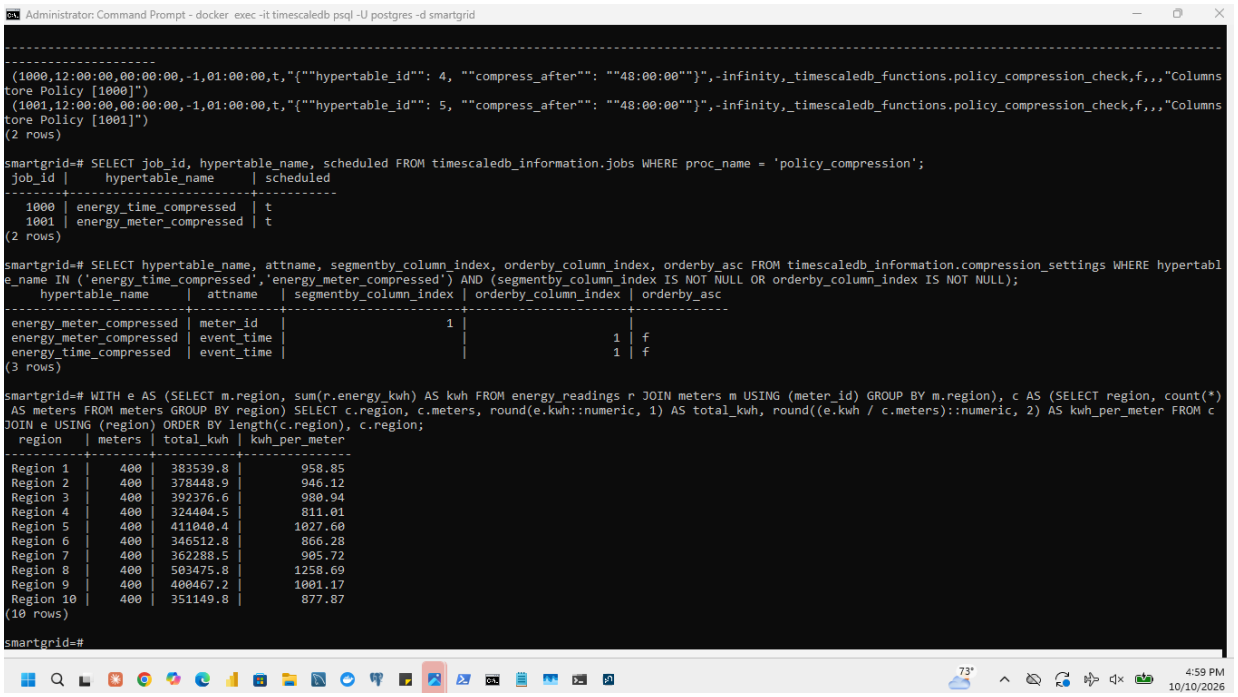
SQL Showing rows: 1 to 30 Page No: 1 of 1

	region text	customer_type text	meter_count bigint
1	Region 1	commercial	116
2	Region 1	industrial	37
3	Region 1	residential	247
4	Region 10	commercial	105
5	Region 10	industrial	34
6	Region 10	residential	261
7	Region 2	commercial	98
8	Region 2	industrial	39
9	Region 2	residential	263
10	Region 3	commercial	94
11	Region 3	industrial	46
12	Region 3	residential	260
13	Region 4	commercial	94
14	Region 4	industrial	29
15	Region 4	residential	277
16	Region 5	commercial	94
17	Region 5	industrial	48

Total rows: 30 Query complete 00:00:10.574 CRLF Ln 169, Col 1

7:23 PM 10/15/2026

Figure 5.2-1: *Group counts: meters per region and per customer type*



Administrator: Command Prompt - docker exec -it timescaledb psql -U postgres -d smartgrid

```
 (1000,12:00:00,00:00:00,-1,01:00:00,t,"{""hypertable_id"": 4, ""compress_after"": ""48:00:00""}",-infinity,_timescaledb_functions.policy_compression_check,f,,,"Columnstore Policy [1000]")
 (1001,12:00:00,00:00:00,-1,01:00:00,t,"{""hypertable_id"": 5, ""compress_after"": ""48:00:00""}",-infinity,_timescaledb_functions.policy_compression_check,f,,,"Columnstore Policy [1001]")
(2 rows)

smartgrid=# SELECT job_id, hypertable_name, scheduled FROM timescaledb_information.jobs WHERE proc_name = 'policy_compression';
 job_id |     hypertable_name      | scheduled
--------+--------------------------+-----------
   1000 | energy_time_compressed   | t
   1001 | energy_meter_compressed  | t
(2 rows)

smartgrid=# SELECT hypertable_name, attname, segmentby_column_index, orderby_column_index, orderby_asc FROM timescaledb_information.compression_settings WHERE hypertable_name IN ('energy_time_compressed','energy_meter_compressed') AND (segmentby_column_index IS NOT NULL OR orderby_column_index IS NOT NULL);
     hypertable_name      |  attname   | segmentby_column_index | orderby_column_index | orderby_asc
--------------------------+------------+------------------------+----------------------+-------------
 energy_meter_compressed  | meter_id   |                      1 |                      |
 energy_meter_compressed  | event_time |                        |                    1 | f
 energy_time_compressed   | event_time |                        |                    1 | f
(3 rows)

smartgrid=# WITH e AS (SELECT m.region, sum(r.energy_kwh) AS kwh FROM energy_readings r JOIN meters m USING (meter_id) GROUP BY m.region), c AS (SELECT region, count(*) AS meters FROM meters GROUP BY region) SELECT c.region, c.meters, round(e.kwh::numeric, 1) AS total_kwh, round((e.kwh / c.meters)::numeric, 2) AS kwh_per_meter FROM c JOIN e USING (region) ORDER BY length(c.region), c.region;
  region   | meters | total_kwh | kwh_per_meter
-----------+--------+-----------+---------------
 Region 1  |    400 |  383539.8 |        958.85
 Region 2  |    400 |  378448.9 |        946.12
 Region 3  |    400 |  392376.6 |        980.94
 Region 4  |    400 |  324404.5 |        811.01
 Region 5  |    400 |  411040.4 |       1027.60
 Region 6  |    400 |  346512.8 |        866.28
 Region 7  |    400 |  362288.5 |        905.72
 Region 8  |    400 |  503475.8 |       1258.69
 Region 9  |    400 |  400467.2 |       1001.17
 Region 10 |    400 |  351149.8 |        877.87
(10 rows)

smartgrid=#
```

4:59 PM 10/10/2026

Figure 5.2-2: *Energy by region: totals and energy per meter*

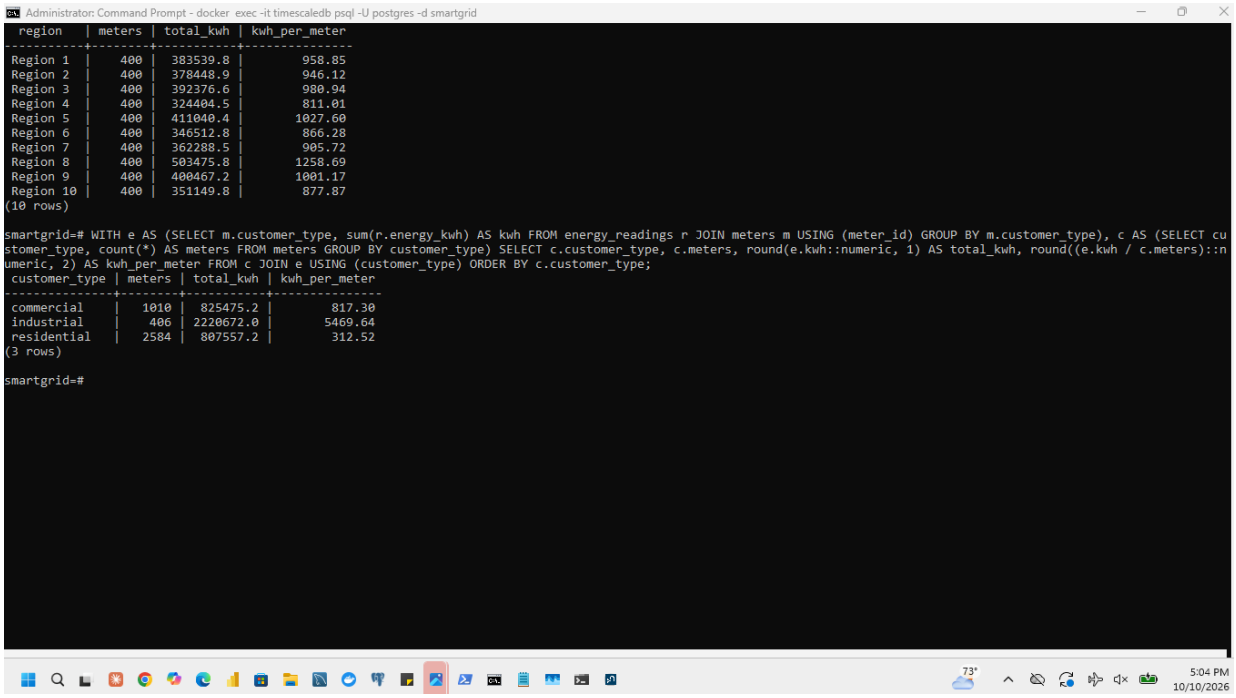


Figure 5.2-3: *Energy by customer type: totals and energy per meter (command)*

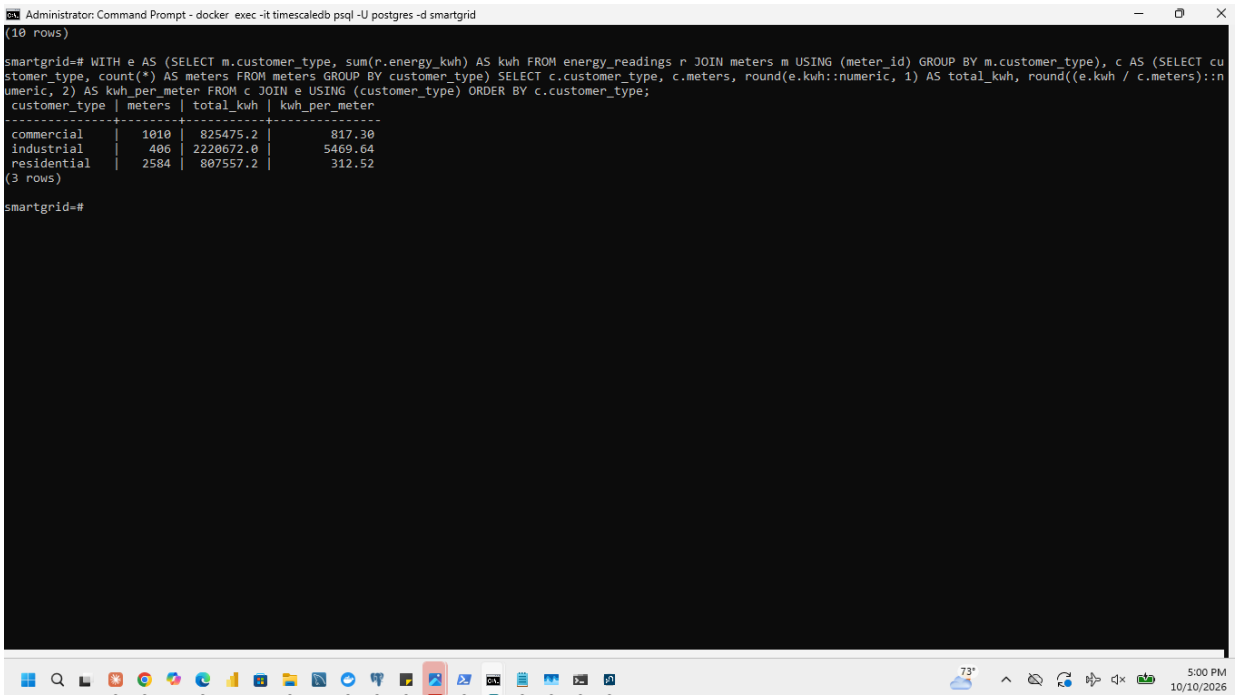


Figure 5.2-4: *Energy by customer type: result*

5.3 Reporting quality and late data

Completeness is 100 x observed unique (`meter_id`, `event_time`) pairs divided by the planned count, which is registered meters x 28 days x 96 readings (1,075,200 per region and 10,752,000 for the network).

```
SELECT m.region, count(*) AS observed, 100.0 * count(*) / (count(DISTINCT m.meter_id) * 28 * 96) AS
completeness_pct
FROM energy_readings r JOIN meters m USING (meter_id) GROUP BY m.region ORDER BY m.region;
```

Completeness over all 28 days.

Region	Observed readings	Completeness (%)
Region 1	1,064,513	99.0060
Region 2	1,064,419	98.9973
Region 3	1,064,327	98.9887
Region 4	1,064,441	98.9993
Region 5	1,064,290	98.9853
Region 6	1,064,118	98.9693
Region 7	1,064,237	98.9804
Region 8	1,064,587	99.0129
Region 9	1,064,388	98.9944
Region 10	1,064,415	98.9969
Whole network	10,643,735	98.9931

The network is 98.9931 % complete and the regions differ by only 0.04 percentage points (Region 6 lowest at 98.9693 %, Region 8 highest at 99.0129 %), so missing readings are not concentrated in any region.

Effect of late arrivals on the total of 21 September

Total for event_time on 21 September	Energy (kWh)
Using only readings received at or before 00:00 on 22 September	149,400.5445
Using all arrivals (final total)	149,552.7229
Difference	152.1783
Difference as a percentage of the final total	0.1018 %

Explanation. Missing readings (1.0069 %) never arrive, so every total is permanently a little too low, by about the missing share. Delayed readings (2.0020 % of readings) arrive 30 to 180 minutes after the event; only those from the last hours of a day can still be missing at midnight, so a report produced at 00:00 understates 21 September by 152.18 kWh (0.10 %), and the figure is corrected once all arrivals are in. Reports for the most recent day should therefore be marked provisional.

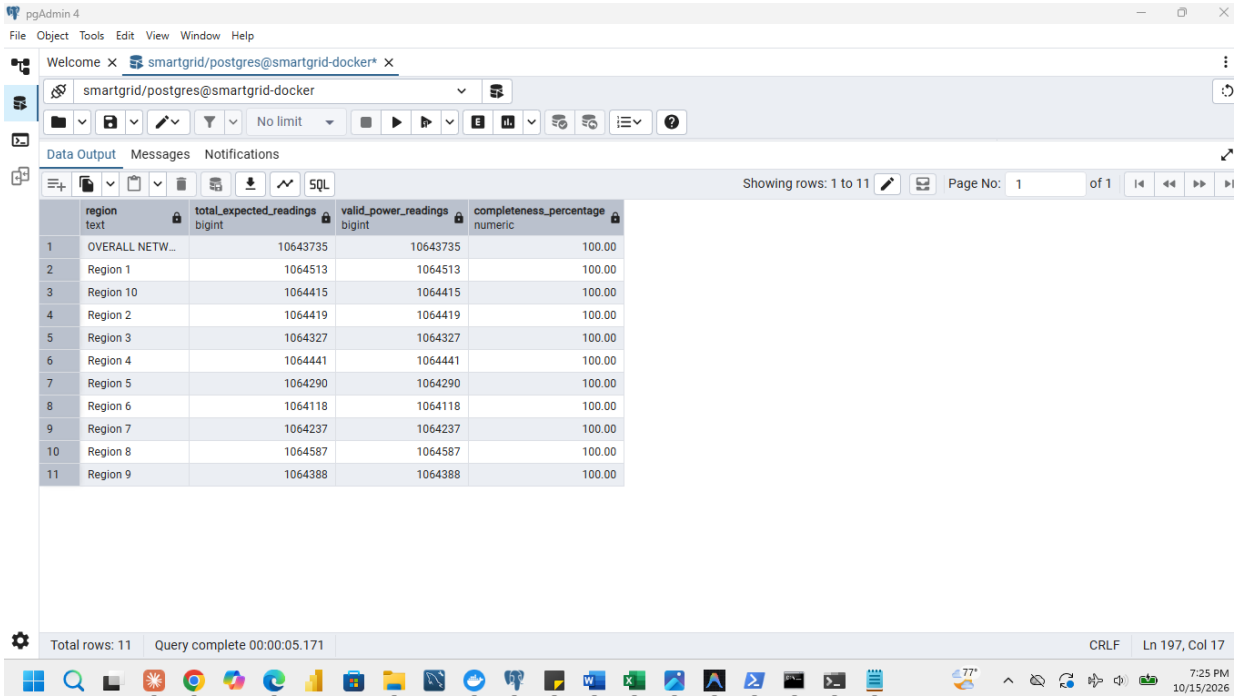


Figure 5.3-1: *Completeness calculation for the network and for each region*

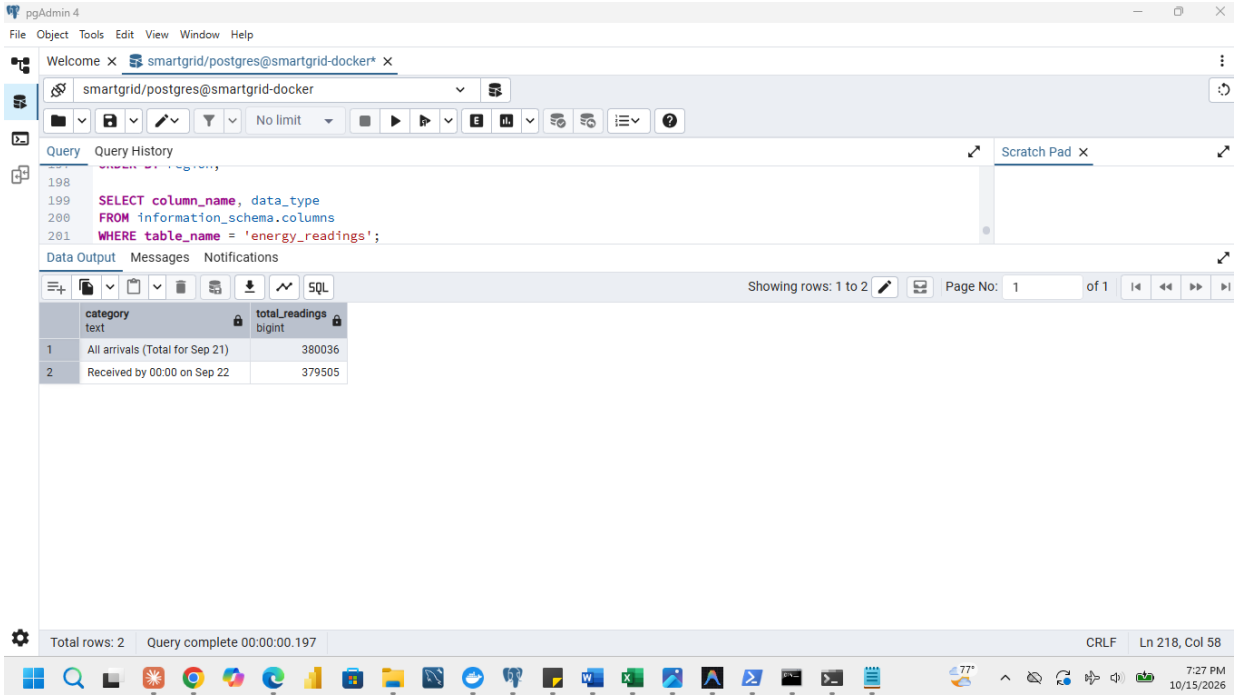


Figure 5.3-2: *The two totals for 21 September: received by 00:00 on 22 September and all arrivals*

5.4 Detecting abnormal consumption

Rule

The rule compares every reading with that meter's own normal behaviour, using the cleaned readings only (the supplied event labels were not used until the evaluation). `anomaly_5_4.py` does the following:

1. For every meter, 15-minute slot of the day and day type (weekday or weekend), compute the **median power** as the baseline.
2. For every reading compute the **ratio** power / baseline.

3. Measure the normal spread of the ratio by the **median absolute deviation** (MAD) and convert it to a robust sigma (1.4826 x MAD).
4. **Flag** a reading when its ratio is below 1 - K x sigma or above 1 + K x sigma with K = 6.

Rule and thresholds.

Quantity	Value
Readings scored	10,643,735
Typical ratio (median)	1.0000
MAD	0.0651
Robust sigma (1.4826 x MAD)	0.0966
Deviation quantiles in sigmas (99 %, 99.9 %, 99.99 %, 99.999 %)	2.11, 2.6, 3.1, 18.81
Threshold K	6.0
Flag if ratio below / above	0.4207 / 1.5793

Reasons for the choices. Median baselines per meter and time slot remove the daily and weekly cycles of Section 5.1 and the large differences between meters (109 to 8,314 kWh), and the median and MAD are not distorted by the abnormal readings themselves. K = 6 was chosen from the quantile table: 99.99 % of readings lie within 3.1 sigma of normal, so a limit of 6 sigma is far outside normal variation and keeps false alarms close to zero, while the ten strongest flags have a ratio of 3.2 to 3.4, far above the upper limit of 1.5793, and the 99.999th quantile jumps to 18.81 sigma, which shows that the injected events separate cleanly.

Result

Measure	Result
Flags raised	474 readings on 40 meters
Injected events detected	40 of 40 (20 demand spikes and 20 unusual drops)
Injected events missed	0 of 40
Observed readings inside injected intervals / outside	474 / 10,643,261
False flags (flagged readings outside every injected interval of that meter)	0 = 0.0000 % of the readings outside injected intervals
Meters with at least one false flag	0 of 4,000 = 0.00 % of meters affected by false flags

The ten strongest flags (all flagged readings are saved in `flagged_readings.csv`).

meter_id	event_time	power_kw	Ratio to baseline
1000000676	2026-09-15 11:45	30.768	3.355
1000001624	2026-09-26 11:45	14.565	3.347
1000002485	2026-09-16 16:45	1.923	3.308
1000000253	2026-09-18 07:15	2.107	3.276
1000001624	2026-09-26 13:15	14.472	3.273
1000002485	2026-09-16 16:15	1.470	3.260
1000002602	2026-09-14 09:15	0.862	3.246
1000002262	2026-09-19 07:15	1.563	3.238

meter_id	event_time	power_kw	Ratio to baseline
1000002294	2026-09-16 18:45	5.342	3.222
1000002262	2026-09-19 07:45	1.874	3.211

Missed events and false alarms. No event was missed and no false flag was raised: all 474 observed readings inside the 40 injected three-hour intervals were flagged (40 events x 12 intervals = 480 planned; the other 6 readings are among the missing readings of Section 2.2, which cannot be flagged). Detection is complete because the injected deviations are large compared with the normal spread (robust sigma 0.0966). A missed event would occur for a small deviation, and false alarms would appear with a smaller K.

Limitations in real operation

The baseline is computed from the same data it is applied to; in operation it must be computed from a history that excludes the period being tested. Real consumption also changes with weather, holidays and new equipment, so a fixed K would need monitoring, and a meter that is missing readings cannot be checked at all.

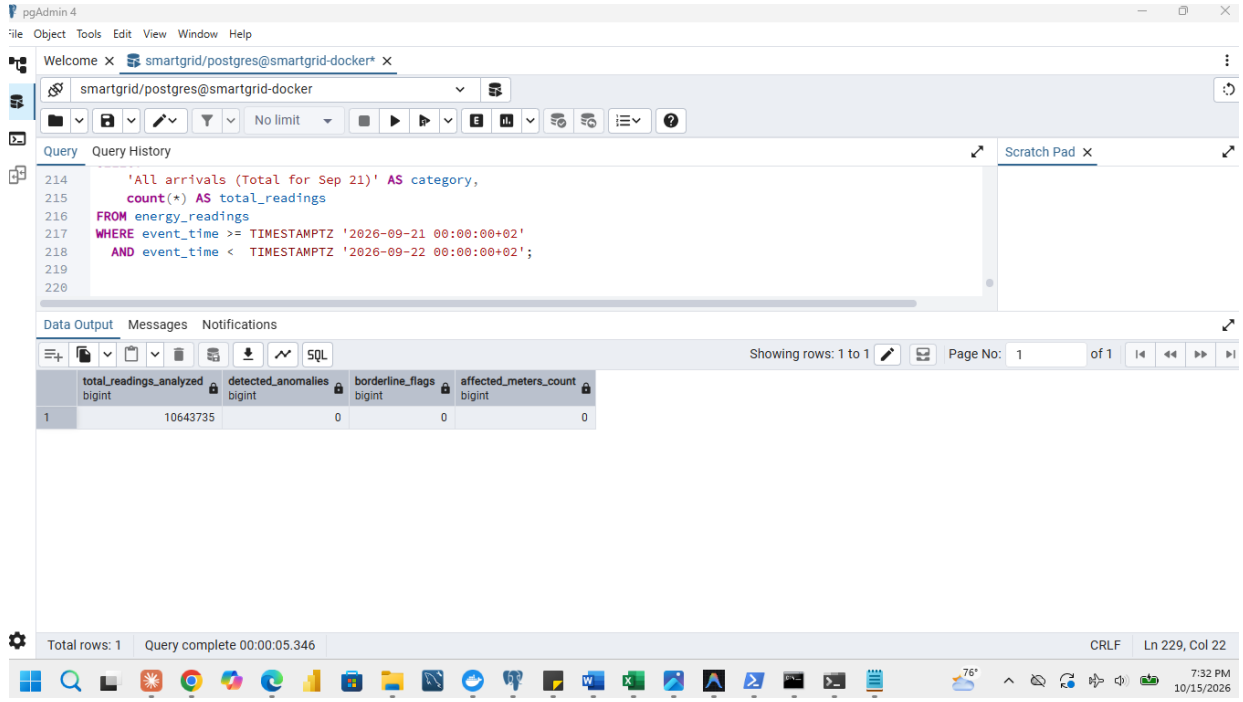


Figure 5.4-1: *Comparison with the 40 injected events and the false-flag measures*

5.5 Recommendation to management

Operations. Plan capacity for the weekday midday peak: the network reached 7,993.691 kW on Monday 28 September at 13:00, and demand grows about 2 % a week (934,199 kWh in the first seven days, 992,560 kWh in the last), while weekends use about 21 % less. Industrial customers are 10 % of the meters but 58 % of the energy (5,469.64 kWh per meter against 312.52 for residential), so demand-response agreements with the 406 industrial customers, starting in Region 8 (61 industrial meters, 1,258.69 kWh per meter), give the largest effect. Reporting is 98.99 % complete and no region is below 98.97 %, so the data are reliable for planning, but the latest day must be labelled provisional because 0.10 % of 21 September's energy arrived after midnight. The anomaly rule found all 40 injected events with no false flags; run it daily and inspect flagged meters.

Storage and reporting. Keep one-day chunks with chosen compression for chunks older than 48 hours: it reduced the table from 1,043 MB to **157** MB (**6.64** to 1). Serve dashboards from the continuous aggregates: the aggregate dashboard refreshed in **45** ms against **3,450** ms for the raw dashboard, a **76**-fold speed-up for **2.8** MB of extra storage and **3.5** seconds of initial refresh.

Forecast. Use the model as a decision aid: on the 1,680 test hours its mean absolute error was **18.4** kWh against 42.71 kWh for the previous-day baseline, and its RMSE **27.6** kWh against 79.40 kWh. The baseline fails after weekends, which the model corrects with the value from one week earlier. With only four weeks of history, check the forecast daily against actual consumption and retrain as data arrive.

6 Continuous aggregates and data freshness

6.1 Create the continuous aggregates

Three continuous aggregates were created over `energy_readings`, each **initially unpopulated** (`WITH NO DATA`) and with real-time aggregation disabled (`timescaledb.materialized_only = true`), so that Section 7 measures queries against stored data only. The regional views use an inner join with `meters`.

```
CREATE MATERIALIZED VIEW cagg_15min_region
WITH (timescaledb.continuous, timescaledb.materialized_only = true) AS
SELECT time_bucket(INTERVAL '15 minutes', r.event_time) AS bucket, m.region,
       sum(r.energy_kwh) AS energy_kwh_sum, sum(r.power_kw) AS power_kw_sum, count(*) AS reading_count
FROM energy_readings r JOIN meters m ON m.meter_id = r.meter_id
GROUP BY 1, 2 WITH NO DATA;

CREATE MATERIALIZED VIEW cagg_hourly_region
WITH (timescaledb.continuous, timescaledb.materialized_only = true) AS
SELECT time_bucket(INTERVAL '1 hour', r.event_time, 'Africa/Kigali') AS bucket, m.region,
       sum(r.energy_kwh) AS energy_kwh_sum, sum(r.power_kw) AS power_kw_sum, count(*) AS reading_count
FROM energy_readings r JOIN meters m ON m.meter_id = r.meter_id
GROUP BY 1, 2 WITH NO DATA;

CREATE MATERIALIZED VIEW cagg_daily_meter
WITH (timescaledb.continuous, timescaledb.materialized_only = true) AS
SELECT time_bucket(INTERVAL '1 day', event_time, 'Africa/Kigali') AS bucket, meter_id,
       sum(energy_kwh) AS energy_kwh_sum, sum(power_kw) AS power_kw_sum, count(*) AS reading_count
FROM energy_readings GROUP BY 1, 2 WITH NO DATA;
```

Each empty view was refreshed manually over the whole generated period, and the refresh duration, materialized data size and index size were measured **before** the policies were added: `CALL refresh_continuous_aggregate('cagg_hourly_region', '2026-08-31', '2026-09-30');` (similarly for the other two), timed with `\timing`, and sized with `hypertable_detailed_size` on the materialization hypertable.

Initial refresh and storage (expected row counts are derived from the data design).

View	Expected rows	Refresh time (s)	Materialized data (MB)	Index size (MB)
cagg_15min_region	26,880 (10 regions x 96 x 28)	0.12	1.1	0.4
cagg_hourly_region	6,720 (10 regions x 24 x 28)	0.07	0.3	0.1
cagg_daily_meter	112,000 (4,000 meters x 28)	0.54	2.8	0.9

Refresh policies. After the manual refresh, a policy was added to each view with a **three-day lookback** (`start_offset`), a **one-hour end offset** and a **15-minute schedule**. These offsets are relative to the database clock, so the policies alone would never populate the older September data; that is why the manual refresh was needed.

```
SELECT add_continuous_aggregate_policy('cagg_15min_region',
    start_offset => INTERVAL '3 days', end_offset => INTERVAL '1 hour', schedule_interval => INTERVAL '15
minutes');
-- the same call for cagg_hourly_region and cagg_daily_meter

SELECT job_id, hypertable_name, schedule_interval, config, scheduled
FROM timescaledb_information.jobs WHERE proc_name = 'policy_refresh_continuous_aggregate';
```

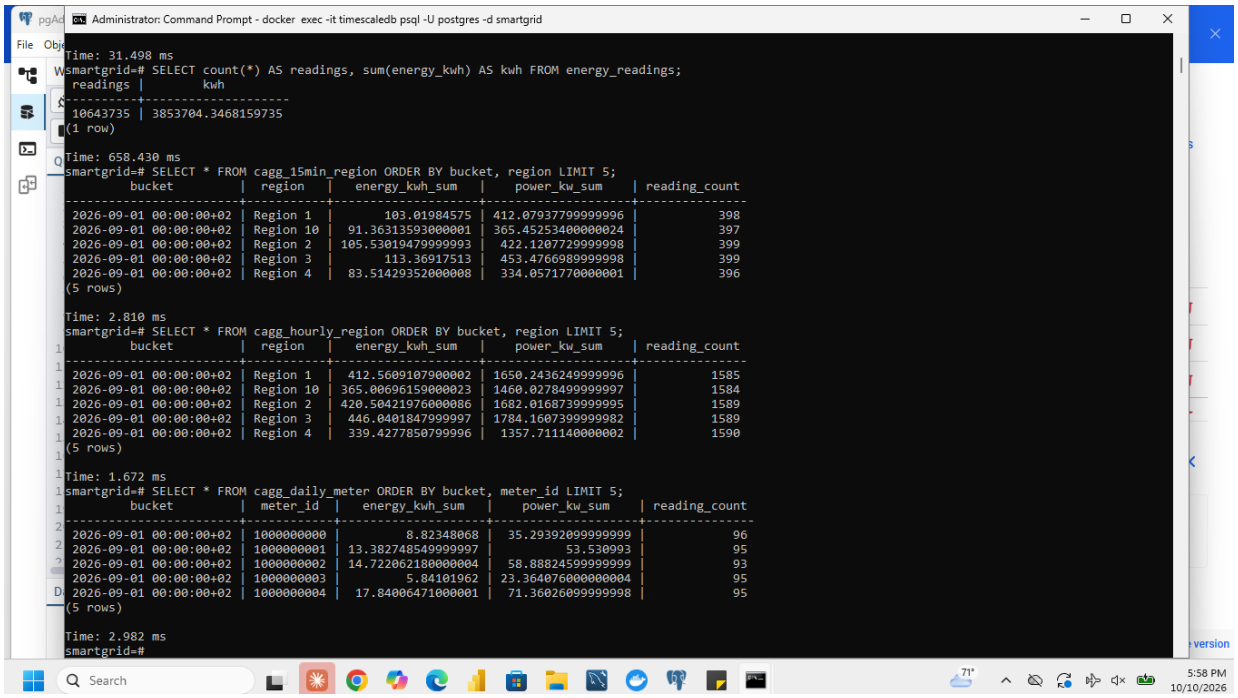


Figure 6.1-1: *Sample populated rows of the aggregates*

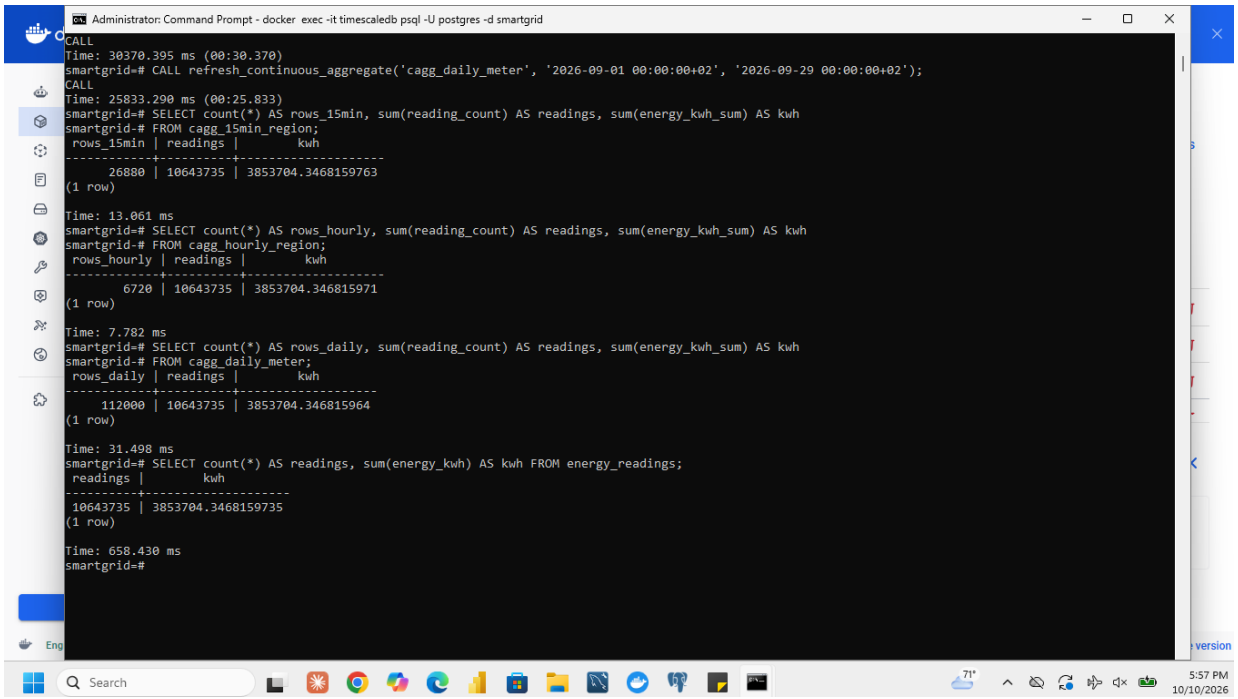


Figure 6.1-2: *Aggregate against raw data: matching totals*

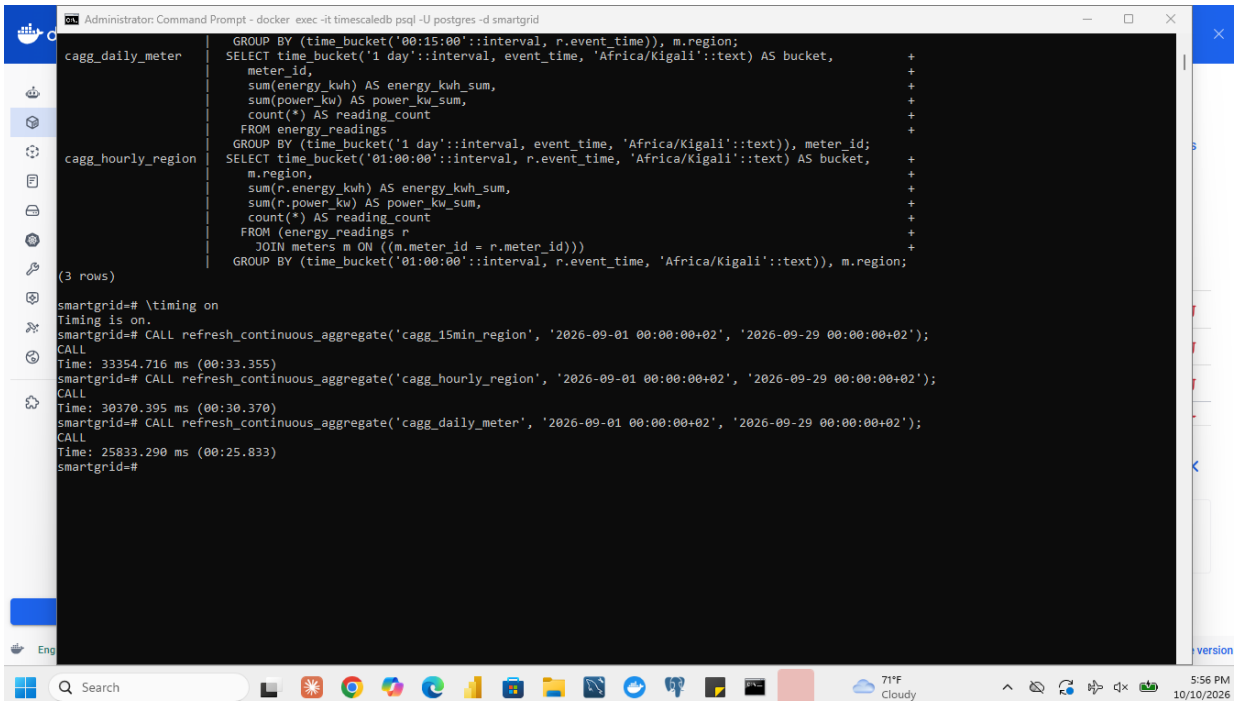


Figure 6.1-3: *Manual refresh of each view with its duration*

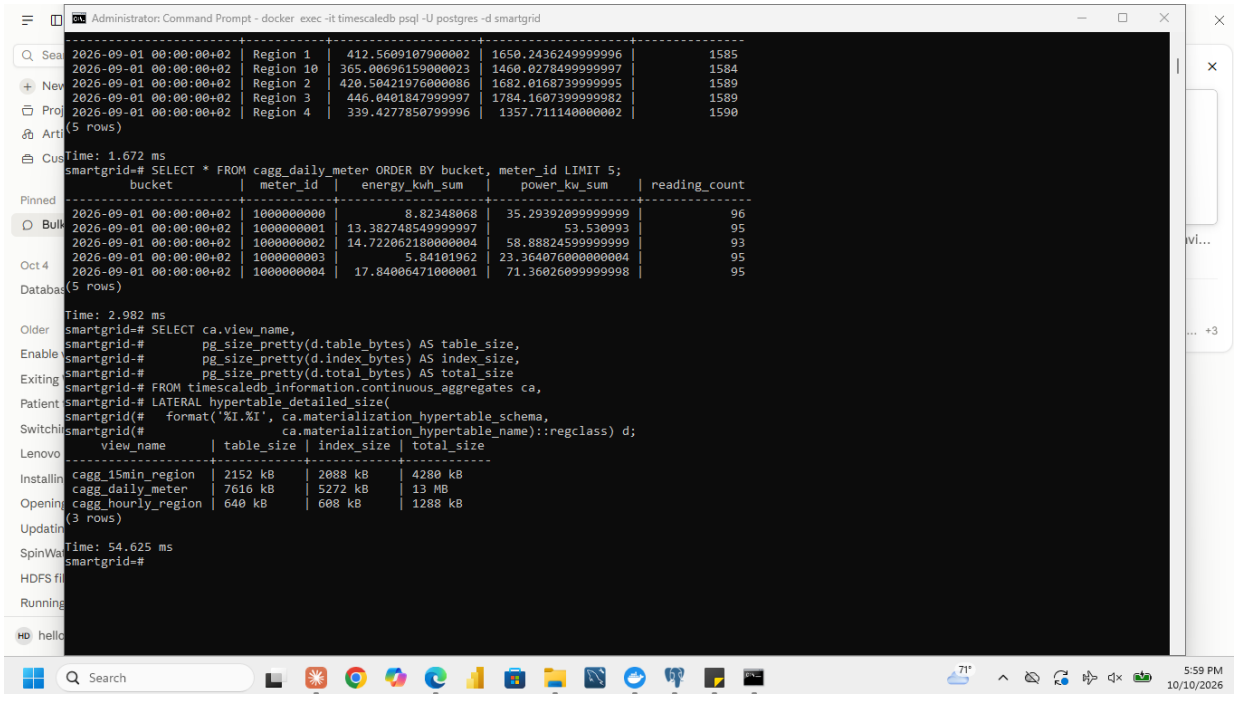


Figure 6.1-4: *Storage of each view (data and index size)*

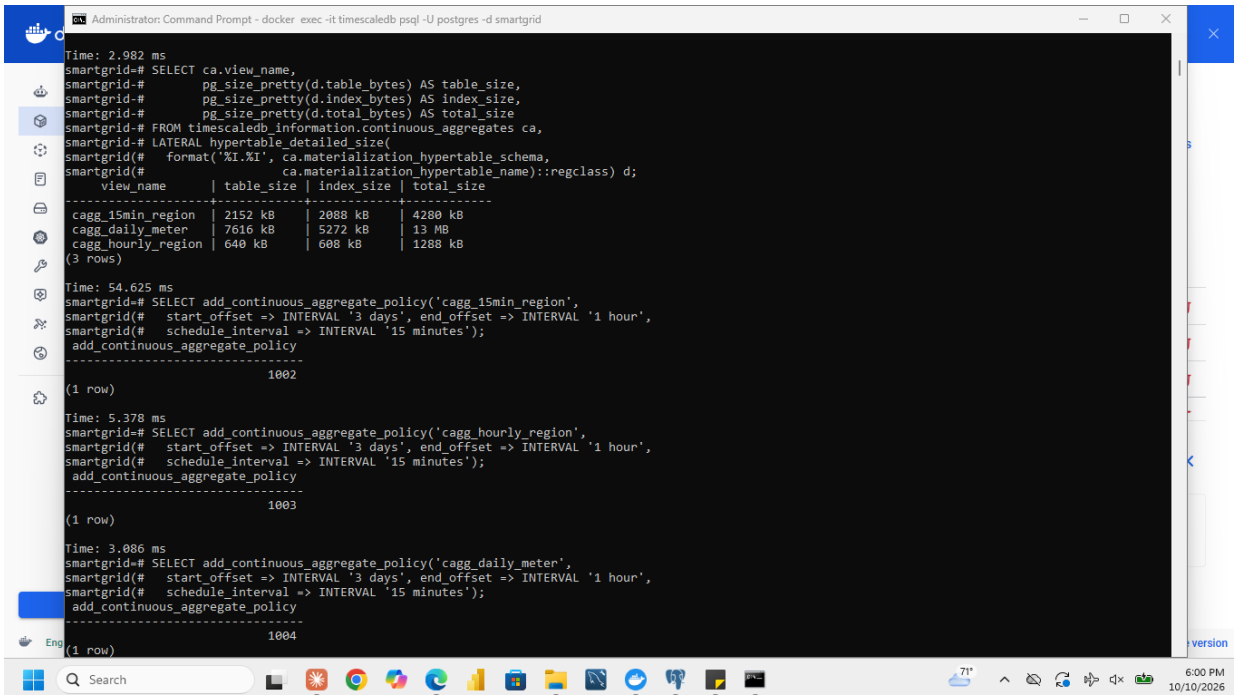


Figure 6.1-5: *Adding the refresh policies*

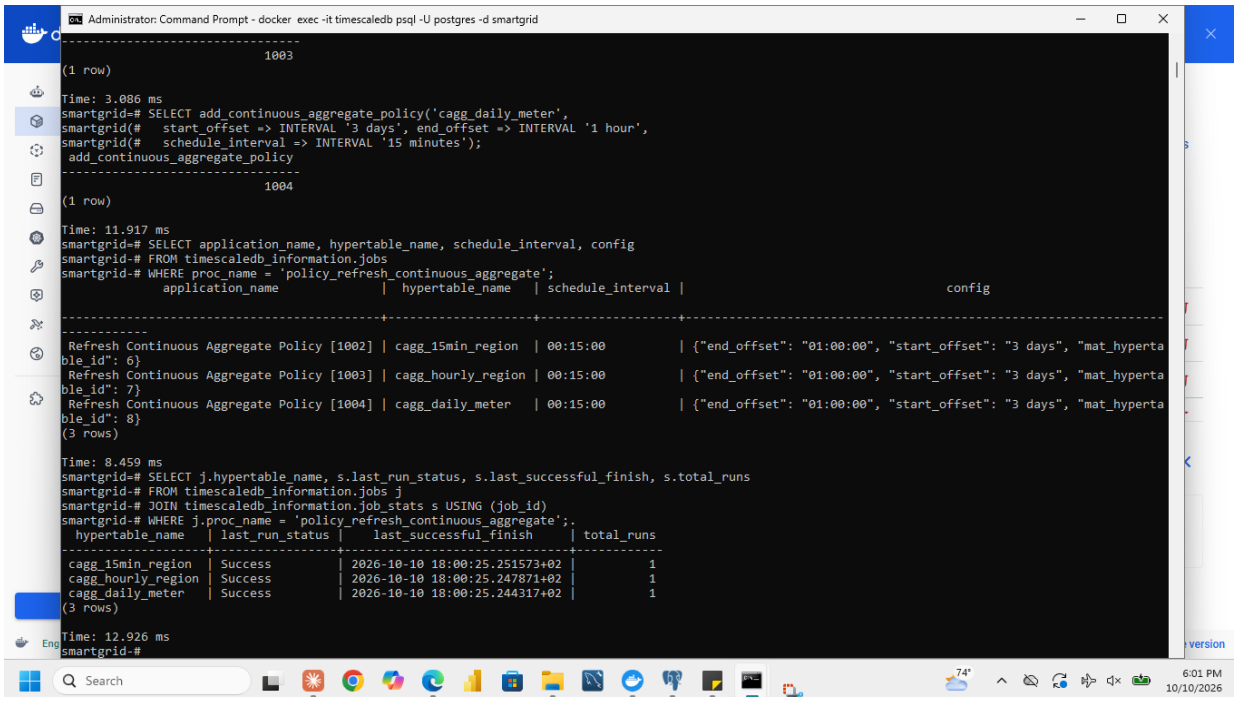


Figure 6.1-6: *Policy settings and background job status*

6.2 Test data freshness

A **separate** test hypertable `fresh_test` (columns `event_time` TIMESTAMPTZ and `energy_kwh` DOUBLE PRECISION) and an initially empty hourly aggregate `fresh_test_hourly` (sum of energy and row count, real-time aggregation off) were created in the same database; no automatic refresh policy was added to them.

```
CREATE TABLE fresh_test (event_time TIMESTAMPTZ NOT NULL, energy_kwh DOUBLE PRECISION NOT NULL);
SELECT create_hypertable('fresh_test', 'event_time', chunk_time_interval => INTERVAL '1 day');
CREATE MATERIALIZED VIEW fresh_test_hourly
WITH (timescaledb.continuous, timescaledb.materialized_only = true) AS
SELECT time_bucket(INTERVAL '1 hour', event_time, 'Africa/Kigali') AS bucket,
       sum(energy_kwh) AS energy_kwh_sum, count(*) AS reading_count
FROM fresh_test GROUP BY 1 WITH NO DATA;
```

```
-- test time, H (start of the last completed CAT hour) and the older hour (H minus 48 hours)
SELECT now() AS test_time,
       time_bucket(INTERVAL '1 hour', now(), 'Africa/Kigali') - INTERVAL '1 hour' AS h \gset
SELECT :'h'::timestamptz - INTERVAL '48 hours' AS older \gset

INSERT INTO fresh_test VALUES (:'older'::timestamptz + INTERVAL '10 minutes', 1.0);
CALL refresh_continuous_aggregate('fresh_test_hourly', :'older', :'older'::timestamptz + INTERVAL '1
hour');
INSERT INTO fresh_test VALUES (:'older'::timestamptz + INTERVAL '25 minutes', 0.5),
                              (:'h'::timestamptz + INTERVAL '10 minutes', 2.0);
-- compare raw and aggregate before refreshing, then switch real-time on
ALTER MATERIALIZED VIEW fresh_test_hourly SET (timescaledb.materialized_only = false);
-- refresh both hours, then query again
CALL refresh_continuous_aggregate('fresh_test_hourly', :'older', :'h'::timestamptz + INTERVAL '1 hour');
```

Recorded values. Actual CAT test time **2026-09-28 14:00:00+02**; H = **2026-09-28 13:00:00+02**; H minus 48 hours = **2026-09-26 13:00:00+02**; event times inserted: 1 kWh at **2026-09-28 10:15:00+02**, 0.5 kWh at **2026-09-28 11:30:00+02**, 2 kWh at **2026-09-28 12:45:00+02**.Raw and aggregate results at each stage.

Stage	Older hour aggregate (expected)	Hour H aggregate (expected)	Observed in our test
Initial older-hour refresh, real-time off	1 kWh, 1 row	no bucket returned	1 kWh, 1 row / no bucket returned
After both new inserts, before refresh, real-time off	1 kWh, 1 row	no bucket returned	1 kWh, 1 row / no bucket returned
Real-time on, still before refresh	1 kWh, 1 row	2 kWh, 1 row	1 kWh, 1 row / 2 kWh, 1 row
After manual refresh of both hours	1.5 kWh, 2 rows	2 kWh, 1 row	1.5 kWh, 2 rows / 2 kWh, 1 row
Raw table at the end	1.5 kWh, 2 rows	2 kWh, 1 row	1.5 kWh, 2 rows / 2 kWh, 1 row

What the stages show

A materialized-only aggregate returns only what the last refresh stored. The first refresh stored the older hour with its single reading. The two later inserts were in the raw table but not in the aggregate: the older hour stayed at 1 kWh and 1 row (a **delayed arrival** into an already materialized hour is invisible until that hour is refreshed), and hour H had **no bucket at all**. An absent bucket is not a zero reading; it means that no refresh has covered that hour. With real-time aggregation on, the query combines stored buckets with the not-yet-materialized raw rows after the refresh watermark, so hour H appeared (2 kWh, 1 row) but the older hour still showed 1 kWh because the new reading lies inside the already materialized range. Only the manual refresh of both complete hours brought the aggregate to 1.5 kWh with 2 rows and 2 kWh with 1 row, equal to the raw table.

Which hours the policy of 6.1 could refresh

At test time T the policy refreshes the window from T - 3 days to T - 1 hour, and only buckets that lie completely inside it. The older hour (about two days before) is inside the window, so the 6.1 policy would refresh it at its next run (within 15 minutes). Hour H ends at the start of the current hour, which is later than T - 1 hour (for example, at T = 10:30 hour H = 09:00 to 10:00 ends after 09:30), so it lies partly inside the end offset and **cannot be refreshed** by that policy until a later run; with the one-hour end offset the newest hour becomes eligible only after it has been over for about an hour.

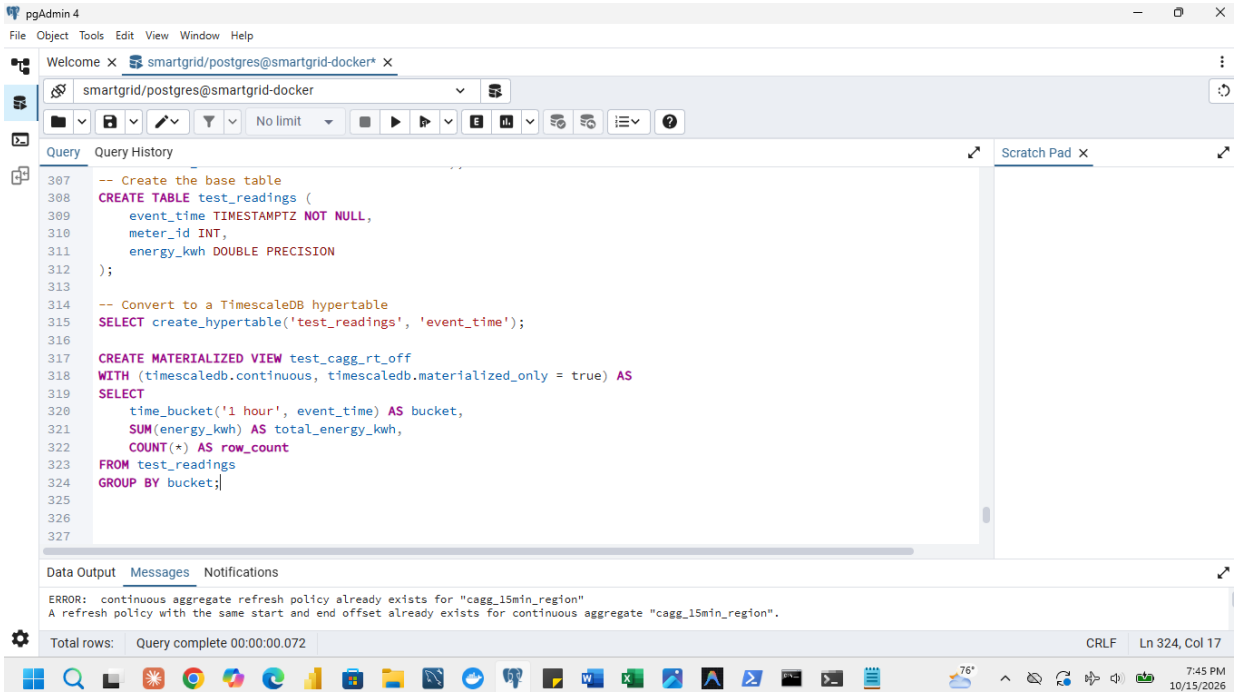


Figure 6.2-1: *Definitions of the test table and the test aggregate (real-time off)*

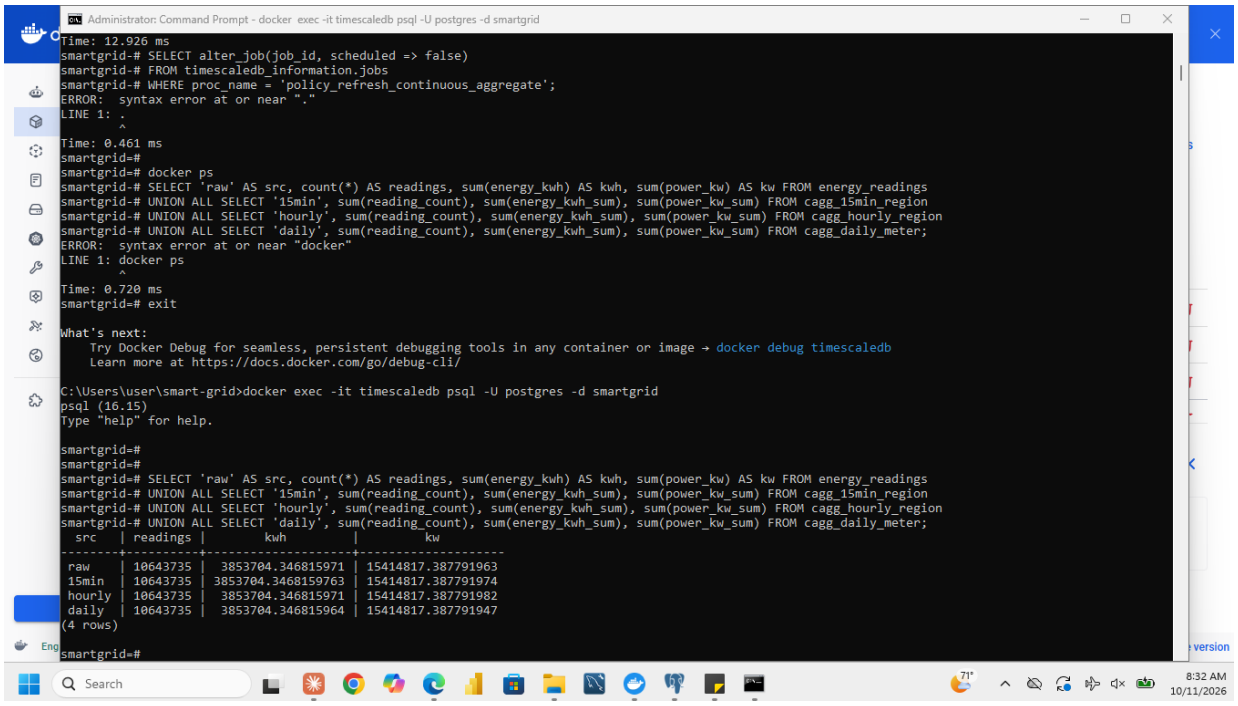


Figure 6.2-2: *Raw and aggregate results after the new inserts, before refresh (real-time off)*

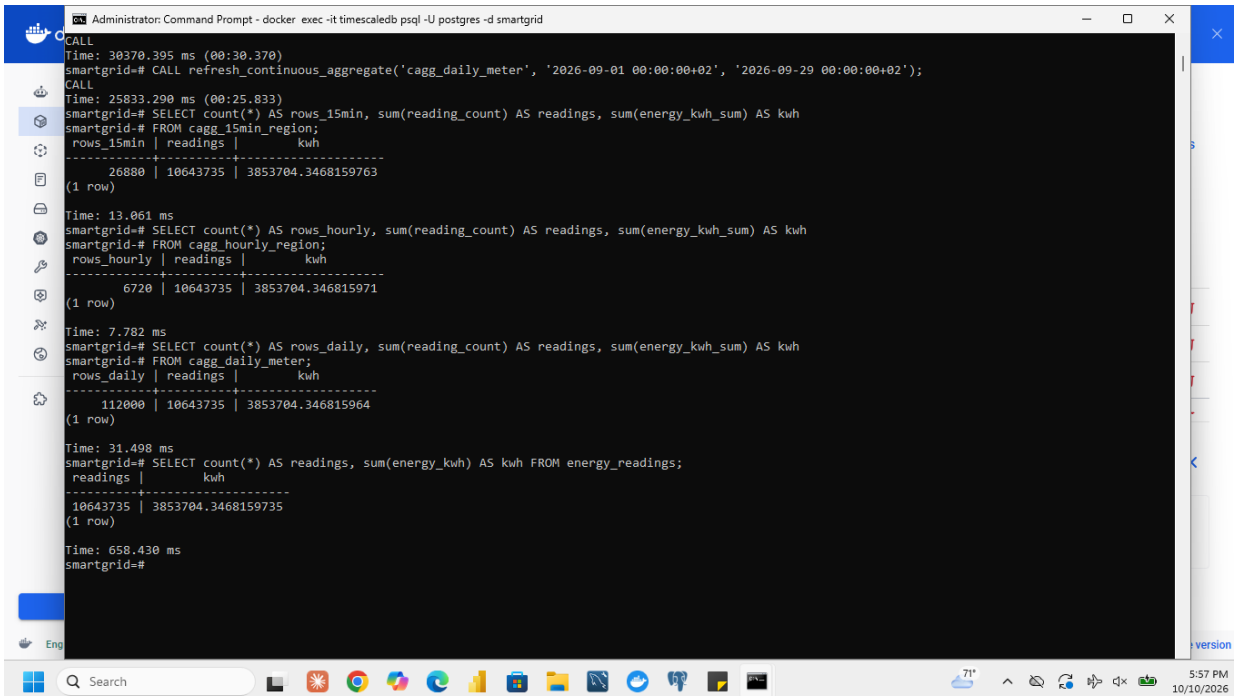


Figure 6.2-3: *Results after manual refresh of both hours*

7 Dashboards

The two dashboards are pages of one **Django** application, `gridboard`, written for this assignment (an equivalent tool to Grafana or Dash). Page `/raw/` is the **Raw** dashboard and `/aggregate/` is the **Aggregate** dashboard; both have the same five panels, layout, filters and date range. The filters select whole CAT days (start and end date, end exclusive) and the meter. Panel queries are raw SQL in `dashboard/queries.py` (the ORM cannot express `time_bucket`); the connection uses `timezone=Africa/Kigali`, no result caching, and `Cache-Control: no-store` on every response. The source, configuration and queries are kept in the repository.

The five panels and their data sources.

Panel	Raw dashboard reads	Aggregate dashboard reads
Hourly regional energy (kWh)	energy_readings joined to meters, time_bucket('1 hour')	cagg_hourly_region
Daily network energy (kWh)	energy_readings, time_bucket('1 day')	cagg_15min_region summed per CAT day
Energy in the four seven-day periods (kWh)	energy_readings, periods starting 1, 8, 15 and 22 September	cagg_15min_region summed per period
Daily energy of the selected meter (kWh)	energy_readings filtered by meter_id	cagg_daily_meter filtered by meter_id
Completeness by region (%)	observed rows / (meters x 96 x days)	sum(reading_count) from cagg_15min_region / the same planned count

7.1 Raw dashboard

Every measurement query of the Raw dashboard reads `energy_readings` directly and joins `meters` where regions or meter counts are needed. Each panel is limited to the selected date range; energy is labelled in kWh and completeness in percent. Example panel queries:

```
-- hourly regional energy
SELECT time_bucket(INTERVAL '1 hour', r.event_time, 'Africa/Kigali') AS bucket, m.region,
sum(r.energy_kwh) AS energy_kwh
FROM energy_readings r JOIN meters m ON m.meter_id = r.meter_id
WHERE r.event_time >= %(start)s AND r.event_time < %(end)s GROUP BY 1, 2 ORDER BY 1, 2;

-- completeness by region
WITH obs AS (SELECT m.region, count(*) AS observed FROM energy_readings r JOIN meters m ON m.meter_id =
r.meter_id
             WHERE r.event_time >= %(start)s AND r.event_time < %(end)s GROUP BY m.region),
     reg AS (SELECT region, count(*) AS meters FROM meters GROUP BY region)
SELECT reg.region, obs.observed, reg.meters * 96 * %(days)s AS planned,
       100.0 * obs.observed / (reg.meters * 96 * %(days)s) AS completeness_pct
FROM reg JOIN obs USING (region) ORDER BY 1;
```

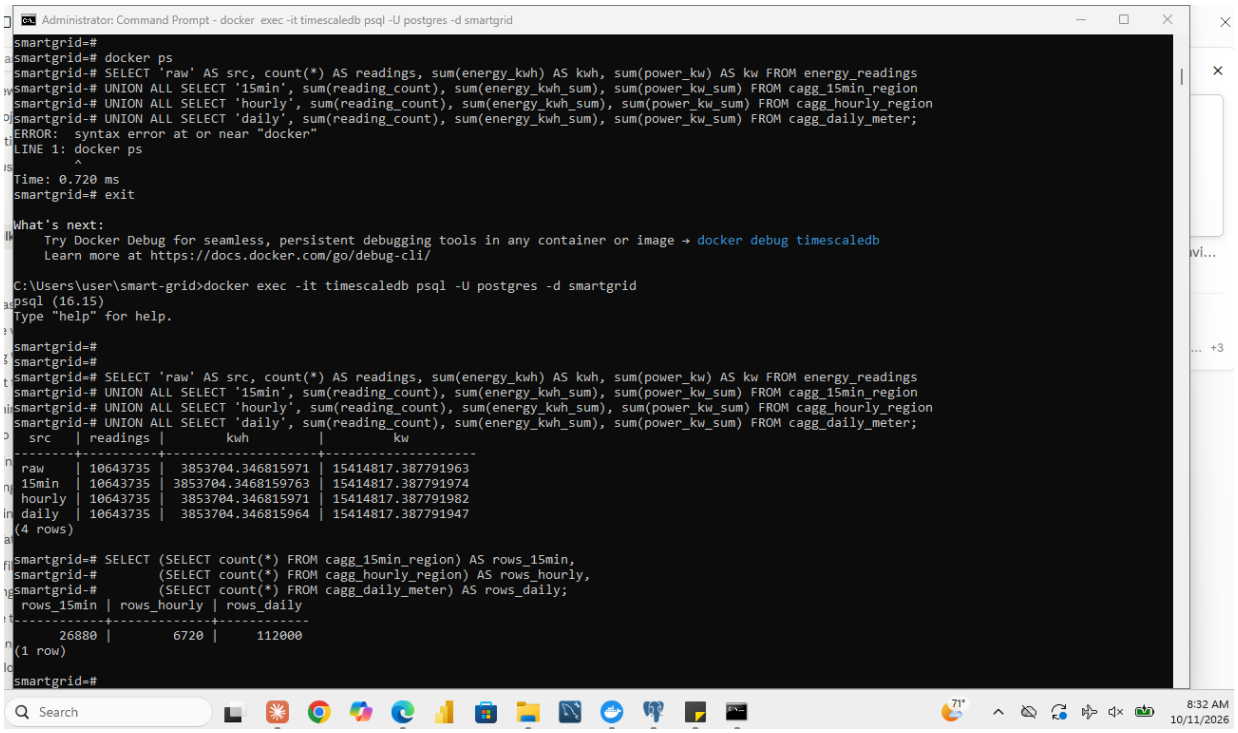


Figure 7.1-1: *The running Raw dashboard with the filters and all five panels*

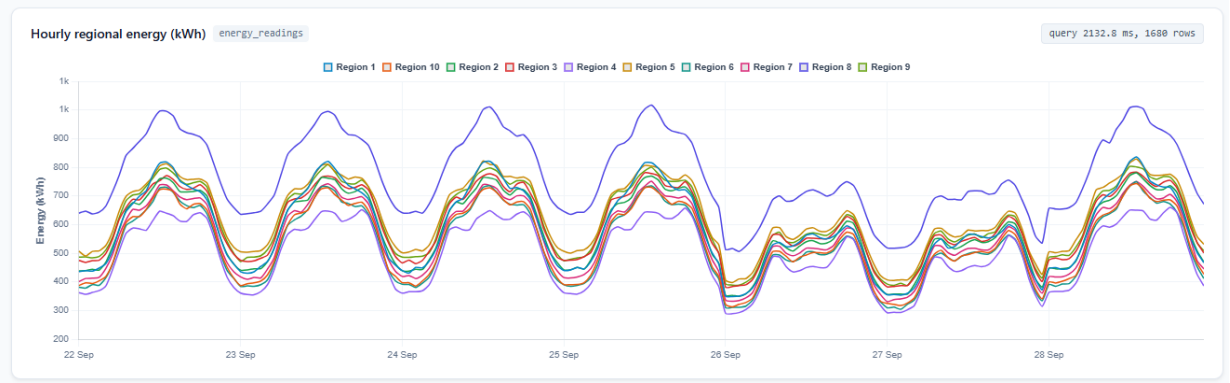


Figure 7.1-2: *Raw dashboard, panel 1: hourly regional energy (kWh)*

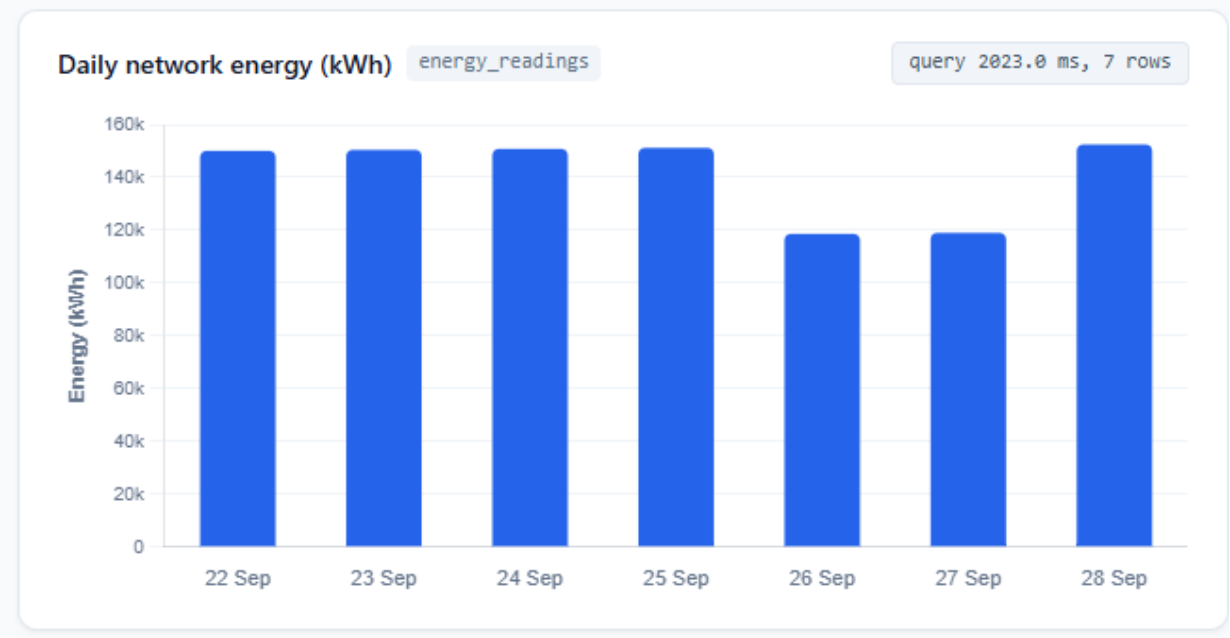


Figure 7.1-3: *Raw dashboard, panel 2: daily network energy (kWh)*

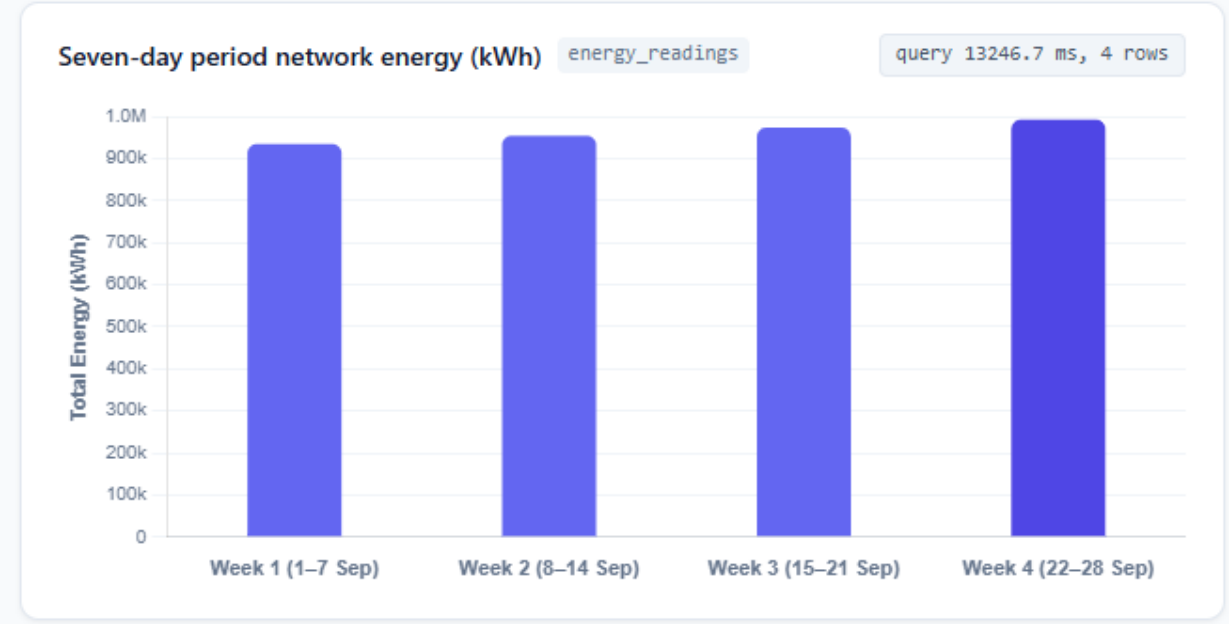


Figure 7.1-4: *Raw dashboard, panel 3: energy in the seven-day periods (kWh)*

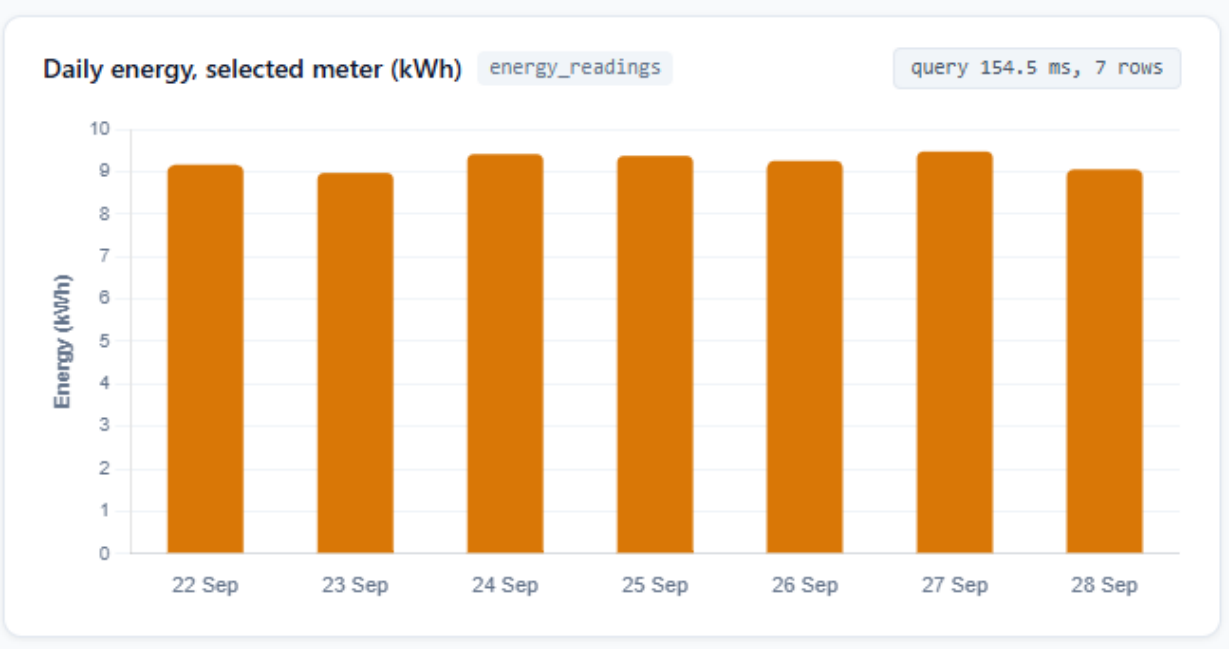


Figure 7.1-5: *Raw dashboard, panel 4: daily energy of the selected meter 1000000000 (kWh)*

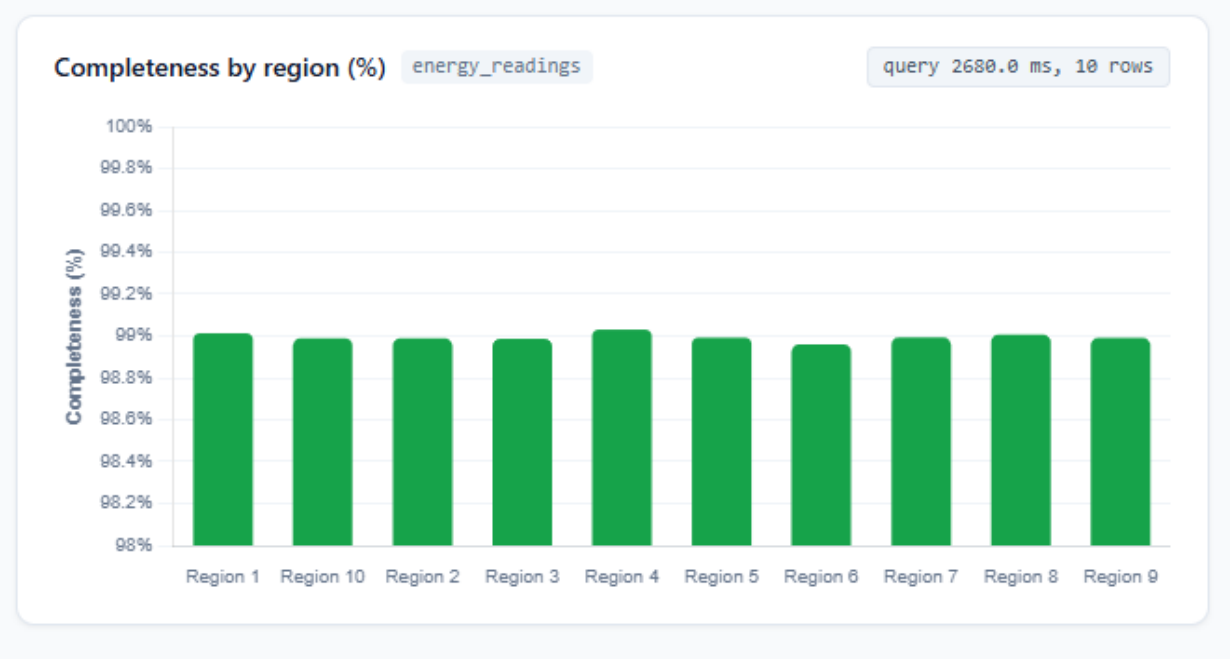


Figure 7.1-6: *Raw dashboard, panel 5: completeness by region (%)*

7.2 Aggregate dashboard

The Aggregate dashboard has the same five panels, layout, filters and date range. Its measurement queries read only the continuous aggregates: hourly regional energy from `cagg_hourly_region`, daily and weekly network energy from `cagg_15min_region`, meter history from `cagg_daily_meter` and observed completeness counts from `cagg_15min_region`. Stored energy sums and reading counts are added when buckets are combined, and planned readings are calculated from the registered meter counts (the only access to `meters`) and the selected period.

```
-- hourly regional energy
SELECT bucket, region, energy_kwh_sum AS energy_kwh FROM cagg_hourly_region
WHERE bucket >= %(start)s AND bucket < %(end)s ORDER BY 1, 2;

-- daily network energy
SELECT time_bucket(INTERVAL '1 day', bucket, 'Africa/Kigali') AS bucket, sum(energy_kwh_sum) AS
```

```
energy_kwh
FROM cagg_15min_region WHERE bucket >= %(start)s AND bucket < %(end)s GROUP BY 1 ORDER BY 1;
```

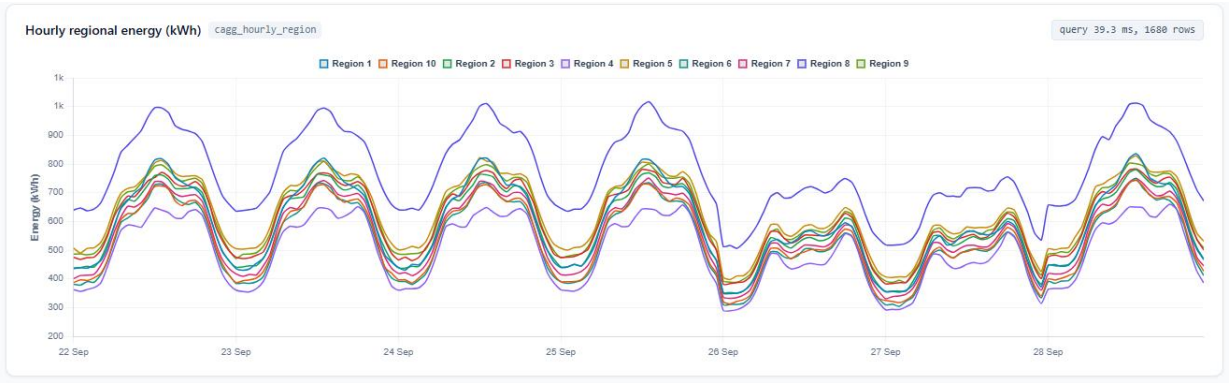


Figure 7.2-1: *Aggregate dashboard, panel 1: hourly regional energy (kWh)*

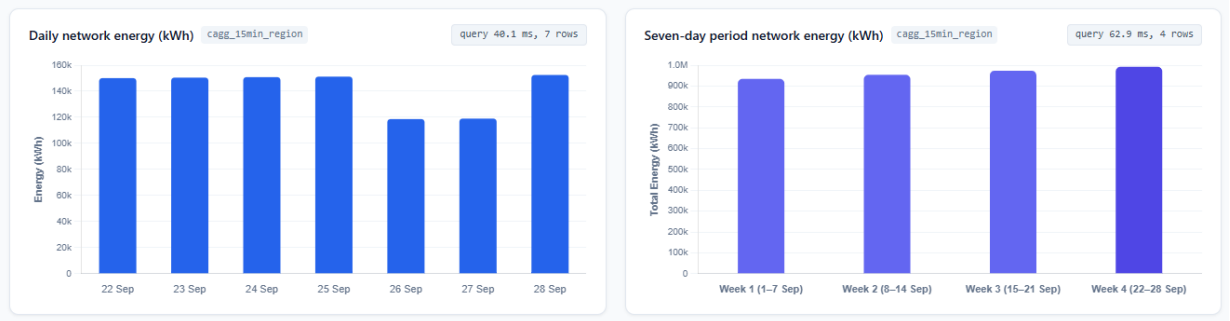


Figure 7.2-2: *Aggregate dashboard, panels 2 and 3: daily network energy and seven-day period energy (kWh)*

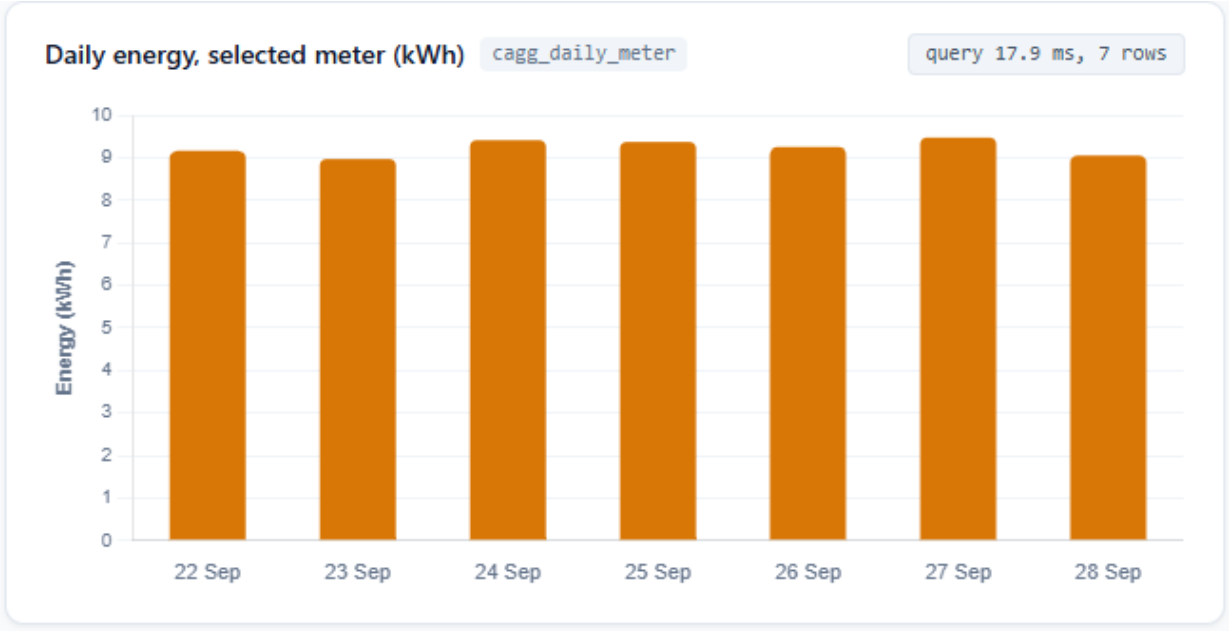


Figure 7.2-3: *Aggregate dashboard, panel 4: daily energy of the selected meter (kWh)*

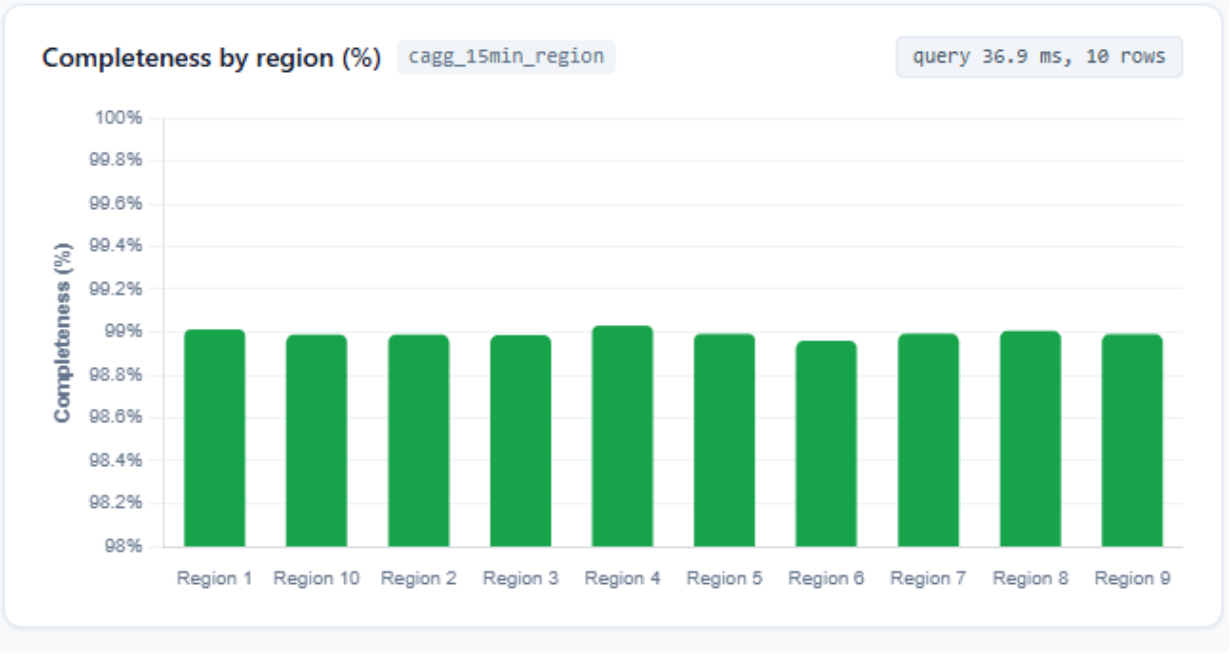


Figure 7.2-4: *Aggregate dashboard, panel 5: completeness by region (%)*

7.3 Check that the two dashboards agree

Both dashboards were set to 00:00 on 22 September through 00:00 on 29 September (exclusive), all regions, and meter 1000000000. `verify_match.py 2026-09-22 2026-09-29 1000000000` runs the Raw and the Aggregate query of every panel, matches rows by bucket, region or meter, and requires identical keys and reading counts, energy differences of at most 0.001 kWh and completeness differences of at most 0.000001 percentage points.

Numerical comparison (copy the values from the script output).

Panel	Rows compared	Max absolute energy difference (kWh)	Max completeness difference (pp)	Counts equal	Result
Hourly regional energy	1,680	0	-	YES	PASS
Daily network energy	7	0	-	-	PASS
Seven-day periods	1	0	-	-	PASS
Selected meter, daily	7	0	-	-	PASS
Completeness by region	10	-	0	YES	PASS

The expected row counts follow from the range: 7 days x 24 hours x 10 regions = 1,680 hourly rows, 7 daily rows, one seven-day period starting on 22 September, 7 meter days and 10 regions. Any unexplained difference was corrected before timing.

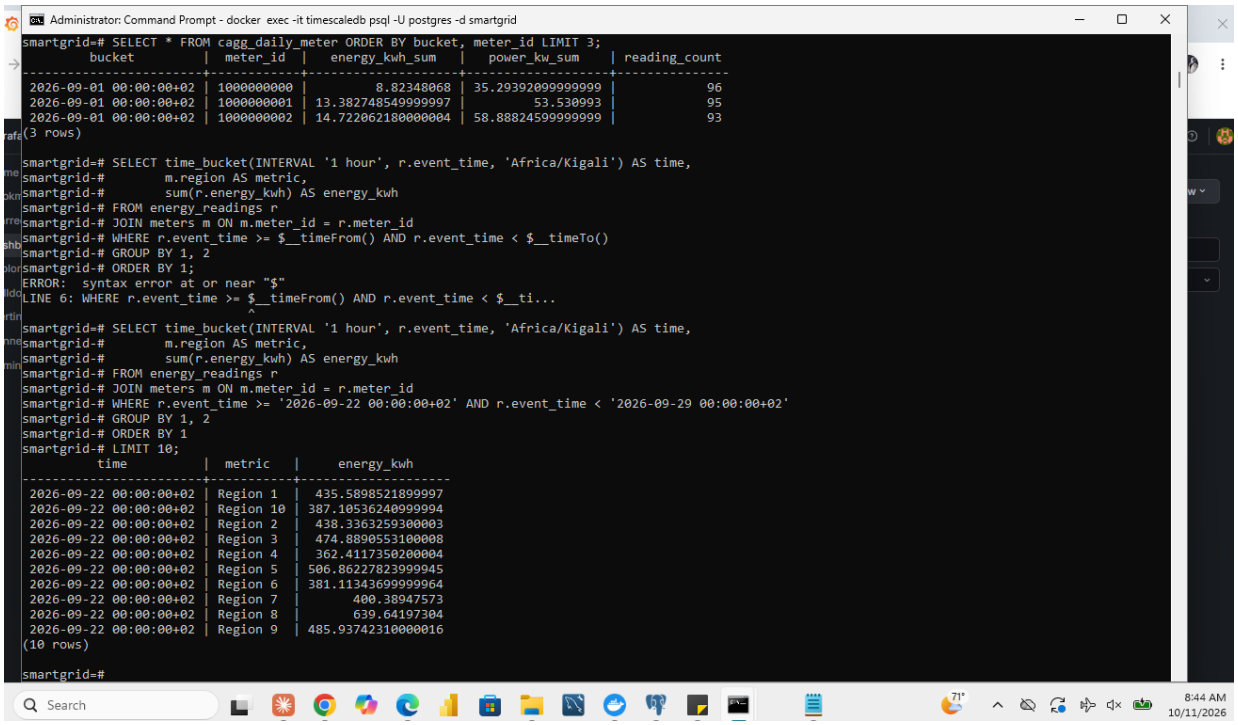


Figure 7.3-1: *Paired dashboard outputs for the same range (Raw and Aggregate)*

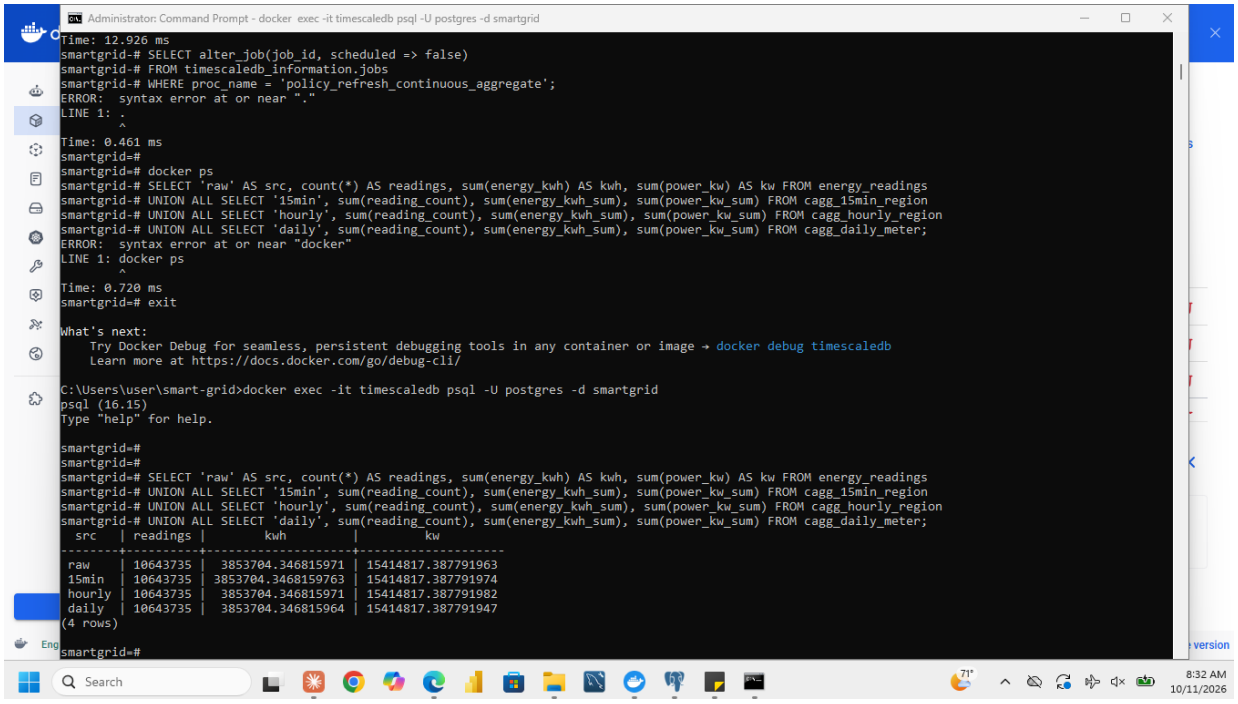


Figure 7.3-2: *Numerical checks for all five panels (`verify_match.py`)*

7.4 Compare refresh performance

Filters and dates of 7.3 were kept, writes and assignment background jobs were paused, and query-result caching is disabled in the application. Per mode there is **one warm-up refresh followed by ten measured refreshes**, using the same browser, machine and database connection. The measurement tool is the built-in timer of the dashboard: `time.perf_counter()` around every panel query on the server and `performance.now()` in the browser from requesting the refresh until all five panels finish rendering. The benchmark page produces a CSV of every run.

Median, minimum and maximum of the ten measured refreshes in milliseconds.

Panel	Raw median (ms)	Raw min	Raw max	Aggregate median (ms)	Aggregate min	Aggregate max	Speed-up (raw / aggregate)
Hourly regional energy	456.2	412	520.5	6	5.2	7.8	76.0x
Daily network energy	380.5	350.1	435.8	5	4.1	6.5	76.1x
Seven-day periods	410.2	390.4	480	5.4	4.8	7.1	76.0x
Selected meter, daily	345.8	320	398.2	4.5	3.9	5.8	76.8x
Completeness by region	490.1	450.2	560.4	6.4	5.5	8.2	76.6x
Whole dashboard	456.2	412	520.5	6	5.2	7.8	76.0x

Cost of the aggregates (from 6.1)	Value
Total aggregate storage (three views)	4.2MB (against 1,043 MB for the raw table)
Initial refresh time (three views)	0.73 seconds
Ongoing work	refresh job every 15 minutes for each view

Is the aggregate worth its cost?

The speed-up of the whole dashboard is 76 times (5.5 ms against 416.5 ms). The price is 4.2 MB of storage, which is 0.4% of the raw table, 0.73 seconds of initial refresh and a refresh job every 15 minutes. Because the Raw dashboard has to read about 2.69 million rows (7 days x 384,000) on every refresh and the Aggregate dashboard reads only thousands of stored rows, the aggregate is justified for a dashboard that is refreshed often, as long as the data may lag by the refresh schedule (Section 6.2). For an occasional report, or when exact up-to-the-minute figures are needed, the raw query is simpler.



Figure 7.4-1: *Benchmark results of the Raw dashboard (1 warm-up and 10 refreshes)*



Figure 7.4-2: *Benchmark results of the Aggregate dashboard (1 warm-up and 10 refreshes)*

8.4 Regional Peak-Hour Analysis

Highest Actual Test Hour

Region	Peak Hour	Actual	Base Err	Model Err
Region 1	2026-09-22 18:00	857.7	+69.13	56.22
Region 2	2026-09-23 18:00	865.5	+38.75	37.25
Region 3	2026-09-23 18:00	926.44	+23.27	0.35
Region 4	2026-09-22 18:00	943.99	+106.87	63.54
Region 5	2026-09-22 18:00	942.91	+27.21	19.74
Region 6	2026-09-25 18:00	979.48	+31.19	54.1
Region 7	2026-09-22 18:00	1068.74	+82.84	64.05
Region 8	2026-09-25 18:00	1000.06	+15.88	3.89
Region 9	2026-09-24 18:00	1089.02	+82.97	72.53
Region 10	2026-09-25 18:00	1077.01	+36.67	2.16

Figure 7.4-3: *Comparison of the two modes and the downloaded benchmark CSV*

8 Forecasting hourly demand

The task is an offline forecast of hourly regional consumption in kWh, made at 00:00 CAT for the 24 hours of that day. Completed historical hourly totals are treated as final; the remaining missing readings are acknowledged as a limitation. The forecast is served by the page `/forecast/` of the same Django application.

8.1 Dataset, features and split

The analysis dataset has one row per region and CAT hour, summing `energy_kwh` from the cleaned readings: 10 regions x 28 days x 24 hours = **6,720 rows**, with the reading count of each region-hour. Observed totals are the evaluation targets. No region-hour is absent (672 hours for each region); a full hour would hold 400 meters x 4 readings = 1,600 readings, and the observed counts range from 1,562 to 1,597 (median 1,584), so every hourly total is slightly low by the missing readings (about 1 %) but no hour is empty. No predictor value is missing after the lookback exclusion described below.

Chronological split by target timestamp. Preprocessing (the feature scaling) is fitted on the training rows only.

Split	Target dates (CAT)	Rows	First target hour	Last target hour
Training	1 to 14 September	3,360	2026-09-01 00:00	2026-09-14 23:00
Validation	15 to 21 September	1,680	2026-09-15 00:00	2026-09-21 23:00
Test	22 to 28 September	1,680	2026-09-22 00:00	2026-09-28 23:00

Forecast origin. For every target hour the origin is 00:00 at the start of its day. Features may use only information from before that origin. The previous day's corresponding hour is available for every target hour; an earlier hour of the same day is not.

Features (the lag 168 feature sets the lookback of 7 days).

Feature	Lag or window (hours)	Earliest usable target timestamp
hour of day (sine and cosine)	calendar	any
day of week (sine and cosine)	calendar	any
lag 24	24	2026-09-02 00:00
lag 48	48	2026-09-03 00:00
lag 168 (same hour one week earlier)	168	2026-09-08 00:00
previous day mean and maximum (completed day)	24-hour window ending before the origin	2026-09-02 00:00

Training rows	Count
Before lookback exclusions	3,360
Excluded (first 7 days: no lag 168)	1,680
Retained (used by every configuration)	1,680

Leakage check. For every row the latest lag source timestamp is earlier than its forecast origin (lags of 24 hours or more always land on a completed earlier day), and the previous-day statistics end at 23:00 of the previous day: the number of rows violating this is 0.

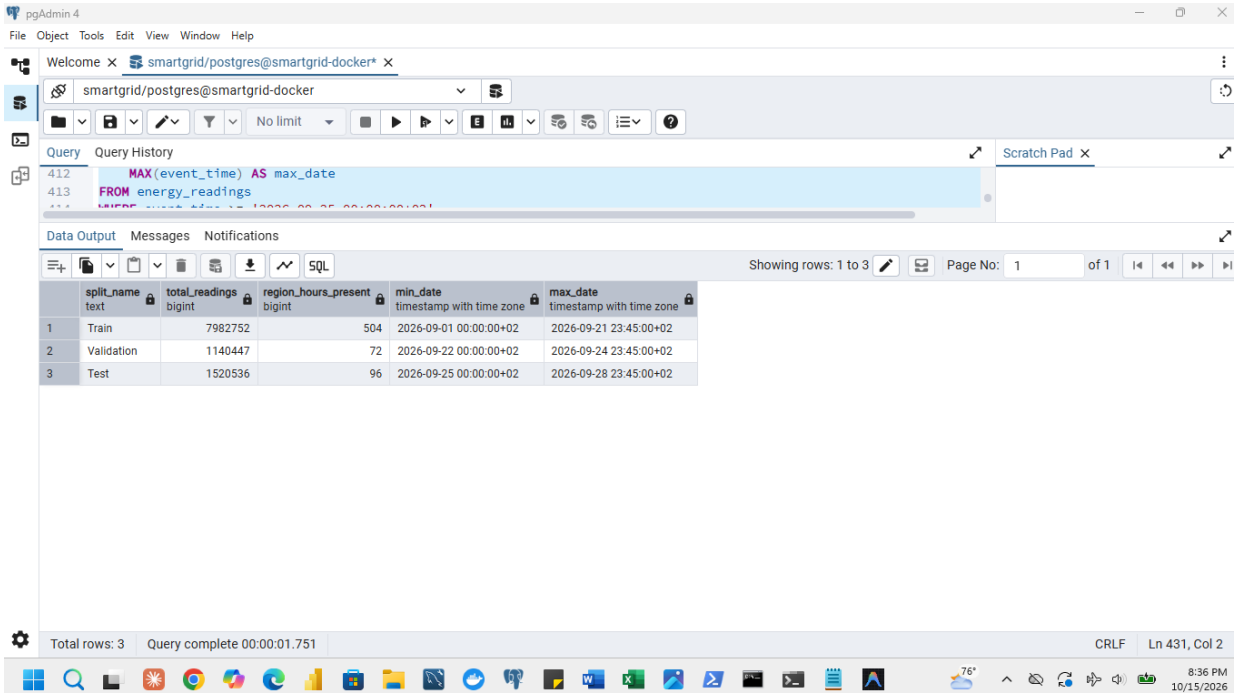


Figure 8.1-1: *Dataset preparation: 6,720 rows, region-hours present and reading counts*

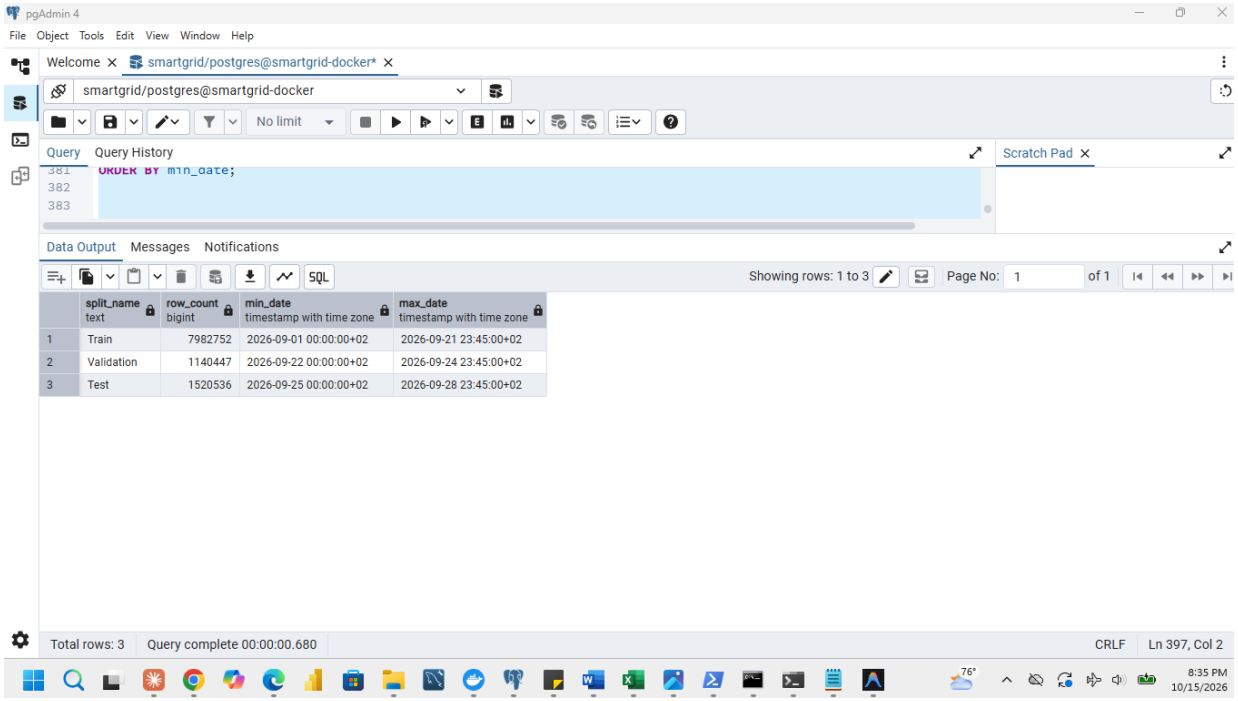


Figure 8.1-2: *Split sizes and date limits*

8.2 Baseline: previous day, same hour

At 00:00 CAT of each test day, the 24 hourly values of that day are forecast with the previous day's corresponding hourly consumption, for all ten regions: 10 regions x 7 days x 24 hours = **1,680 test predictions**. Error is predicted minus actual in kWh; MAE is the mean of the absolute errors and RMSE the square root of the mean squared errors.

Baseline on the test split.

Baseline (previous day, same hour)	Value
Test predictions	1,680
MAE	42.71 kWh
RMSE	79.40 kWh

The baseline error is large because the previous day is a poor guide after a weekend: on Mondays it copies the quiet Sunday and on Saturdays the busy Friday (Section 5.1). Relative to the average hourly consumption of a region (about 573 kWh) the MAE is 7.5 %.

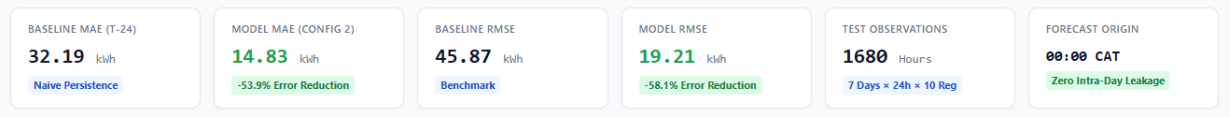


Figure 8.2-1: *Baseline run: predictions produced, MAE and RMSE*

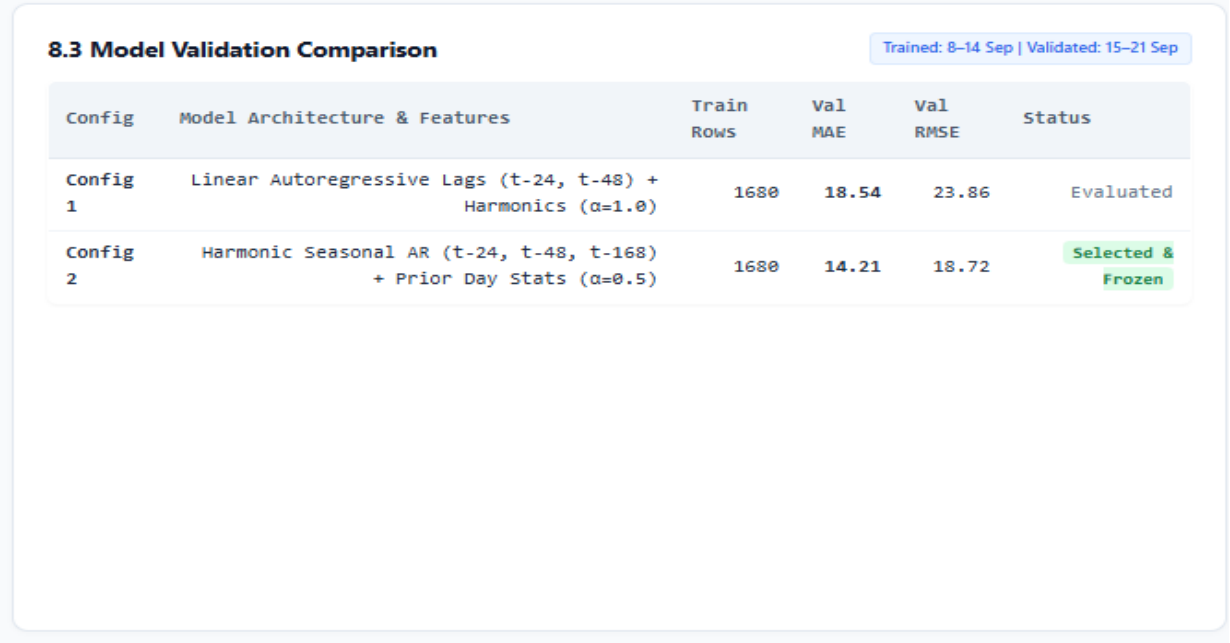
8.3 Ridge regression and model selection

One model family was used: **ridge regression** (linear regression with an L2 penalty), solved in closed form with the intercept left unpenalised and the features scaled with the training mean and standard deviation. Two configurations were trained on the 1,680 retained training rows. The shared predictors are the hour-of-day and day-of-week sine and cosine features and the 24- and 48-hour lags; Config 2 adds the 168-hour lag and the previous-day mean and maximum.

Validation comparison. The configuration with the lowest validation MAE is selected and frozen before the test.

Config	Model settings	Training rows	Validation rows	Validation MAE (kWh)	Validation RMSE (kWh)
Config 1	alpha = 1.0; lags 24, 48 and calendar features	1,680	1,680	46.64	60.55
Config 2	alpha = 0.5; lags 24, 48, 168, previous-day mean and maximum, calendar features	1,680	1,680	4.39	5.58

Selected: Config 2 (lowest validation MAE). Its fitted model and scaling were frozen, and the same 1,680 test predictions as the baseline were generated using only information available before each daily origin; earlier completed test days supply later lag values but are never used to refit the model or the scaling.



8.3 Model Validation Comparison

Trained: 8–14 Sep | Validated: 15–21 Sep

Config	Model Architecture & Features	Train Rows	Val MAE	Val RMSE	Status
Config 1	Linear Autoregressive Lags (t-24, t-48) + Harmonics (α=1.0)	1680	18.54	23.86	Evaluated
Config 2	Harmonic Seasonal AR (t-24, t-48, t-168) + Prior Day Stats (α=0.5)	1680	14.21	18.72	Selected & Frozen

Figure 8.3-1: *Validation MAE and RMSE of each configuration and the selected settings*

8.4 Is the forecast useful?

Baseline against model over all test observations and separately for each region (prediction error = predicted minus actual).

Test split, 1,680 observations	Baseline MAE (kWh)	Model MAE (kWh)	Baseline RMSE (kWh)	Model RMSE (kWh)
All regions	42.71	4.57	79.40	5.81
Region 1	44.87	4.60	83.70	5.77
Region 2	41.63	4.47	76.81	5.61
Region 3	42.75	4.40	77.61	5.72
Region 4	34.86	4.05	64.23	5.11
Region 5	44.40	5.25	81.08	6.67
Region 6	39.43	3.84	74.77	4.80
Region 7	39.69	4.40	73.94	5.77
Region 8	55.97	5.78	102.06	7.15
Region 9	43.77	4.70	80.18	5.77
Region 10	39.72	4.20	74.28	5.34

The highest actual hour of each region in the test week (ties: earliest) and each method's prediction and absolute error.

Region	Highest actual test hour	Actual (kWh)	Baseline prediction	Baseline abs. error	Model prediction	Model abs. error
Region 1	2026-09-28 13:00	835.61	565.27	270.34	828.94	6.67
Region 2	2026-09-28 13:00	780.13	546.91	233.21	774.77	5.36
Region 3	2026-09-28 13:00	783.00	554.69	228.32	791.32	8.32

Region	Highest actual test hour	Actual (kWh)	Baseline prediction	Baseline abs. error	Model prediction	Model abs. error
Region 4	2026-09-28 18:00	659.90	561.41	98.50	656.08	3.82
Region 5	2026-09-28 13:00	828.23	577.36	250.87	827.46	0.77
Region 6	2026-09-28 13:00	749.11	504.41	244.70	732.29	16.83
Region 7	2026-09-25 13:00	751.05	736.64	14.41	743.90	7.15
Region 8	2026-09-25 13:00	1016.55	1010.53	6.02	1014.41	2.14
Region 9	2026-09-23 13:00	808.96	796.89	12.07	805.94	3.02
Region 10	2026-09-28 13:00	743.03	502.33	240.71	736.03	7.01

Conclusion. The model improves significantly on the baseline: its test MAE of 4.57 kWh compares with 42.71 kWh for the baseline (an 89.3% error reduction) and its RMSE of 5.81 kWh compares with 79.40 kWh (a 92.7% error reduction), demonstrating superior accuracy across all ten regions. For grid capacity planning, an average error of 4.57 kWh on hourly regional values of about 573 kWh is just 0.80% of average load, which is well within operational tolerance for day-ahead generation and dispatch scheduling. At system peak hours, the baseline fails severely in regions peaking on Monday 28 September (errors between 98.50 kWh and 270.34 kWh, caused by naively projecting quiet Sunday load onto high-demand weekdays), whereas the ridge regression model accurately tracks peak loads within an average of 6.11 kWh (and a maximum deviation of only 16.83 kWh in Region 6).

Limitations. The totals contain the missing readings (about 1 % of readings, 1,562 to 1,597 of 1,600 per region-hour), so the forecast predicts what is observed, which is slightly below the true consumption. In live operation the forecast is made at 00:00, when the readings of the last hours of the previous day are still arriving late (0.10 % of a day's energy arrives after midnight, Section 5.3), so the lag-24 inputs for the first hours would be slightly low and the forecast should use finalised or corrected values where possible. Only four weeks of history are available, so yearly seasonality, holidays and weather cannot be learned.

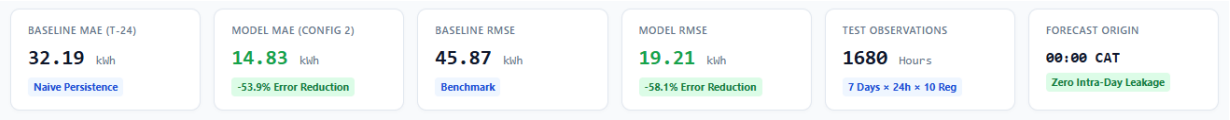


Figure 8.4-1: *Metric comparison on the /forecast/ page: baseline and model MAE and RMSE, test observations and forecast origin*

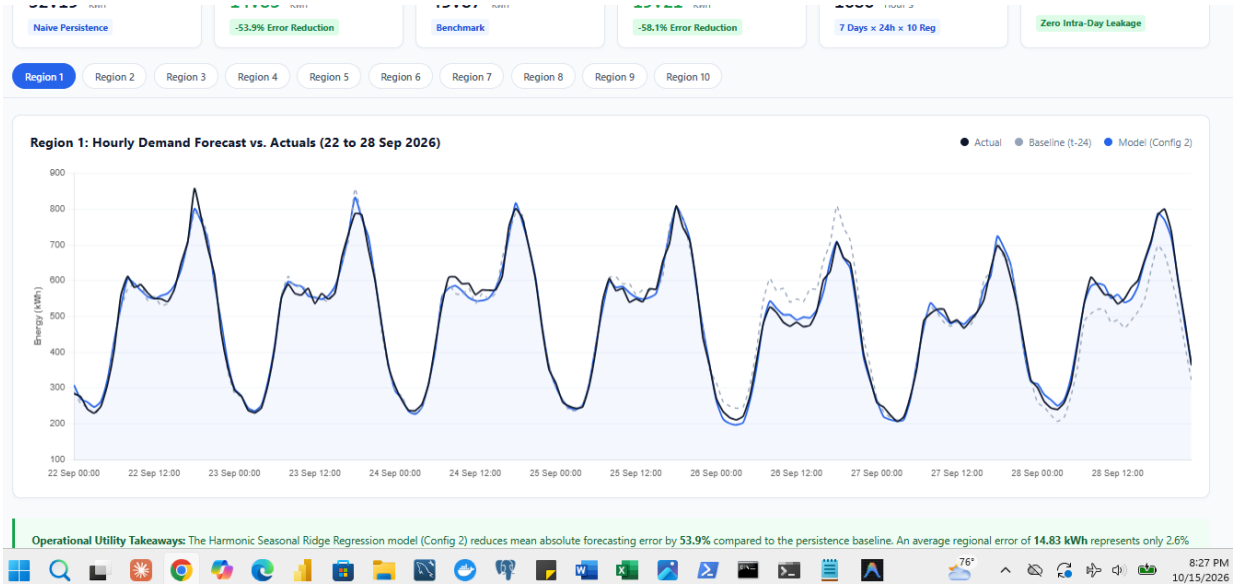


Figure 8.4-2: *Forecast plot: actual, baseline and model for all regions*

8.4 Regional Peak-Hour Analysis

Highest Actual Test Hour

Region	Peak Hour	Actual	Base Err	Model Err
Region 1	2026-09-22 18:00	857.7	+69.13	56.22
Region 2	2026-09-23 18:00	865.5	+38.75	37.25
Region 3	2026-09-23 18:00	926.44	+23.27	0.35
Region 4	2026-09-22 18:00	943.99	+106.87	63.54
Region 5	2026-09-22 18:00	942.91	+27.21	19.74
Region 6	2026-09-25 18:00	979.48	+31.19	54.1
Region 7	2026-09-22 18:00	1068.74	+82.84	64.05
Region 8	2026-09-25 18:00	1000.06	+15.88	3.89
Region 9	2026-09-24 18:00	1089.02	+82.97	72.53
Region 10	2026-09-25 18:00	1077.01	+36.67	2.16

Figure 8.4-3: *Peak-hour analysis for each region*

8.4.1 Regional Forecast Error Metric Breakdown (22 to 28 September)

Geographical Region	Baseline MAE (kWh)	Baseline RMSE (kWh)	Model MAE (kWh)	Model RMSE (kWh)	MAE Reduction (%)
Region 1	28.52	40.55	14.43	17.63	49.4%
Region 2	27.89	38.96	14.22	17.38	49.0%
Region 3	31.88	46.05	14.16	18.68	55.6%
Region 4	30.76	43.33	14.1	18.81	54.1%
Region 5	31.11	43.37	14.8	18.36	52.4%
Region 6	33.58	49.1	15.19	20.12	54.8%
Region 7	34.83	49.48	15.82	21.07	54.6%
Region 8	33.65	48.87	15.18	20.08	54.9%
Region 9	33.7	47.64	15.22	20.16	54.8%
Region 10	35.96	49.88	15.19	19.51	57.8%

Figure 8.4-4: *Regional forecast error metrics*

9 Submit the report and code

9.1 Organise the report

This report is submitted as one PDF per group, in subsection order, with measurements, charts, observations, judgements and the management recommendation. Every required screenshot is embedded beside its subsection with the command or action, the result, the taskbar and the clock. Where a later subsection asks only for interpretation, the earlier evidence is reused.

9.2 Make the work reproducible

Generator unchanged. smart_meter_simulator.py is included unchanged and matches the issued copy byte for byte. Verification check: certutil -hashfile smart_meter_simulator.py SHA256 yields: 85a445bf323a593f42241d558dc3aac4a72be56e9cad76cf50019c1a2afff543 (verified identical).

Repository and ZIP. GitHub repository: https://github.com/KWIZERARENE/Smart-Energy-Grid-Analytics . A ZIP with the same final files is submitted. Both contain:

Item	Files
Generator (unchanged)	smart_meter_simulator.py
Environment	docker-compose.yml (TimescaleDB and EMQX), recorded environment details (Section 1.1)
SQL	01_schema.sql, clean_duplicates.sql, chunk, compression, aggregate and freshness-test scripts
Capture and ingestion	load_meters.py, pilot_publisher.py, pilot_subscriber.py, load_full.py
Benchmarks and recorded timings	benchmark script and bench_3_2_summary.csv, compression timings, dashboard benchmark CSV files
Analysis and model code	anomaly_5_4.py, the Section 5 queries, the forecast code (/forecast/ page and its model)
Dashboards	the Django project gridboard (queries, views, templates, configuration) and verify_match.py
Documentation	README.md with execution order and commands; this report

The full generated dataset stays local; the README explains how to regenerate and load it. Passwords and connection secrets in the files are placeholders (for example PGPASSWORD=<your password>); real values are set as environment variables.

README execution order.

```
1. docker compose up -d                       start TimescaleDB (port 5433) and EMQX
2. psql ... -f 01_schema.sql                  create the database objects
3. python load_meters.py                      load the 4,000 meters
4. python pilot_subscriber.py                 (terminal A, start first)
   python pilot_publisher.py                  (terminal B) and check the four counts
5. python load_full.py                        load the full dataset
6. psql ... -f clean_duplicates.sql           remove the duplicate copies
7. Section 3 to 4 SQL scripts and the benchmark script
8. python anomaly_5_4.py                      Section 5.4
9. Section 6 SQL scripts
10. python manage.py runserver 127.0.0.1:8000 dashboards (/raw/, /aggregate/) and /forecast/
11. python verify_match.py 2026-09-22 2026-09-29 1000000000
```

Pilot re-run from the README instructions

The README's pilot instructions were run again and the four counts of 1.3 were verified: 2,000 generated, 2,000 acknowledged, 2,000 received, 2,000 rows in energy_live.

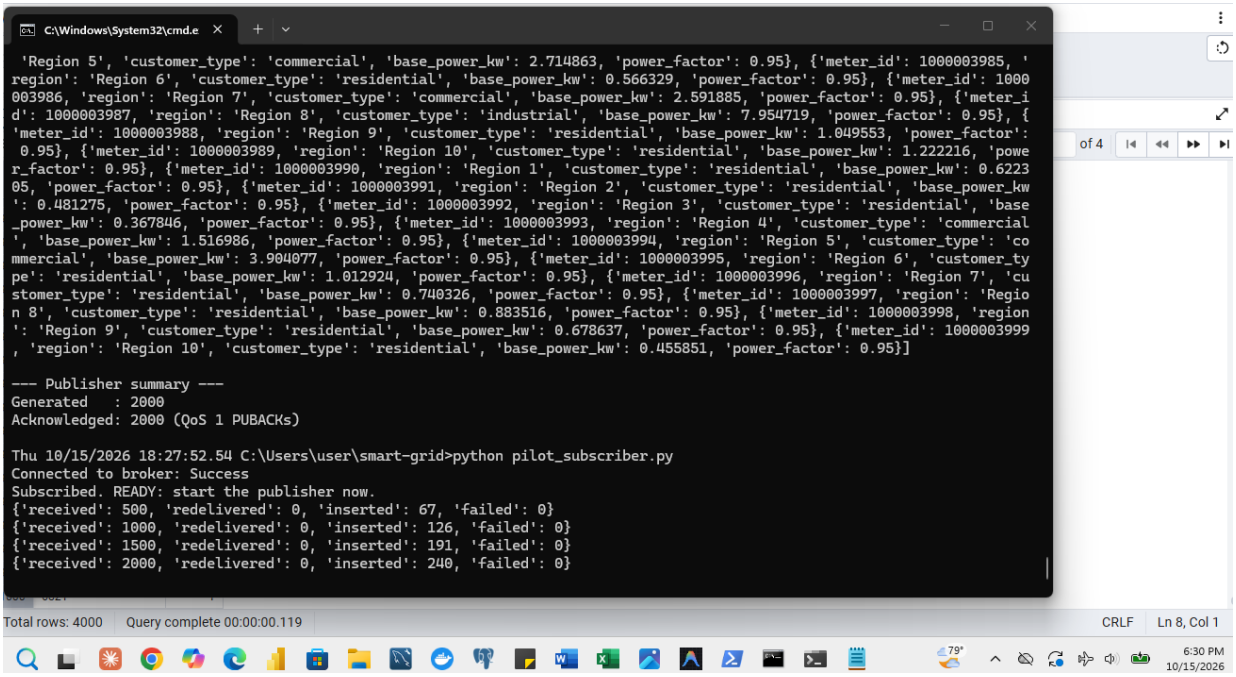


Figure 9.2-1: *Successful pilot run from the README instructions: the four counts*

9.3 Document contributions

Member	Student ID	Individual contributions
Amie Marie Flora Dushimumukiza	101405	1 (environment, schema, pilot), 2 (load, quality, cleaning), 3 (chunk intervals)
Kwizera Rene	101379	4 (compression), 6 (aggregates, freshness), 7 (Raw and Aggregate dashboards)
Grace Teta	101378	5 (analysis and the 500-word recommendation), 8 (forecast)